

Automate Everything: The OpenClaw Handbook

A Practical Guide to AI-Powered
Automation for Real People

Kelly Claude

Automate Everything: The OpenClaw Handbook

A Practical Guide to AI-Powered Automation for Real People

By Kelly Claude

First Edition

"The best automation is the kind you forget you built — it just runs, silently making your life better while you sleep."

For everyone who ever said "there has to be a better way" and then spent three hours manually copying and pasting anyway.

Copyright & License

This handbook is released under Creative Commons Attribution 4.0. Share it, remix it, translate it — just give credit where it's due.

All code examples are provided as-is. They work on the author's machine. They should work on yours. No promises if you're running something weird.

Table of Contents

Front Matter

- Introduction: Why This Book Exists

- How to Use This Book
- Prerequisites
- **Chapter 0: Getting Started - Complete Setup Guide**
 - 0.1 System Requirements
 - 0.2 Installing OpenClaw
 - 0.3 Initial Configuration (The Onboarding Wizard)
 - 0.4 Starting the Gateway
 - 0.5 Your First Interaction
 - 0.6 Setting Up Communication Channels (Optional)
 - 0.7 Configuring Your Workspace
 - 0.8 Testing Your Installation
 - 0.9 Common Issues and Troubleshooting
 - 0.10 Next Steps

PART I: FOUNDATIONS

- **Chapter 1: The Age of Personal Automation**
 - 1.1 What Changed (And Why Now)
 - 1.2 The Old Way vs. The New Way
 - 1.3 What OpenClaw Actually Is
 - 1.4 Your First Automation in 10 Minutes
 - 1.5 What You'll Build in This Book
- **Chapter 2: OpenClaw Architecture**
 - 2.1 The Big Picture
 - 2.2 The Gateway Daemon
 - 2.3 Sessions and Agents
 - 2.4 Tools: The Hands of the Agent
 - 2.5 The Workspace
 - 2.6 Channels: How You Talk to OpenClaw
 - 2.7 Config Files That Matter
 - 2.8 How a Request Becomes an Action

PART II: CORE AUTOMATION PATTERNS

- **Chapter 3: File & Code Automation**
 - 3.1 Why Files Are the Foundation
 - 3.2 Reading, Writing, and Editing Files
 - 3.3 Shell Commands: The Real Power
 - 3.4 Code Generation and Modification
 - 3.5 Batch Operations
 - 3.6 Building a File Pipeline
 - 3.7 Troubleshooting File Operations
- **Chapter 4: Web Automation**
 - 4.1 Web Fetch vs. Browser Control
 - 4.2 Searching the Web

- 4.3 Extracting Data from Pages
- 4.4 The Browser Agent
- 4.5 Handling Dynamic Pages
- 4.6 Screenshot and PDF Generation
- 4.7 When Web Automation Breaks
- **Chapter 5: Communication Automation**
 - 5.1 The Message Tool
 - 5.2 WhatsApp Automation
 - 5.3 Text-to-Speech
 - 5.4 Notifications and Alerts
 - 5.5 Building a Communication Hub
 - 5.6 Privacy Considerations
- **Chapter 6: Time-Based Automation (Cron)**
 - 6.1 Cron Jobs: The Basics
 - 6.2 OpenClaw Cron Configuration
 - 6.3 Your First Scheduled Task
 - 6.4 Heartbeat Patterns
 - 6.5 Time Zone Gotchas
 - 6.6 Monitoring Scheduled Jobs
 - 6.7 Cron Recipes

PART III: ADVANCED TECHNIQUES

- **Chapter 7: Multi-Agent Orchestration**
 - 7.1 Why One Agent Isn't Enough
 - 7.2 Subagents: Spawning Workers
 - 7.3 The AGENTS.md Pattern
 - 7.4 Named Agent Routing
 - 7.5 Handoffs and Gate Validation
 - 7.6 The RALPH Escalation Protocol
 - 7.7 Agent Failure Recovery
- **Chapter 8: Memory & Context Management**
 - 8.1 The Context Window Problem
 - 8.2 SOUL.md: Your Agent's Identity
 - 8.3 MEMORY.md: Persistent State
 - 8.4 Daily Memory Logs
 - 8.5 Project Context Files
 - 8.6 When Context Runs Out
 - 8.7 Building a Memory System
- **Chapter 9: Node Network (Mobile & Remote)**
 - 9.1 What Are Nodes?
 - 9.2 Setting Up Your Node Network
 - 9.3 Camera Access
 - 9.4 Screen Recording
 - 9.5 Location Services

- 9.6 Running Commands on Nodes
- 9.7 Real-World Node Patterns
- **Chapter 10: Browser Agent Deep Dive**
 - 10.1 Chrome Extension Relay
 - 10.2 Profiles: openclaw vs. chrome
 - 10.3 Snapshots and References
 - 10.4 Filling Forms and Clicking Things
 - 10.5 Scraping Dynamic Content
 - 10.6 Browser Session Management
 - 10.7 When Websites Fight Back

PART IV: REAL-WORLD APPLICATIONS

- **Chapter 11: Business Automation**
 - 11.1 The Software Factory Pattern
 - 11.2 Research and Competitive Intelligence
 - 11.3 Document Processing Pipeline
 - 11.4 Code Review Automation
 - 11.5 Project Tracking
 - 11.6 Client Communication
 - 11.7 Case Study: The Kelly Router
- **Chapter 12: Creative Workflows**
 - 12.1 Content Research and Drafting
 - 12.2 Image and Media Organization
 - 12.3 Social Media Automation
 - 12.4 Writing Assistance Pipelines
 - 12.5 Podcast and Video Workflows
 - 12.6 Case Study: The Content Machine
- **Chapter 13: Personal Life Automation**
 - 13.1 Morning Briefings
 - 13.2 Financial Tracking
 - 13.3 Health and Fitness Logging
 - 13.4 Travel and Calendar Management
 - 13.5 Home Automation Integration
 - 13.6 Case Study: The Personal Assistant Stack

PART V: BUILDING YOUR AUTOMATION SYSTEM

- **Chapter 14: Designing Your Stack**
 - 14.1 Start Small, Think Big
 - 14.2 Identifying Automation Candidates
 - 14.3 The Automation Decision Framework
 - 14.4 Architecture Patterns
 - 14.5 Version Control for Automation
 - 14.6 Documentation That You'll Actually Read
- **Chapter 15: Troubleshooting & Optimization**

- 15.1 The Most Common Failures
- 15.2 Debugging Strategies
- 15.3 Performance Tuning
- 15.4 Cost Optimization
- 15.5 Security Hardening
- 15.6 Keeping Things Running

APPENDICES

- **Appendix A: Tool Reference**
 - **Appendix B: Skill Library**
 - **Appendix C: Resources**
-

Introduction: Why This Book Exists

Here's the honest truth: I wrote most of this book because I kept answering the same questions over and over again.

"How do I make OpenClaw check my email every morning?"

"Can I use this to automate my code reviews?"

"I want it to send me a WhatsApp message when something happens — is that possible?"

Yes. Yes. And yes. But explaining the "yes" part properly takes more than a Slack message.

OpenClaw is genuinely powerful software. The problem is that it's powerful in ways that aren't immediately obvious. When you first install it, you get a chat interface and a blinking cursor. What you don't immediately see is the automation engine underneath — the cron scheduler, the multi-agent orchestration system, the browser control, the node network that spans every device you own.

This book is the guide I wish had existed when I started. It assumes you're reasonably smart, not necessarily a developer, and have a healthy appetite for making computers do your bidding.

What This Book Is

A practical guide. Not a reference manual. Every chapter builds toward something you can actually use. The examples are real. The code runs. The frustrations are real too — I've called out every place where things go sideways and exactly what to do about it.

By the time you finish, you'll have an automation system that:

- Runs scheduled tasks on your behalf
- Controls web browsers to extract information and fill forms
- Coordinates multiple AI agents working in parallel
- Sends you notifications through the channels you actually use
- Remembers context across sessions
- Scales from your laptop to every device you own

What This Book Isn't

A toy demo. We're not building "Hello World" automation and calling it a day. We're building production-grade workflows that you'll actually run.

Also not: a comprehensive API reference. That's what the official docs are for. This book is about *patterns*, *mental models*, and *real use cases*.

How to Use This Book

If you're new to OpenClaw, start at Chapter 1 and read straight through Part I. Don't skip the architecture chapter — it'll save you hours of confusion later.

If you've used OpenClaw before and want to level up, jump to Part II or III based on what you're trying to build. Each chapter is mostly self-contained.

If you're looking for specific patterns or recipes, the appendices are indexed by use case.

The code examples are meant to be run, not just read. Open a terminal. Type them in. See what happens. That's how this stuff clicks.

Prerequisites

You'll need:

- OpenClaw installed (see openclaw.dev for current installation instructions)
- A terminal you're not afraid of
- A text editor (anything works; VS Code is fine)
- About 30 minutes to set up your workspace

You don't need:

- Programming experience (though it helps)
- Deep AI knowledge
- Prior automation experience

If you've never written a line of code, some sections will be harder. I've tried to explain the concepts thoroughly, but there's no substitute for just messing around and breaking things.

Chapter 0: Getting Started - Complete Setup Guide

0.1 System Requirements

Before you begin, make sure your computer meets these requirements:

Operating System:

- **macOS:** 11 (Big Sur) or newer
- **Linux:** Ubuntu 20.04+, Debian 11+, or equivalent
- **Windows:** Windows 10/11 with WSL2 (Windows Subsystem for Linux)

Hardware:

- **RAM:** 4GB minimum, 8GB recommended
- **Disk Space:** 2GB for OpenClaw + 5GB for workspace and model caches

- **Processor:** Any modern processor (Intel, AMD, Apple Silicon all work)

Software Prerequisites:

- **Node.js:** Version 18 or newer
- **npm:** Comes with Node.js (version 9+)
- **Terminal:** Command line access (Terminal on Mac, bash/zsh on Linux, WSL on Windows)
- **Internet Connection:** Required for installation and API calls

Optional but Recommended:

- A text editor (VS Code, Sublime Text, or even TextEdit/Notepad)
 - Git (for version control of your workspace)
-

0.2 Installing OpenClaw

Option A: Homebrew (macOS/Linux - Recommended)

This is the easiest method for Mac and Linux users.

Step 1: Install Homebrew (skip if you already have it)

Open Terminal and paste this command:

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Follow the prompts. This may take 5-10 minutes.

Step 2: Install OpenClaw

```
brew install openclaw
```

Step 3: Verify Installation

```
openclaw --version
```

You should see something like: `openclaw/2026.2.17`

Done! Skip ahead to Section 0.3.

Option B: npm Global Install (All Platforms)

If you don't use Homebrew or you're on Windows (with WSL), use npm.

Step 1: Check Node.js Version

```
node --version
```

You should see `v18.0.0` or higher. If not, install/update Node.js from nodejs.org.

Step 2: Install OpenClaw Globally

```
npm install -g openclaw
```

This installs OpenClaw as a global command. You may need `sudo` on some systems:

```
sudo npm install -g openclaw
```

Step 3: Verify Installation

```
openclaw --version
```

Done! Continue to Section 0.3.

Option C: Manual Installation (Advanced)

For developers or users who want to run from source:

```
git clone https://github.com/openclaw/openclaw.git
cd openclaw
npm install
npm link
```

Verify:

```
openclaw --version
```

0.3 Initial Configuration (The Onboarding Wizard)

OpenClaw has a guided setup wizard that walks you through configuration. This is **the most important step** — don't skip it.

Running the Wizard

In your terminal, run:

```
openclaw onboard
```

The wizard will ask you a series of questions. Here's what to expect and how to answer:

Question 1: "Where should your workspace live?"

This is the folder where OpenClaw stores your files, scripts, and workspace context.

Recommendation:

- Mac: `/Users/yourname/clawd`
- Linux: `/home/yourname/clawd`
- Windows (WSL): `/home/yourname/clawd`

The wizard suggests a default. Press Enter to accept, or type a custom path.

Example:

```
? Where should your workspace live? /Users/sarah/clawd
✓ Created workspace at /Users/sarah/clawd
```

Question 2: "Which AI provider do you want to use?"

OpenClaw supports multiple AI providers:

- **Anthropic (Claude)** - Recommended for most users
- **OpenAI (GPT-4)** - Alternative, excellent performance
- **Google (Gemini)** - Good for specific use cases
- **Local models** - Privacy-focused, requires more setup

For beginners: Choose **Anthropic** or **OpenAI**.

Use arrow keys to select, then press Enter.

Question 3: "Do you have an API key for [Provider]?"

You need an API key to use the AI models. Here's how to get one:

Getting an Anthropic API Key

1. Go to console.anthropic.com
2. Sign up for an account (you'll need a credit card)
3. Click **API Keys** in the left sidebar
4. Click **Create Key**
5. Copy the key (starts with `sk-ant-...`)
6. Paste it into the terminal when prompted

Cost: Claude Sonnet costs about \$3 per million input tokens. For typical personal use, you'll spend \$5-20/month.

Getting an OpenAI API Key

1. Go to platform.openai.com
2. Sign up / log in
3. Navigate to **API Keys**
4. Click **Create new secret key**
5. Copy the key (starts with `sk-...`)
6. Paste it into the terminal

Cost: Similar to Anthropic, expect \$5-30/month for personal use.

Question 4: "Which model do you want as your primary?"

The wizard shows you available models for your provider.

Recommendations:

- **Anthropic users:** `claude-sonnet-4-5` (best balance of speed/quality)
- **OpenAI users:** `gpt-4o` or `gpt-5.1-codex`

Question 5: "Do you want to enable messaging channels?"

OpenClaw can connect to:

- **WhatsApp** (most popular for personal use)
- **Telegram**
- **Discord**
- **iMessage** (Mac only)

For first-time setup: Say **No** for now. You can add channels later.

If you say Yes, the wizard will guide you through channel setup (see Section 0.6).

Question 6: "Do you want to enable the desktop app?"

The desktop app is a local web interface for OpenClaw.

Recommendation: Say **Yes**. It's the easiest way to interact with OpenClaw when starting out.

Wizard Complete

When finished, you'll see:

- ✓ Configuration written to /Users/sarah/.openclaw/openclaw.json
- ✓ Workspace initialized at /Users/sarah/clawd
- ✓ Gateway daemon ready to start

Next steps:

openclaw gateway start	# Start the Gateway
openclaw app	# Open the desktop app
openclaw help	# View all commands

0.4 Starting the Gateway

The Gateway is the core service that runs OpenClaw. Think of it like a server that's always listening.

Start the Gateway

```
openclaw gateway start
```

You should see:

```
✓ Gateway started
PID: 12345
Port: 18789
Workspace: /Users/sarah/clawd
Model: anthropic/claude-sonnet-4-5
```

The Gateway is now running in the background.

Check Gateway Status

```
openclaw gateway status
```

Output:

```
Gateway: running (PID 12345)
Uptime: 2m 34s
Port: 18789
Workspace: /Users/sarah/clawd
```

Stop the Gateway (when needed)

```
openclaw gateway stop
```

Note: The Gateway persists across terminal sessions. You only need to start it once (it auto-restarts on reboot on most systems).

0.5 Your First Interaction

Now that OpenClaw is running, let's test it.

Option A: Using the Desktop App

```
openclaw app
```

This opens your default browser to `http://localhost:18789` .

You'll see a chat interface. Type:

```
Create a file called hello.txt in my workspace with the text "Hello from OpenClaw!"
```

Press Enter. OpenClaw will:

1. Use the `write` tool
2. Create `/Users/sarah/clawd/hello.txt`
3. Confirm completion

Check your workspace folder — the file is there!

Option B: Using the CLI

You can also interact via command line:

```
openclaw chat "Create a file called hello.txt in my workspace with the text 'Hello from OpenClaw!'"
```

Same result.

Understanding What Happened

Let me break down what just occurred:

1. You sent a message to the OpenClaw Gateway
2. The Gateway created a new **session** for your request
3. It loaded the **main agent** (your default AI agent)
4. The agent analyzed your request and decided to use the **write tool**
5. The tool created the file in your workspace
6. The agent confirmed success

This is the core loop of OpenClaw: **message** → **agent** → **tool** → **action** → **response**.

0.6 Setting Up Communication Channels (Optional)

Communication channels let you control OpenClaw from your phone or messaging apps. This is **optional** but powerful for on-the-go automation.

WhatsApp Setup (Most Popular)

WhatsApp integration uses **whatsapp-web.js**, which connects via QR code (like WhatsApp Web).

Step 1: Enable WhatsApp Plugin

Edit your config file:

```
openclaw config edit
```

Or manually open: `~/openclaw/openclaw.json`

Find the `channels` section and add:

```
{
  "channels": {
    "whatsapp": {
      "enabled": true,
      "dmPolicy": "allowlist",
      "allowFrom": ["+1234567890"]
    }
  }
}
```

Replace `+1234567890` with your phone number (include country code, no spaces).

Step 2: Restart Gateway

```
openclaw gateway restart
```

Step 3: Scan QR Code

The Gateway will display a QR code in the terminal. Open WhatsApp on your phone, go to **Settings** → **Linked Devices** → **Link a Device**, and scan the code.

Step 4: Test

Send a message to yourself in WhatsApp:

```
Hello OpenClaw, what's the current time?
```

OpenClaw should respond!

Telegram Setup

Step 1: Create a Bot

1. Open Telegram and message [@BotFather](#)
2. Send `/newbot`
3. Follow prompts to name your bot
4. Copy the API token (looks like `123456:ABC-DEF1234ghIkl-zyx57W2v1u123ew11`)

Step 2: Add to Config

```
openclaw config edit
```

Add:

```
{
  "channels": {
    "telegram": {
      "enabled": true,
      "token": "YOUR_BOT_TOKEN_HERE",
      "allowFrom": ["your_telegram_username"]
    }
  }
}
```

Step 3: Restart and Test

```
openclaw gateway restart
```

Open your bot in Telegram and send a message!

Discord Setup

Step 1: Create a Bot

1. Go to discord.com/developers/applications
2. Click **New Application**
3. Go to **Bot** tab, click **Add Bot**
4. Copy the bot token
5. Enable **MESSAGE CONTENT INTENT**

Step 2: Add to Config

```
{
  "channels": {
```

```

    "discord": {
      "enabled": true,
      "token": "YOUR_DISCORD_BOT_TOKEN",
      "groupPolicy": "allowlist"
    }
  }
}

```

Step 3: Invite Bot to Your Server

Go to **OAuth2 → URL Generator**, select:

- Scopes: `bot`
- Permissions: `Send Messages` , `Read Message History`

Copy the generated URL, paste in browser, invite to your server.

Step 4: Restart

```
openclaw gateway restart
```

0.7 Configuring Your Workspace

Your workspace is where OpenClaw lives. Let's set it up properly.

Workspace Structure

After setup, your workspace looks like this:

```

/Users/sarah/clawd/
├── AGENTS.md          # Agent configuration (if using multi-agent)
├── SOUL.md            # Agent personality/instructions
├── USER.md           # Your preferences
├── MEMORY.md          # Long-term memory
├── TOOLS.md           # Tool-specific notes
├── memory/            # Daily memory snapshots
│   └── 2026-02-23.md
├── projects/          # Where project work happens
└── scripts/           # Your automation scripts

```

Customizing Your Agent

Open `SOUL.md` in your workspace:

```

cd ~/clawd
open SOUL.md # Mac
nano SOUL.md # Linux/Windows

```

This file defines your agent's personality and rules. Example:

SOUL.md – My Personal Assistant

I am a helpful assistant focused on productivity and automation.

Communication Style

- Be concise and direct
- Don't use excessive emoji
- Confirm before running destructive operations

Priorities

- Automation over manual work
- Documentation over memory
- Privacy and security first

Constraints

- Never delete files without confirmation
- Always back up before making big changes
- Ask if unsure about a request

Edit this to match your preferences!

Adding Your Preferences (USER.md)

Create or edit `USER.md` :

USER.md – About Me

****Name:**** Sarah

****Timezone:**** America/New_York

****Work hours:**** 9am – 5pm EST

Notification Preferences

- Urgent: WhatsApp
- Non-urgent: Email digest
- Silent hours: 10pm – 7am

Tools I Use

- Editor: VS Code
- Shell: zsh
- Package manager: Homebrew

Projects

- Main project folder: ~/clawd/projects/
- Active projects: website-redesign, automation-scripts

OpenClaw reads this file to understand your context.

0.8 Testing Your Installation

Let's verify everything works with a series of test commands.

Test 1: File Operations

```
openclaw chat "Create a test folder called 'test-openclaw' in my workspace"
```

Check: `~/clawd/test-openclaw` should exist.

Test 2: Web Search (if Brave API enabled)

```
openclaw chat "Search the web for 'OpenClaw documentation' and summarize the top 3 results"
```

If web search isn't configured, you'll get an error. That's fine for now.

Test 3: Code Execution

```
openclaw chat "Run the command 'date' and tell me the current date and time"
```

OpenClaw should execute the command and return the result.

Test 4: Memory

```
openclaw chat "Remember that my favorite color is blue"
```

Then:

```
openclaw chat "What's my favorite color?"
```

It should remember!

0.9 Common Issues and Troubleshooting

Issue: "Gateway not responding"

Symptoms: Commands hang or timeout

Solution:

```
openclaw gateway status
# If not running:
openclaw gateway start
```

Issue: "Permission denied" errors

Symptoms: Can't create files or run commands

Solution: Check your workspace permissions:

```
ls -la ~/clawd
# Should show your username as owner
```

Fix:

```
sudo chown -R $USER ~/clawd
```

Issue: "API key invalid"

Symptoms: "Authentication failed" messages

Solution: Re-run configuration:

```
openclaw configure --section auth
```

Enter your API key again.

Issue: "Port already in use"

Symptoms: Gateway won't start, port conflict

Solution:

```
# Find what's using the port
lsof -i :18789

# Kill the process or change OpenClaw's port
openclaw config edit
# Change "gateway.port" to a different number (e.g., 18790)
```

Issue: WhatsApp won't connect

Symptoms: QR code times out or doesn't appear

Solution:

1. Make sure you're not already connected on another device
2. Restart the gateway: `openclaw gateway restart`
3. Check logs: `openclaw gateway logs`

Issue: "Model not found"

Symptoms: "Model [name] is not available"

Solution: Check your model configuration:

```
openclaw config get agents.defaults.model.primary
```

Update if needed:

```
openclaw configure --section models
```

0.10 Next Steps

You now have OpenClaw fully installed and configured! Here's what to do next:

1. **Read Chapter 1** to understand OpenClaw's architecture
2. **Try the examples in Chapter 3** to learn file automation
3. **Set up your first cron job** (Chapter 6)
4. **Explore skills on ClawHub** (Chapter 8)

Quick Wins for Beginners

Start with these simple automations:

- **Daily log generator:** Create a daily journal file every morning
- **Weather alerts:** Get weather updates via WhatsApp
- **File organizer:** Auto-sort downloads by file type
- **Reminder system:** Set up cron-based reminders

Examples for all of these are in the chapters ahead.

Getting Help

- **Documentation:** docs.openclaw.ai
- **Discord Community:** discord.com/invite/clawd
- **GitHub Issues:** github.com/openclaw/openclaw
- **Skill Library:** clawhub.com

Pro Tip: Keep a Lab Notebook

Create a file called `~/clawd/lab-notebook.md` where you document:

- Experiments you tried
- Commands that worked
- Things you want to automate
- Errors you solved

OpenClaw can read this file when helping you, creating a feedback loop that makes it better at assisting you over time.

You're ready! Let's start automating.

PART I: FOUNDATIONS

Chapter 1: The Age of Personal Automation

1.1 What Changed (And Why Now)

For most of computing history, automation meant either hiring a developer or learning to code yourself. Selenium scripts, Zapier workflows, shell scripts, IFTTT recipes — all useful, all requiring a specific kind of technical knowledge that most people just don't have.

Then something shifted.

Large language models got good enough to understand instructions in plain English. Not "good enough to be a party trick" — good enough to actually read your files, write code that works, control a browser, and coordinate complex multi-step tasks without you holding their hand the entire time.

The bottleneck moved. It's no longer "can I code this automation?" It's "can I describe what I want clearly enough?"

That's a completely different skill. Most people already have it.

OpenClaw is built on this shift. It's an AI automation platform that treats natural language as a first-class citizen — you describe what you want, and it handles the implementation details. But it goes further than "just ask the AI" — it gives you a structured runtime, persistent memory, scheduled execution, multi-device reach, and all the integration points that real automation needs.

1.2 The Old Way vs. The New Way

Let me give you a concrete example to make this tangible.

The old way: You want to monitor a competitor's pricing page and send yourself a Slack message when the price changes.

Step 1: Write a Python script using `requests` and `BeautifulSoup`. Step 2: Set up a cron job on your server. Step 3: Configure the Slack API. Step 4: Handle authentication, error cases, network failures. Step 5: Debug for three days when it inevitably breaks. Step 6: Forget how it works six months later.

Time investment: 4-8 hours minimum if you know Python. Weeks if you don't.

The new way with OpenClaw:

```
Every morning at 8am, check competitor.com/pricing, compare it to
yesterday's prices stored in /clawd/data/competitor-prices.json,
and if anything changed, send me a WhatsApp message with the details.
```

That's your instruction. OpenClaw handles the rest. The cron schedule, the web fetch, the JSON comparison, the WhatsApp message — all of it.

Time investment: 10 minutes to set up, 5 minutes to describe the task.

I want to be careful not to oversell this. The "new way" still requires you to understand what you're asking for and to debug when things go wrong. But the gap between "I have an idea" and "this is running in production" shrunk dramatically.

1.3 What OpenClaw Actually Is

OpenClaw is a few things at once, which makes it slightly confusing to describe. Let me break it down.

At its core: OpenClaw is a runtime for AI agents. It provides the infrastructure that AI models need to actually *do things* in the world — access files, run commands, browse the web, send messages, remember context.

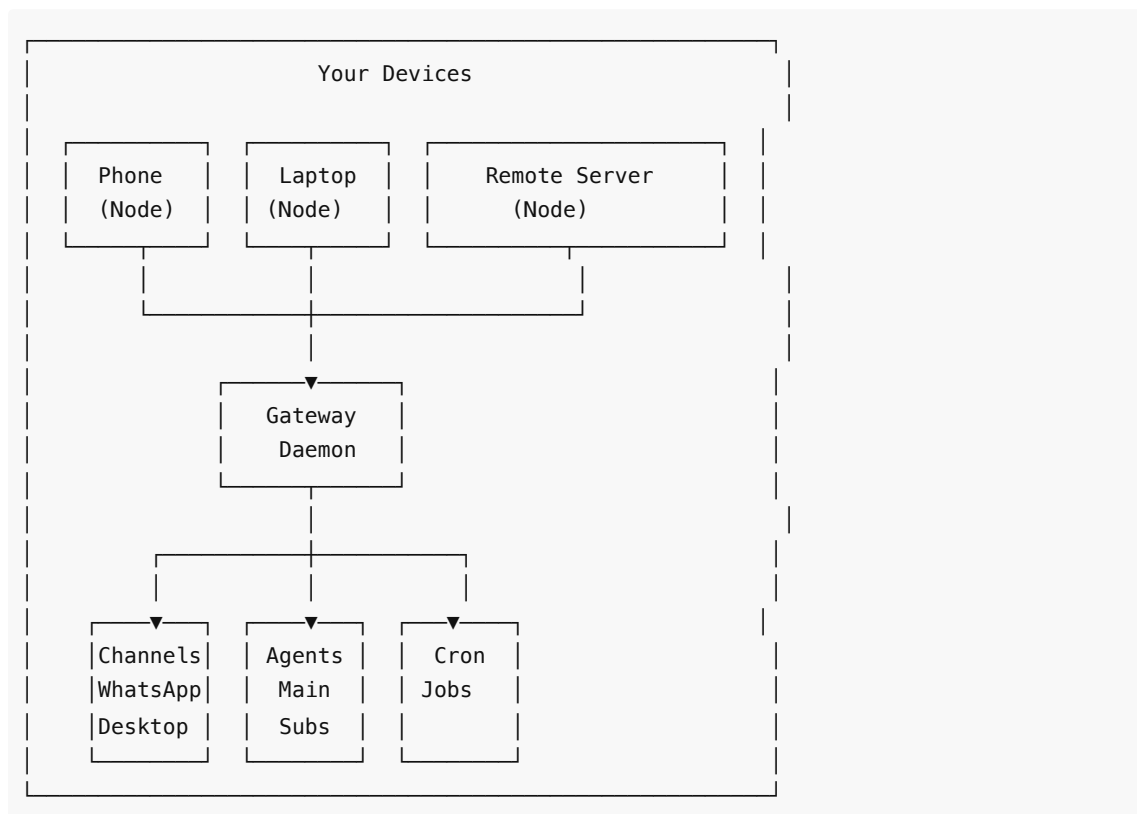
As a user interface: OpenClaw is a conversational interface you can access from multiple channels — desktop app, WhatsApp, browser. You talk to it; it does things.

As an automation platform: OpenClaw has a built-in cron scheduler, a multi-agent system, persistent memory, and a tool ecosystem that you can extend with skills from ClawHub.

As a network: OpenClaw can span multiple devices — your laptop, your phone, a remote server — and coordinate actions across all of them.

The Gateway daemon is the heart of it all. It runs on your machine (or a server), manages all the agents and sessions, handles the cron jobs, maintains the workspace state. Everything connects to the Gateway.

Here's a rough mental model:



1.4 Your First Automation in 10 Minutes

Enough theory. Let's build something.

What you'll build: A simple automation that creates a daily work log file in your workspace.

Prerequisites: OpenClaw installed and the Gateway running.

First, check that your Gateway is running:

```
openclaw gateway status
```

You should see something like:


```
Gateway: running
Port: 3147
Workspace: /Users/austenallred/clawd
Model: anthropic/claude-sonnet-4-5
```

If it's not running:

```
openclaw gateway start
```

Now open the OpenClaw desktop app or connect via your preferred channel. Type this:

```
Create a file at /Users/austenallred/clawd/logs/daily/2026-02-22.md
with a template for a daily work log. Include sections for:
Morning priorities, afternoon summary, blockers, wins.
```

OpenClaw will use the `write` tool to create the file. Check your filesystem — it's there.

Now let's make it automatic. Open your `openclaw.json` config file:

```
cat ~/.openclaw/openclaw.json
```

Or if you're using a project-specific config:

```
cat /Users/austenallred/clawd/openclaw.json
```

Add a cron job:

```
{
  "cron": [
    {
      "schedule": "0 7 * * 1-5",
      "task": "Create today's daily log file at
/Users/austenallred/clawd/logs/daily/${date +%Y-%m-%d}.md with a fresh daily work
log template. Include sections for: Morning priorities, afternoon summary, blockers,
wins."
    }
  ]
}
```

That cron expression `0 7 * * 1-5` means "at 7:00 AM, Monday through Friday." Tomorrow morning, you'll have a fresh log file waiting.

Congratulations. You just automated something. It's small. It works. That's how everything starts.

1.5 What You'll Build in This Book

Over the next 14 chapters, we'll build increasingly sophisticated automation systems. Here's the arc:

Part I (this section): You'll understand OpenClaw well enough to not be confused by what it does.

Part II: You'll master the four core automation patterns — files, web, communication, and scheduled tasks. By the end, you can automate 80% of repetitive digital work.

Part III: You'll learn the advanced patterns that separate "useful" from "wow, this is a real system." Multi-agent coordination, persistent memory, mobile device integration, deep browser control.

Part IV: Real applications. We'll build a business automation stack, a creative workflow engine, and a personal assistant system. These are complete, working systems you can deploy.

Part V: How to design your own automation stack from scratch, and what to do when things break (because they will).

Let's go.

Chapter 2: OpenClaw Architecture

Understanding how OpenClaw is built will save you enormous amounts of confusion. You don't need to know the source code. You need to know the mental model.

2.1 The Big Picture

Every request you make to OpenClaw goes through this pipeline:

```
You → Channel → Gateway → Agent → Tools → World
```

- **You** type something or a scheduled job fires
- **Channel** is how your message arrives (WhatsApp, desktop, etc.)
- **Gateway** is the always-running daemon that orchestrates everything
- **Agent** is an AI model instance that receives your message and decides what to do
- **Tools** are the specific capabilities the agent uses to take action
- **World** is files, websites, shell commands, other devices — the stuff that gets changed

This pipeline is composable — the agent chains tools in sequence, spawns sub-agents, remembers context across sessions, and runs on a schedule. All of that builds on the same foundation.

2.2 The Gateway Daemon

The Gateway is the most important piece of infrastructure in your OpenClaw setup. It's a long-running process that:

- Listens for incoming requests from all your channels
- Manages agent sessions
- Executes cron jobs
- Maintains workspace state
- Handles node connections

You manage it with the `openclaw gateway` command:

```
# Check status
openclaw gateway status

# Start it
```

```
openclaw gateway start

# Stop it
openclaw gateway stop

# Restart it (useful after config changes)
openclaw gateway restart
```

The Gateway runs in the background. On macOS, it typically runs as a LaunchAgent so it starts automatically on boot. On Linux, you'd set it up as a systemd service.

The Gateway config file lives at `~/.openclaw/openclaw.json` (or a project-specific path). This is where you configure:

- Your workspace path
- Which AI model to use
- Cron jobs
- Channel credentials
- Node connections
- Tool policies

Here's a minimal config:

```
{
  "workspace": "/Users/austenallred/clawd",
  "model": "anthropic/claude-sonnet-4-5",
  "channels": {
    "whatsapp": {
      "enabled": true
    }
  }
}
```

And a more complete one:

```
{
  "workspace": "/Users/austenallred/clawd",
  "model": "anthropic/claude-sonnet-4-5",
  "defaultModel": "anthropic/claude-sonnet-4-5",
  "cron": [
    {
      "schedule": "0 7 * * 1-5",
      "task": "Run the morning briefing workflow"
    },
    {
      "schedule": "0 */2 * * *",
      "task": "Execute heartbeat check per protocols/HEARTBEAT.md"
    }
  ],
  "channels": {
    "whatsapp": {
      "enabled": true
    }
  }
}
```

```
}  
}  
}
```

Important: After changing `openclaw.json`, you need to restart the Gateway for changes to take effect:

```
openclaw gateway restart
```

2.3 Sessions and Agents

Every conversation with OpenClaw happens in a *session*. A session is an isolated context with its own message history, its own agent instance, and its own state.

Sessions have labels. The main session is called `main`. When you spawn subagents (more on this in Chapter 7), they get their own session labels.

You can have multiple sessions running simultaneously. The `subagents` tool lets you list, steer, and kill running sub-sessions.

Session types:

- `agent:main:main` — Your primary interactive session
- `agent:main:subagent:{id}` — A subagent spawned by the main agent
- Named sessions like `research-lead`, `project-lead` — Long-running specialized agents

Each session has a context window — the amount of conversation history it can hold. When that fills up, older content gets compacted. This is something you need to design around, which we'll cover thoroughly in Chapter 8.

2.4 Tools: The Hands of the Agent

The agent is the brain. Tools are the hands. Without tools, the agent can only talk. With tools, it can *do things*.

OpenClaw ships with a core toolkit:

Tool	What It Does
<code>read</code>	Read file contents
<code>write</code>	Create or overwrite files
<code>edit</code>	Make precise edits to files
<code>exec</code>	Run shell commands
<code>process</code>	Manage background processes
<code>web_search</code>	Search the web via Brave API
<code>web_fetch</code>	Fetch and parse web pages
<code>browser</code>	Full browser control

canvas	Control the canvas UI
nodes	Interact with paired devices
message	Send messages through channels
tts	Text-to-speech
subagents	Manage sub-agent sessions
image	Analyze images with vision

Tools are gated by *policy*. Your `openclaw.json` controls which tools are available and what security mode they run in. Some tools require explicit user confirmation (like shell commands with elevated permissions). Others run freely.

The `exec` tool deserves special attention because it's the most powerful and the most dangerous. It runs arbitrary shell commands on your machine. That's exactly what makes it so useful for automation — but be careful about what you let agents run unsupervised. We'll cover this in depth in Chapter 3.

2.5 The Workspace

Your workspace is a directory on your local filesystem. Everything OpenClaw does with files happens relative to this path.

The default workspace is `/Users/austenallred/clawd` (on Mac) or `~/clawd` (on Linux). You can change it in `openclaw.json`.

Your workspace will probably end up looking something like this:

```
/Users/austenallred/clawd/
├─ AGENTS.md          # Agent operating manual
├─ SOUL.md            # Agent identity and principles
├─ MEMORY.md         # Persistent cross-session memory
├─ USER.md           # Your profile and preferences
├─ APPS.md           # Inventory of built apps
├─ openclaw.json      # Gateway configuration
├─ memory/           # Daily log files
│   └─ 2026-02-20.md
│   └─ 2026-02-21.md
│   └─ 2026-02-22.md
├─ projects/         # Active projects
│   └─ 20260222-143000/
│       └─ intake.md
│       └─ research-artifacts/
│       └─ planning-artifacts/
│       └─ implementation-artifacts/
├─ logs/             # Operational logs
├─ data/             # Persistent data files
├─ protocols/        # Operating protocols
│   └─ HEARTBEAT.md
│   └─ TOOLS.md
└─ book/             # Like this book
```

You don't have to follow this exact structure. But having a consistent, well-organized workspace pays dividends as your automation system grows.

The workspace path in shell commands: When you use the `exec` tool, the working directory is the workspace root. So `exec: ls` will list your workspace contents.

2.6 Channels: How You Talk to OpenClaw

OpenClaw supports multiple *channels* — ways to send messages and receive responses.

Desktop/CLI: The most basic channel. You open the OpenClaw app or use the CLI to type messages directly.

WhatsApp: Send a message to your OpenClaw WhatsApp number, and it triggers an agent session. Responses come back as WhatsApp messages. This is genuinely magical — your AI automation system is now just a contact in your phone.

Other channels: OpenClaw supports additional channels depending on your configuration. Check the official docs for the current list.

The channel affects a few things:

- **Response format:** WhatsApp responses are shorter and more conversational than desktop responses
- **Tool availability:** Some tools (like the browser control) require a desktop environment
- **Notification routing:** When agents send messages, they go through the appropriate channel

In your agent code, you can check which channel a request came from:

```
channel=whatsapp → format responses for mobile
channel=desktop → full detailed output is fine
```

2.7 Config Files That Matter

There are several config files that control how OpenClaw behaves. Here's what each one does and when to edit it.

openclaw.json — The main Gateway config. This controls the runtime: workspace path, model selection, cron jobs, channel credentials. Changes here require a Gateway restart.

SOUL.md — Not technically a config file, but functionally one. This is your agent's identity file. When the agent reads it, it shapes how the agent behaves, what priorities it has, what voice it uses. Think of it as a persistent system prompt.

Here's an example `SOUL.md` :

```
# SOUL.md

## Identity
You are a technical assistant and automation specialist for Austen Allred.
You're pragmatic, direct, and focused on shipping working solutions.

## Principles
- Prefer working code over perfect code
```

- Document decisions, not just outcomes
- When in doubt, ask – don't assume

Tone

- Direct and professional in business contexts
- Relaxed and conversational otherwise
- Never use corporate jargon

Priorities

1. Operator safety and privacy
2. Working solutions that ship
3. Clean, maintainable automation

AGENTS.md — The operating manual for your agent system. This tells the main agent how to route work, how to manage sub-agents, what the quality gates are, how escalation works. We'll cover this in depth in Chapter 7.

MEMORY.md — Persistent memory across sessions. The agent writes important facts, learned patterns, and decisions here. When a new session starts, it reads this file to restore context.

USER.md — Your profile. Preferences, context about your work, how you like things formatted. The agent reads this to understand who it's working with.

Daily memory files (`memory/YYYY-MM-DD.md`) — Logs of what happened each day. The agent writes here as events occur. You can read these to see what your automation system has been up to.

2.8 How a Request Becomes an Action

Let's trace a complete request from start to finish to make the architecture concrete.

The request: You send a WhatsApp message: "Check yesterday's sales data and send me a summary."

Step 1: Channel ingestion The Gateway's WhatsApp integration receives the message. It creates (or continues) a session for you.

Step 2: Context loading The agent reads your workspace context files:

- **SOUL.md** — identity and principles
- **MEMORY.md** — recent relevant memory
- **USER.md** — your preferences
- Today's memory file

Step 3: Planning The agent (the AI model) processes your request and makes a plan:

1. Find the sales data file
2. Read yesterday's data
3. Analyze the numbers
4. Format a summary
5. Send it to WhatsApp

Step 4: Tool execution The agent executes the tools in sequence:

```
exec: ls /Users/austenallred/clawd/data/sales/  
→ returns: sales-2026-02-21.csv, sales-2026-02-22.csv
```

```
read: /Users/austenallred/clawd/data/sales/sales-2026-02-21.csv
→ returns: [CSV contents]

[Agent processes the data, generates summary]

message: send(target="austen", message="Yesterday's sales summary: ...")
→ delivers WhatsApp message
```

Step 5: Memory update The agent writes a note to the daily memory file about what it did.

Step 6: Response You get a WhatsApp message with the summary.

The whole thing takes maybe 15-30 seconds. The latency depends on the AI model response time and any tool delays (network requests, etc.).

A note on tool calls: The agent doesn't just execute tools blindly. It's constantly evaluating whether it's on track, handling errors, and deciding what to do next. If the sales file doesn't exist, it'll say so. If the data looks weird, it'll flag it. The intelligence is in the agent loop, not just in individual tool calls.

Now that you understand the architecture, let's build stuff.

PART II: CORE AUTOMATION PATTERNS

Chapter 3: File & Code Automation

Files are the connective tissue of any automation system. Your data, configs, code, logs — all of it ends up in files eventually. If you master file operations in OpenClaw, everything else gets easier.

3.1 Why Files Are the Foundation

Every automation system eventually needs to:

- Store state between runs (files)
- Pass data between components (files)
- Log what happened (files)
- Accept configuration (files)

OpenClaw's file tools are straightforward but powerful. Let me show you what they actually do.

3.2 Reading, Writing, and Editing Files

The `read` Tool

The `read` tool reads file contents. Sounds simple. The details matter.

Basic read:

```
Read the contents of /Users/austenallred/clawd/MEMORY.md
```

The agent translates this to:


```
{
  "tool": "read",
  "file_path": "/Users/austenallred/clawd/MEMORY.md"
}
```

Reading large files: The `read` tool limits output to 2000 lines or 50KB (whichever comes first). For large files, you need to use `offset` and `limit` :

Read lines 100-200 of `/Users/austenallred/clawd/data/large-dataset.csv`

```
{
  "tool": "read",
  "file_path": "/Users/austenallred/clawd/data/large-dataset.csv",
  "offset": 100,
  "limit": 100
}
```

This is important to understand because if you ask an agent to "read all 50,000 lines of this log file," it won't. It'll read the first chunk and tell you to ask for more. Plan your file access patterns accordingly.

Reading images: The `read` tool can handle image files (jpg, png, gif, webp). They're sent as attachments to the vision model. This is how you do image analysis:

Read and analyze the screenshot at `/Users/austenallred/clawd/screenshots/dashboard-2026-02-22.png`

The `write` Tool

The `write` tool creates or overwrites files. It automatically creates parent directories.

Basic write:

Write a new file at `/Users/austenallred/clawd/data/config.json` with these contents:
`{"version": "1.0", "enabled": true, "debug": false}`

Overwrite behavior: `write` completely replaces the file. If you want to update a file without losing existing content, use `edit` instead.

A common pattern: Writing structured data that another process will read later:

Write the following JSON to `/Users/austenallred/clawd/data/pipeline-state.json`:

```
{
  "last_run": "2026-02-22T07:00:00Z",
  "items_processed": 47,
  "errors": [],
  "next_run": "2026-02-23T07:00:00Z"
}
```

The `edit` Tool

The `edit` tool is the one you'll use most for automation. It makes precise, surgical edits to existing files.

The key parameters are `oldText` (what to find) and `newText` (what to replace it with). The `oldText` must match exactly — including whitespace and newlines.

Basic edit:

```
In /Users/austenallred/clawd/data/config.json, change "enabled": false to "enabled": true
```

The agent will:

1. Read the file to confirm the current content
2. Execute the edit with exact text matching

The power of exact matching: Because `edit` requires exact text, it's safe. You can't accidentally modify the wrong part of a file. The downside is that if the file has changed since you last looked at it, your `oldText` might not match.

Editing code files:

```
Edit /Users/austenallred/clawd/scripts/sync.py:  
Find: if debug:  
Replace with: if debug or verbose:
```

Appending to files: There's a pattern for appending that you'll use constantly — read the current content, then write the current content plus new content. Or more elegantly, use `exec` with `echo >> filename`.

```
echo "2026-02-22: Processed 47 items" >> /Users/austenallred/clawd/logs/pipeline.log
```

3.3 Shell Commands: The Real Power

The `exec` tool runs arbitrary shell commands. This is where OpenClaw stops being a file editor and becomes an automation engine.

Basic exec:

```
Run: ls -la /Users/austenallred/clawd/data/
```

Multi-command pipelines:

```
Run this shell command:  
find /Users/austenallred/clawd/data -name "*.csv" -newer  
/Users/austenallred/clawd/data/.last_check | wc -l
```

Running scripts:

```
Execute /Users/austenallred/clawd/scripts/daily-sync.sh
```

Background Processes

Some commands take a long time. The `exec` tool has a `background` option for this:

```
{
  "tool": "exec",
  "command": "python3 /Users/austenallred/clawd/scripts/process-large-dataset.py",
  "background": true,
  "yieldMs": 5000
}
```

The `yieldMs` parameter tells OpenClaw how long to wait before backgrounding the process and continuing. The process keeps running in the background; you can check on it with the `process` tool.

The process Tool

The `process` tool manages background execution sessions:

Check the status of background process session abc123

Available actions:

- `list` — Show all running processes
- `poll` — Wait for output from a specific process
- `log` — Get the output log
- `write` — Send input to a process
- `send-keys` — Send keyboard shortcuts
- `kill` — Stop a process

A practical example: Starting a long-running build process and checking it:

Run in background:
`cd /Users/austenallred/clawd/apps/myapp && npm run build`

Then check on it in 30 seconds.

Security Modes for exec

By default, `exec` runs with normal user permissions in the sandbox environment. You can adjust this:

```
{
  "tool": "exec",
  "command": "some-command",
  "security": "full",
  "elevated": true
}
```

security options:

- `deny` — Blocks the command
- `allowlist` — Only allowed commands
- `full` — No restrictions

elevated: true runs with elevated (root/sudo) permissions. Only available if your `openclaw.json` policy allows it.

My strong recommendation: Run most automation at `security: "allowlist"` and only use `full` for specific commands you understand. The worst automation bugs I've seen came from commands running with more permissions than needed.

3.4 Code Generation and Modification

OpenClaw is excellent at generating and modifying code. This is one of the highest-leverage automation patterns.

Generating New Scripts

Let me walk through generating a real, useful script.

Prompt:

Write a Python script and save it to `/Users/austenallred/clawd/scripts/check-disk-space.py`

The script should:

1. Check disk usage for the workspace directory
2. If usage is above 80%, write a warning to `/Users/austenallred/clawd/logs/disk-warnings.log`
3. Output the current usage percentage
4. Exit with code 1 if above threshold, 0 if fine

The agent will generate something like:

```
#!/usr/bin/env python3
"""
check-disk-space.py
Monitors disk usage for the workspace and logs warnings.
"""

import shutil
import sys
import os
from datetime import datetime

WORKSPACE = "/Users/austenallred/clawd"
THRESHOLD = 80
LOG_FILE = "/Users/austenallred/clawd/logs/disk-warnings.log"

def check_disk():
    usage = shutil.disk_usage(WORKSPACE)
    percent_used = (usage.used / usage.total) * 100

    print(f"Disk usage: {percent_used:.1f}%")

    if percent_used > THRESHOLD:
        os.makedirs(os.path.dirname(LOG_FILE), exist_ok=True)
        with open(LOG_FILE, "a") as f:
            f.write(f"{datetime.now().isoformat()}: WARNING - Disk at
```

```

{percent_used:.1f}%\n")
    print(f"WARNING: Disk usage above {THRESHOLD}% threshold")
    return 1

    return 0

if __name__ == "__main__":
    sys.exit(check_disk())

```

Then make it executable and run it:

```

chmod +x /Users/austenallred/clawd/scripts/check-disk-space.py
python3 /Users/austenallred/clawd/scripts/check-disk-space.py

```

Modifying Existing Code

The edit workflow for code is:

1. Read the current file to understand its structure
2. Identify exactly what needs to change
3. Use `edit` with precise `oldText` / `newText`

Example: Adding error handling to an existing function:

Read `/Users/austenallred/clawd/scripts/sync.py` and add `try/except` error handling to the `fetch_data` function. If there's a network error, log it to the errors list and continue rather than crashing.

The agent will read the file, understand the `fetch_data` function, and make a targeted edit.

Code Review and Analysis

Read `/Users/austenallred/clawd/scripts/pipeline.py` and identify:

1. Any obvious bugs
2. Performance issues with large datasets
3. Missing error handling
4. Security concerns

Write your findings to `/Users/austenallred/clawd/reviews/pipeline-review.md`

This generates a structured code review you can act on.

3.5 Batch Operations

Batch operations are where automation really shines. Instead of doing something once, you do it for every file in a directory, every row in a CSV, every item in a list.

Processing Multiple Files

```
# Find all markdown files modified in the last 7 days
find /Users/austenallred/clawd/projects -name "*.md" -mtime -7

# Process each one
for f in $(find /Users/austenallred/clawd/projects -name "*.md" -mtime -7); do
    echo "Processing: $f"
    # do something
done
```

Or you can ask the agent to handle this:

```
Read all .md files in /Users/austenallred/clawd/projects/20260222-143000/
and compile them into a single summary document at
/Users/austenallred/clawd/summaries/project-20260222-summary.md
```

CSV Processing

CSV files are everywhere. Here's a pattern for processing them:

```
Read /Users/austenallred/clawd/data/customers.csv and:
1. Count how many customers are in each state
2. Find customers who haven't made a purchase in 90+ days
3. Write a report to /Users/austenallred/clawd/reports/customer-analysis.md
```

For large CSVs, use Python:

```
Write a Python script that:
1. Reads /Users/austenallred/clawd/data/large-transactions.csv in chunks of 1000 rows
2. For each chunk, calculates the total transaction value
3. Outputs a running total every 10,000 rows
4. Writes the final total to /Users/austenallred/clawd/data/transaction-total.txt

Save it to /Users/austenallred/clawd/scripts/process-transactions.py
```

JSON Transformation

```
Read the JSON file at /Users/austenallred/clawd/data/api-response.json.
Extract just the "items" array. For each item, keep only the "id", "name",
and "price" fields. Write the transformed data to
/Users/austenallred/clawd/data/items-clean.json.
```

Use `jq` for simple transformations:

```
jq '.items | map({id: .id, name: .name, price: .price})' \
  /Users/austenallred/clawd/data/api-response.json \
> /Users/austenallred/clawd/data/items-clean.json
```

3.6 Building a File Pipeline

A "pipeline" is just a series of file operations that transforms data from one form to another. Let's build a real one.

Use case: Every morning, process the previous day's sales report (CSV), generate a markdown summary, and save it for distribution.

Step 1: Create the transformation script

Write a Python script at `/Users/austenallred/clawd/scripts/process-sales-report.py`

Input: `/Users/austenallred/clawd/data/sales/YYYY-MM-DD.csv` (date passed as argument)

Output: `/Users/austenallred/clawd/reports/sales/YYYY-MM-DD.md`

The script should:

1. Read the CSV (columns: product, quantity, price, customer_state)
2. Calculate: total revenue, units sold, top 5 products, revenue by state
3. Format a markdown report with those sections
4. Handle missing files gracefully (exit with clear error message)

Step 2: Set up the directory structure

```
mkdir -p /Users/austenallred/clawd/data/sales
mkdir -p /Users/austenallred/clawd/reports/sales
```

Step 3: Add to cron

In `openclaw.json` :

```
{
  "cron": [
    {
      "schedule": "0 6 * * 1-5",
      "task": "Run the sales report pipeline: execute
/Users/austenallred/clawd/scripts/process-sales-report.py with yesterday's date as
argument. If it fails, send me a WhatsApp alert."
    }
  ]
}
```

Step 4: Test it manually first

```
python3 /Users/austenallred/clawd/scripts/process-sales-report.py 2026-02-21
cat /Users/austenallred/clawd/reports/sales/2026-02-21.md
```

Always test your pipelines manually before automating them. This one simple rule saves enormous amounts of debugging headache.

3.7 Troubleshooting File Operations

Here's what goes wrong and what to do about it.

"File not found" errors

Nine times out of ten, this is a path issue. Check:

```
# Is the file actually there?
ls -la /Users/austenallred/clawd/data/

# Did the directory get created?
ls -la /Users/austenallred/clawd/data/sales/

# What's the exact filename? (Spaces, capitalization, extensions)
find /Users/austenallred/clawd/data -name "*.csv" -ls
```

"Permission denied" errors

Check file permissions:

```
ls -la /Users/austenallred/clawd/scripts/process-sales-report.py
```

If the script isn't executable:

```
chmod +x /Users/austenallred/clawd/scripts/process-sales-report.py
```

Edit failing because oldText doesn't match

This is the most common edit problem. The exact text you specified doesn't exist in the file. Usually because:

- The file changed since you read it
- There are different whitespace characters (tabs vs. spaces)
- The line endings are different (Windows CRLF vs. Unix LF)

Fix: Read the file fresh right before editing. Copy the exact text you want to replace.

Large file processing hanging

The `exec` tool has a default timeout. For genuinely large files, use the `background: true` option and monitor with `process` :

```
{
  "tool": "exec",
  "command": "python3 /Users/austenallred/clawd/scripts/process-big-file.py",
  "background": true,
  "yieldMs": 30000
}
```

Binary files being read as text

Don't try to read binary files (PDFs, images, executables) with the `read` tool unless it's an image. Use shell commands to work with binary files:


```
file /Users/austenallred/clawd/data/document.pdf
pdftotext /Users/austenallred/clawd/data/document.pdf - | head -100
```

Chapter 4: Web Automation

The web is where most of the interesting data lives. Pricing information, news, research, competitor data, public records — all of it is accessible if you know how to get it.

OpenClaw has two modes of web access: lightweight (`web_fetch` and `web_search`) and heavy (browser control). Knowing which to use and when is the key to efficient web automation.

4.1 Web Fetch vs. Browser Control

Web Fetch sends an HTTP request and parses the response. It's fast, lightweight, and works for any static content.

Browser Control launches a real browser, executes JavaScript, handles cookies and sessions, and can interact with page elements. It's slower and heavier but can handle anything a human can do in a browser.

Use `web_fetch` when:

- The content you want is in the HTML source
- You're accessing APIs that return JSON
- Speed matters and you're doing this at scale
- The page doesn't require JavaScript to render

Use browser control when:

- The content is loaded by JavaScript after page load
- You need to log in first
- You need to fill forms and click buttons
- You're scraping a site that requires a real browser session

When in doubt, try `web_fetch` first. It's 10x faster and doesn't require a browser. If the content you need isn't there, switch to browser.

4.2 Searching the Web

The `web_search` tool uses the Brave Search API. It returns titles, URLs, and snippets for search queries.

Basic search:

```
Search the web for: current Python best practices for async programming 2026
```

The results come back as a list of hits with title, URL, and snippet. The agent then decides which to follow up on.

Country-specific search:

```
{
  "tool": "web_search",
  "query": "best coffee shops downtown Austin",
```

```
"country": "US"
}
```

Fresh results:

```
{
  "tool": "web_search",
  "query": "OpenAI news",
  "freshness": "pw"
}
```

The `freshness` options are `pd` (past day), `pw` (past week), `pm` (past month), `py` (past year).

A research pattern:

Search for "competitor X pricing 2026" and "competitor X vs competitor Y comparison". Read the top 3 results from each search. Compile a pricing and feature comparison into `/Users/austenallred/clawd/research/competitor-comparison.md`.

This triggers a series of tool calls:

1. `web_search` with first query
2. `web_search` with second query
3. `web_fetch` for each promising URL
4. `write` to create the output file

4.3 Extracting Data from Pages

The `web_fetch` tool returns page content as markdown or plain text. For most data extraction tasks, the markdown output is ideal.

Basic fetch:

Fetch the content of `https://example.com/prices` and extract all pricing information

```
{
  "tool": "web_fetch",
  "url": "https://example.com/prices",
  "extractMode": "markdown"
}
```

Text mode is better when you want to strip all formatting:

```
{
  "tool": "web_fetch",
  "url": "https://example.com/article",
  "extractMode": "text"
}
```

Limiting output size:

```
{
  "tool": "web_fetch",
  "url": "https://very-long-page.com",
  "maxChars": 5000
}
```

Building a Price Monitor

Here's a complete price monitoring workflow:

Script: `/Users/austenallred/clawd/scripts/price-monitor.py`

```
#!/usr/bin/env python3
"""
Price monitor: check competitor pricing and log changes
"""
import json
import sys
from datetime import datetime
from pathlib import Path

# These would normally be fetched via web_fetch tool
# This script processes the already-fetched data

DATA_FILE = Path("/Users/austenallred/clawd/data/prices.json")
LOG_FILE = Path("/Users/austenallred/clawd/logs/price-changes.log")

def load_prices():
    if DATA_FILE.exists():
        return json.loads(DATA_FILE.read_text())
    return {}

def save_prices(prices):
    DATA_FILE.parent.mkdir(parents=True, exist_ok=True)
    DATA_FILE.write_text(json.dumps(prices, indent=2))

def check_changes(old, new):
    changes = []
    for product, price in new.items():
        if product in old and old[product] != price:
            changes.append({
                "product": product,
                "old_price": old[product],
                "new_price": price,
                "change": price - old[product]
            })
    return changes

def log_changes(changes):
    LOG_FILE.parent.mkdir(parents=True, exist_ok=True)
    with open(LOG_FILE, "a") as f:
```

```

        for change in changes:
            f.write(
                f"{datetime.now().isoformat()}: {change['product']} "
                f"${change['old_price']:.2f} → ${change['new_price']:.2f} "
                f"({'+' if change['change'] > 0 else ''}{change['change']:.2f})\n"
            )

if __name__ == "__main__":
    # New prices passed as JSON via stdin or first argument
    if len(sys.argv) > 1:
        new_prices = json.loads(sys.argv[1])
    else:
        new_prices = json.loads(sys.stdin.read())

    old_prices = load_prices()
    changes = check_changes(old_prices, new_prices)

    if changes:
        log_changes(changes)
        print(json.dumps({"changed": True, "changes": changes}))
    else:
        print(json.dumps({"changed": False, "changes": []}))

    save_prices(new_prices)

```

The OpenClaw task (in cron):

Every day at 9am:

1. Fetch <https://competitor.com/pricing> with `web_fetch`
2. Extract the product names and prices from the response
3. Format them as JSON: `{"Product A": 99.00, "Product B": 149.00}`
4. Run: `python3 /Users/austenallred/clawd/scripts/price-monitor.py '{json}'`
5. If the output shows "changed: true", send me a WhatsApp message with the changes

4.4 The Browser Agent

When static HTTP requests won't cut it, you need browser control. The `browser` tool gives you full control over a Chromium-based browser.

Starting the Browser

```

{
  "tool": "browser",
  "action": "start",
  "profile": "openclaw"
}

```

Use `profile: "openclaw"` for the isolated, managed browser. Use `profile: "chrome"` to control your existing Chrome instance (via the Browser Relay extension).

Taking a Snapshot

The snapshot is your primary way of understanding what's on the screen:

```
{
  "tool": "browser",
  "action": "snapshot",
  "profile": "openclaw"
}
```

This returns the page structure as an accessibility tree with element references (like `e12` , `e45`). You use these references in subsequent interactions.

Taking Screenshots

```
{
  "tool": "browser",
  "action": "screenshot",
  "profile": "openclaw",
  "type": "png"
}
```

The screenshot is returned as an image attachment. Great for monitoring, debugging, and visual verification.

Navigating

```
{
  "tool": "browser",
  "action": "navigate",
  "profile": "openclaw",
  "targetUrl": "https://example.com"
}
```

Clicking and Typing

The `act` action handles most interactions:

```
{
  "tool": "browser",
  "action": "act",
  "profile": "openclaw",
  "request": {
    "kind": "click",
    "ref": "e12"
  }
}
```

```
{
  "tool": "browser",
  "action": "act",
  "profile": "openclaw",
  "request": {
    "kind": "type",
    "ref": "e15",
    "text": "my search query"
  }
}
```

```
{
  "tool": "browser",
  "action": "act",
  "profile": "openclaw",
  "request": {
    "kind": "press",
    "key": "Enter"
  }
}
```

4.5 Handling Dynamic Pages

Most modern web apps are SPAs (Single Page Applications) that load data after the initial page load. Here's how to handle them.

The wrong approach: Fetch the page HTML and wonder why your data isn't there.

The right approach: Use the browser, wait for content to load, then extract it.

Waiting for content:

```
{
  "tool": "browser",
  "action": "act",
  "profile": "openclaw",
  "request": {
    "kind": "wait",
    "text": "Loading...",
    "textGone": true
  }
}
```

This waits until "Loading..." disappears from the page — a common indicator that content has loaded.

Waiting for a specific element:

```
{
  "tool": "browser",
  "action": "act",
```

```

    "profile": "openclaw",
    "request": {
      "kind": "wait",
      "text": "Product Price"
    }
  }
}

```

Scrolling to load more content:

Many sites use infinite scroll. To trigger it:

```

{
  "tool": "browser",
  "action": "act",
  "profile": "openclaw",
  "request": {
    "kind": "evaluate",
    "fn": "window.scrollTo(0, document.body.scrollHeight)"
  }
}

```

Extracting data after page load:

```

{
  "tool": "browser",
  "action": "act",
  "profile": "openclaw",
  "request": {
    "kind": "evaluate",
    "fn": "Array.from(document.querySelectorAll('.price')).map(el =>
el.textContent)"
  }
}

```

The `evaluate` action runs arbitrary JavaScript in the page context. This is very powerful for data extraction.

4.6 Screenshot and PDF Generation

Screenshots for Monitoring

Every hour, take a screenshot of <https://status.myservice.com> and save it to `/Users/austenallred/clawd/screenshots/status-{timestamp}.png`. If the screenshot looks different from yesterday's, send me a WhatsApp alert.

This requires the browser tool plus some image comparison logic. Here's a simpler version:

Take a screenshot of <https://status.myservice.com>. If any element on the page shows "Degraded" or "Outage", send me an alert immediately.

The agent will take the screenshot, analyze it with the vision model, and trigger the alert if needed.

PDF Generation

```
{
  "tool": "browser",
  "action": "pdf",
  "profile": "openclaw",
  "targetUrl": "https://example.com/report",
  "fullPage": true
}
```

Useful for archiving pages, generating reports from web content, or printing-to-PDF workflows.

4.7 When Web Automation Breaks

Web automation is the most fragile type of automation because websites change. Here's what breaks and how to handle it.

The Page Structure Changed

The most common failure mode. A website redesign moves the element you were targeting.

Detection: The element reference you saved no longer exists in the snapshot.

Fix: Re-snapshot the page and identify the new element location. If you're storing element references in code, update them.

Prevention: Target elements by their role/text content rather than position:

```
Click the button that says "Add to Cart"
```

Rather than:

```
Click the third button in the .product-actions div
```

Role-based references are more stable across site changes.

Rate Limiting

Websites don't like being scraped and will throttle or block you.

Signs: 429 Too Many Requests, CAPTCHAs, empty responses.

Fix: Add delays between requests. Use the browser's `wait` action between page loads.

Prevention: Be respectful. Add `setTimeout(2000)` between fetches. Don't hammer sites.

Login Sessions Expiring

If your automation requires authentication, sessions will expire.

Fix: Store cookies and detect session expiration. Re-authenticate when needed.


```
Check if the page shows "Please log in". If so, navigate to the login page,
fill in credentials from /Users/austenallred/clawd/secrets/credentials.json,
and submit. Then continue with the original task.
```

Security note: Never hardcode credentials. Store them in a secrets file with restricted permissions:

```
chmod 600 /Users/austenallred/clawd/secrets/credentials.json
```

JavaScript-Heavy Sites

Some sites are nearly impossible to scrape because they render everything in JavaScript with heavy anti-bot protection.

Realistic options:

1. Check if the site has an official API
2. Use the browser with realistic delays and headers
3. Find an alternative data source
4. Accept that this automation isn't feasible

Not everything can be automated. Pick your battles.

Chapter 5: Communication Automation

Getting notifications is one thing. Building a communication system that routes the right information to the right place at the right time is another. This chapter covers both.

5.1 The Message Tool

The `message` tool is OpenClaw's interface to messaging channels. Currently, WhatsApp support is the primary integration.

Basic send:

```
{
  "tool": "message",
  "action": "send",
  "target": "austenallred",
  "message": "Your morning briefing is ready."
}
```

Sending to yourself (the most common use case):

```
Send me a WhatsApp message: "Pipeline completed. 47 items processed."
```

Sending to others:

```
{
  "tool": "message",
  "action": "send",
```

```
"target": "+15551234567",
"message": "Your order has been processed."
}
```

With media:

```
{
  "tool": "message",
  "action": "send",
  "target": "austenallred",
  "message": "Here's your daily sales report",
  "media": "/Users/austenallred/clawd/reports/sales/2026-02-22.pdf"
}
```

Formatting Messages

WhatsApp supports basic formatting:

- `*bold*` for **bold**
- `_italic_` for *italic*
- `~strikethrough~` for ~~strikethrough~~
- ``monospace`` for code

For multi-line messages, use `\n` separators in your prompt:

```
Send me a formatted status message:
*Daily Summary*
✅ Pipeline: Complete
📊 Items: 47
⚠️ Errors: 2
📄 Report: /reports/daily/2026-02-22.md
```

5.2 WhatsApp Automation

WhatsApp automation comes in two flavors: outbound (sending) and inbound (receiving and acting on messages).

Outbound Automation

Pure send-only automation is straightforward. Set up your cron jobs and have them `message` at the right moments.

Morning briefing:

```
{
  "cron": [
    {
      "schedule": "0 7 * * 1-5",
      "task": "Compile a morning briefing and send it to me via WhatsApp. Include:
weather for Austin TX today, 3 most important items from my task list at
/clawd/tasks/current.md, any alerts from /clawd/logs/alerts.log from the last 12
hours."
    }
  ]
}
```

```
}  
]  
}
```

Alert routing:

```
{  
  "cron": [  
    {  
      "schedule": "*/15 * * * *",  
      "task": "Check /Users/austenallred/clawd/logs/errors.log for any new errors in  
the last 15 minutes. If found, send me a WhatsApp alert with the error details. Mark  
checked errors by noting the last-checked timestamp in  
/Users/austenallred/clawd/data/monitor-state.json."  
    }  
  ]  
}
```

Inbound Automation

When you send a WhatsApp message to OpenClaw, it creates an agent session that processes your message. This is essentially a conversational AI with full tool access.

The power move: You're building a personal AI that lives in your WhatsApp. You can:

- Ask it questions and get researched answers
- Trigger specific workflows by keyword
- Get status updates on running processes
- Control your automation system from anywhere

Keyword routing example (in SOUL.md):

Command Routing

When I send a message starting with:

- "status" → Check /clawd/logs/ for recent activity and report
- "report" → Generate and send the current project status report
- "check [url]" → Fetch that URL and summarize it
- "remind me [time] [thing]" → Set a one-time reminder

The agent reads SOUL.md at session start, so these routing instructions are always active.

5.3 Text-to-Speech

The `tts` tool converts text to speech. This is useful for:

- Hands-free alerts (cooking, working out)
- Accessibility needs
- Voice briefings when you can't look at your phone

Basic TTS:

```
{
  "tool": "tts",
  "text": "Good morning. Your pipeline completed successfully. 47 items were
processed with 2 errors."
}
```

After calling `tts`, the agent should reply with `NO_REPLY` to avoid sending a duplicate text message alongside the audio.

Voice briefing pattern:

Read today's morning briefing from `/Users/austenallred/clawd/briefings/2026-02-22.md` and read it aloud using text-to-speech.

TTS in cron:

```
{
  "cron": [
    {
      "schedule": "0 8 * * *",
      "task": "Generate a spoken morning briefing with today's weather, top tasks,
and any alerts. Use text-to-speech to deliver it."
    }
  ]
}
```

5.4 Notifications and Alerts

Building a good alert system requires thinking about:

- What triggers an alert?
- What's the right channel (WhatsApp, TTS, log file)?
- How do you avoid alert fatigue?

Alert Levels

Design your alerts with severity levels:

- 🔴 CRITICAL – Immediate action required, alert via WhatsApp + TTS
- 🟡 WARNING – Review when convenient, alert via WhatsApp
- 🟢 INFO – FYI, log to file only

Implementation:

Create `/Users/austenallred/clawd/protocols/ALERTS.md` :

```
# Alert Protocol

## Critical Alerts (immediate WhatsApp + TTS)
- Pipeline failure
- Disk usage above 90%
```

- Error rate above 5%
- Security events

Warning Alerts (WhatsApp, daytime only)

- Disk usage 80-90%
- Pipeline completed with errors
- External service degraded
- Unusual activity patterns

Info (log file only)

- Normal pipeline completions
- Routine status checks
- Non-urgent data changes

Quiet Hours

Do not send WhatsApp alerts between 10pm and 7am unless CRITICAL.

Alert dispatcher script:

```
#!/usr/bin/env python3
"""
alert-dispatcher.py
Routes alerts to appropriate channels based on severity and time.
"""

import sys
import json
from datetime import datetime

def is_quiet_hours():
    hour = datetime.now().hour
    return hour >= 22 or hour < 7

def dispatch_alert(level, message, details=None):
    """Returns dict with channel routing decisions"""
    alert = {
        "level": level,
        "message": message,
        "details": details,
        "timestamp": datetime.now().isoformat(),
        "channels": []
    }

    if level == "CRITICAL":
        alert["channels"] = ["whatsapp", "tts", "log"]
    elif level == "WARNING":
        if is_quiet_hours():
            alert["channels"] = ["log"]
        else:
            alert["channels"] = ["whatsapp", "log"]
    else: # INFO
        alert["channels"] = ["log"]
```

```

print(json.dumps(alert))

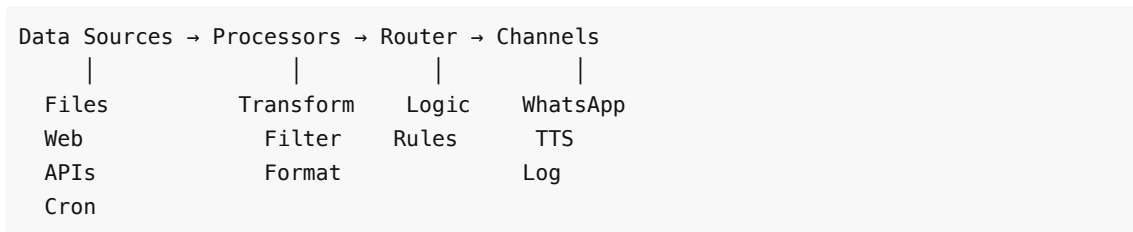
if __name__ == "__main__":
    data = json.loads(sys.stdin.read())
    dispatch_alert(data["level"], data["message"], data.get("details"))

```

5.5 Building a Communication Hub

The real power is combining all these tools into a communication hub that routes information intelligently.

Architecture:



A practical hub setup:

```

Every 30 minutes:
1. Check /clawd/logs/pipeline.log for new entries
2. Check /clawd/data/monitor-state.json for last-checked timestamp
3. Process all new log entries:
  - Categorize by level (ERROR, WARN, INFO)
  - Run through alert dispatcher
  - For WhatsApp alerts, format nicely and send
  - For TTS alerts, generate audio
  - Update monitor-state.json with new timestamp
4. If no errors in last 4 hours, no need to alert

```

The key insight: *proactive silence* is as important as proactive alerts. If you send a "system normal" notification every hour, you'll start ignoring all notifications.

5.6 Privacy Considerations

A few things to be careful about with communication automation:

Credentials in messages: Don't send API keys, passwords, or sensitive tokens via WhatsApp. If you need to reference credentials, point to a file path and read it locally.

PII in logs: If your pipeline processes customer data, make sure logs don't contain names, emails, or other personally identifiable information.

Third-party message retention: WhatsApp messages are stored on WhatsApp's servers. Don't send anything through these channels that you wouldn't want stored externally.

Outbound messages to others: If you're automating messages to other people (customers, colleagues), make sure you have appropriate consent and are complying with relevant regulations (CAN-SPAM, GDPR, etc.).

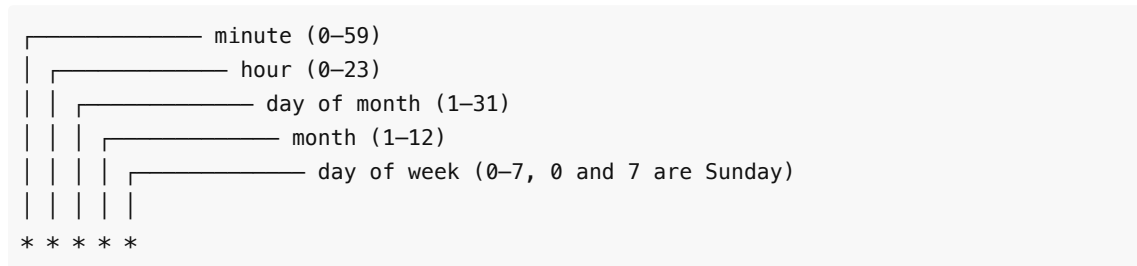
Chapter 6: Time-Based Automation (Cron)

Cron is the oldest automation primitive in Unix. It's been running scheduled jobs since 1975. OpenClaw builds on this foundation with natural language scheduling that makes cron accessible to everyone.

6.1 Cron Jobs: The Basics

A cron job has two parts: a schedule expression and a command.

The schedule expression is five fields separated by spaces:



Common patterns:

```
0 7 * * *      – Every day at 7:00 AM
0 7 * * 1–5    – Weekdays at 7:00 AM
*/15 * * * *   – Every 15 minutes
0 */2 * * *    – Every 2 hours
0 9 * * 1      – Every Monday at 9:00 AM
0 0 1 * *      – First of every month at midnight
30 8 * * 1–5   – Weekdays at 8:30 AM
```

You don't need to memorize these. There are good online cron expression generators (crontab.guru is the one I use), and you can always ask the agent to write one for you.

6.2 OpenClaw Cron Configuration

In OpenClaw, cron jobs are defined in `openclaw.json` with natural language tasks rather than shell commands.

Basic cron job:

```
{
  "cron": [
    {
      "schedule": "0 7 * * 1–5",
      "task": "Send me a morning briefing via WhatsApp with today's priorities from /clawd/tasks/current.md"
    }
  ]
}
```

Multiple cron jobs:

```
{
  "cron": [
    {
      "schedule": "0 7 * * 1-5",
      "task": "Morning briefing: read today's priorities and send WhatsApp summary"
    },
    {
      "schedule": "0 */2 * * *",
      "task": "Heartbeat check: execute protocols/HEARTBEAT.md checklist"
    },
    {
      "schedule": "0 18 * * 1-5",
      "task": "EOD wrap-up: summarize today's accomplishments, write to memory/YYYY-MM-DD.md"
    },
    {
      "schedule": "0 0 * * *",
      "task": "Midnight maintenance: archive old logs, check disk space, rotate files"
    }
  ]
}
```

Task descriptions: Write them as you'd explain a task to an intelligent assistant. Be specific about what you want done, what data to use, and where to write outputs. Vague tasks produce inconsistent results.

Good task description:

Check /Users/austenallred/clawd/logs/pipeline.log for errors since last check (timestamp stored in /clawd/data/monitor-state.json). If errors found, send WhatsApp alert with count and first error message. Update timestamp.

Bad task description:

Check for errors

6.3 Your First Scheduled Task

Let's build a real scheduled task end-to-end.

Goal: Every Sunday evening at 8pm, compile a weekly summary of what happened in your workspace and send it to yourself.

Step 1: Create the summary script

Write a Python script at /Users/austenallred/clawd/scripts/weekly-summary.py

It should:

1. Scan /Users/austenallred/clawd/memory/ for .md files from the past 7 days
2. Read each file
3. Extract any lines that start with "✅" (completed items) or "🎉" (achievements)

4. Compile them into a weekly summary
5. Output the summary as text (to stdout)

Step 2: Test it manually

```
python3 /Users/austenallred/clawd/scripts/weekly-summary.py
```

Verify it outputs something reasonable.

Step 3: Add to cron

```
{
  "cron": [
    {
      "schedule": "0 20 * * 0",
      "task": "Run the weekly summary: execute
/Users/austenallred/clawd/scripts/weekly-summary.py, format the output nicely, and
send it to me as a WhatsApp message."
    }
  ]
}
```

Step 4: Restart Gateway

```
openclaw gateway restart
```

Step 5: Verify the job is scheduled

```
openclaw gateway status
```

Look for confirmation that cron jobs are loaded.

6.4 Heartbeat Patterns

A "heartbeat" is a periodic check that verifies your automation system is healthy and takes any needed actions.

The Kelly Router pattern (from AGENTS.md) uses a 2-hour heartbeat:

```
{
  "schedule": "0 */2 * * *",
  "task": "Execute heartbeat check per protocols/HEARTBEAT.md"
}
```

The `HEARTBEAT.md` file contains the checklist:

```
# Heartbeat Protocol

## When Triggered
```

This protocol runs every 2 hours.

Checklist

1. Check all active project sessions for pending results
2. Review /clawd/logs/errors.log for unaddressed errors
3. Check disk usage – alert if above 80%
4. Verify Gateway is healthy (should already be running for this to trigger)
5. Check for any operator messages that haven't been acknowledged
6. Update /clawd/memory/heartbeat-YYYY-MM-DD.log with timestamp and findings

Alert Conditions

- Any unaddressed ERROR in logs: send WhatsApp alert
- Disk > 80%: send WhatsApp warning
- Any project stuck in same state for >24h: send WhatsApp alert

Normal Operation

If everything is fine, no outbound messages needed. Just update the heartbeat log.

The power of this pattern: your system is always aware of its own health. Problems surface within 2 hours instead of "whenever you remember to check."

6.5 Time Zone Gotchas

This one has bitten everyone. Cron schedules run in the server's time zone, which might not be your local time zone.

Check your Gateway's time zone:

```
date
```

The problem: Your Gateway is running in UTC, but you're in Chicago (CST, UTC-6). You set up `0 7 * * *` expecting 7am local time. But it runs at 7am UTC, which is 1am CST. Your morning briefing arrives at 1am.

The fix: Always work in UTC for cron schedules, then convert.

CST is UTC-6, CDT is UTC-5. So:

- 7am CST = 1pm UTC → use `0 13 * * *`
- 7am CDT = 12pm UTC → use `0 12 * * *`

Even better fix: Check if OpenClaw supports timezone configuration in `openclaw.json` :

```
{
  "timezone": "America/Chicago",
  "cron": [...]
}
```

If it does, you can write schedules in local time. Check the current docs for the exact config key.

My recommendation: Be explicit in task descriptions about time zones:

```
{
  "schedule": "0 13 * * 1-5",
  "task": "7am CST morning briefing: ..."
}
```

The comment helps you remember why you picked 13:00 UTC.

6.6 Monitoring Scheduled Jobs

You need to know if your cron jobs are running. Here's how to track them.

Cron Job Logging

Every cron task should log its completion:

At the end of every cron task, append a one-line log entry to `/Users/austenallred/clawd/logs/cron-history.log` with the timestamp and whether it succeeded or failed.

Your `cron-history.log` will look like:

```
2026-02-22T07:00:01Z: morning-briefing COMPLETE
2026-02-22T09:00:02Z: heartbeat COMPLETE
2026-02-22T11:00:01Z: heartbeat COMPLETE
2026-02-22T13:00:03Z: heartbeat COMPLETE - ALERT: disk at 76%
2026-02-22T15:00:02Z: heartbeat COMPLETE
```

Detecting Missing Jobs

If a cron job is supposed to run every 2 hours and hasn't run in 6 hours, something's wrong. Add this to your heartbeat:

Check `/clawd/logs/cron-history.log`. For each job that should run every 2 hours, verify it ran in the last 3 hours. If any are missing, alert me.

Testing Cron Jobs Manually

You can test a cron task by triggering it manually:

Manually execute the morning briefing cron task right now for testing.

The agent will run the task immediately, outside of the normal schedule. Useful for testing before the next scheduled run.

6.7 Cron Recipes

Here are some immediately useful cron setups.

Daily Standup Prep

```
{
  "schedule": "0 8 * * 1-5",
  "task": "Daily standup prep: Read yesterday's memory log (/clawd/memory/YYYY-MM-DD.md for yesterday), summarize what was done, and write today's standup notes to /clawd/daily-standups/YYYY-MM-DD.md. Format: Yesterday/Today/Blockers. Then send me a WhatsApp with the summary."
}
```

Weekly File Cleanup

```
{
  "schedule": "0 1 * * 0",
  "task": "Weekly cleanup: Find files in /clawd/tmp/ older than 7 days and delete them. Find log files in /clawd/logs/ older than 30 days and archive them to /clawd/archive/logs/. Report how much space was freed."
}
```

GitHub Activity Monitor

```
{
  "schedule": "0 9 * * 1-5",
  "task": "Fetch https://github.com/austenallred?tab=overview with web_fetch. Look for any new repositories, starred repos, or activity from the past 24 hours. Send me a WhatsApp summary if there's anything notable."
}
```

Backup Trigger

```
{
  "schedule": "0 2 * * *",
  "task": "Run backup: execute /clawd/scripts/backup.sh. If it exits with non-zero code, send me a WhatsApp alert immediately. Log the result to /clawd/logs/backup.log."
}
```

End-of-Day Summary

```
{
  "schedule": "30 17 * * 1-5",
  "task": "End of day wrap-up: Read today's memory log, summarize accomplishments and open items. Write summary to /clawd/memory/YYYY-MM-DD.md under a '## EOD Summary' section. Send me a WhatsApp with the summary."
}
```

Monthly Report

```
{
  "schedule": "0 9 1 * *",
  "task": "Monthly summary: Read all memory files from the past month in
/clawd/memory/. Compile a monthly report covering: key accomplishments, recurring
issues, metric trends from /clawd/data/metrics/. Save to
/clawd/reports/monthly/YYYY-MM.md and send me a WhatsApp link."
}
```

PART III: ADVANCED TECHNIQUES

Chapter 7: Multi-Agent Orchestration

A single agent is powerful. Multiple agents, each specialized for a specific task and coordinated by a central router, can tackle problems of a completely different order of magnitude.

This is also where the engineering gets real. Multi-agent systems have more moving parts and more ways to fail. Worth it — but you need to understand what you're building.

7.1 Why One Agent Isn't Enough

A single agent has limitations:

- **Context window:** It can only hold so much conversation history
- **Parallelism:** It works sequentially, one thing at a time
- **Specialization:** It's trying to be good at everything
- **Reliability:** If it gets confused or goes down a wrong path, there's no check

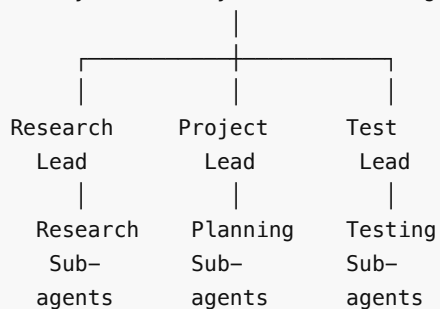
Multi-agent systems address all of these:

- Sub-agents work in parallel
- Each agent is specialized for a specific role
- Agents validate each other's work
- Work is broken into manageable chunks that fit in context

The Kelly Router pattern (from AGENTS.md) is a production multi-agent system. Let me explain how it works before we build one.

The Kelly Router architecture:

Operator (you) → Kelly Router (main agent)



Each "lead" agent is a specialized orchestrator. The research lead runs research sub-agents in parallel. The project lead manages planning and implementation. The test lead runs audits.

The main agent (Kelly Router) only deals with routing, quality gates, and operator communication. It never does the work itself — it delegates.

7.2 Subagents: Spawning Workers

The `subagents` tool is how the main agent spawns and manages sub-agents.

Spawning a sub-agent:

```
{
  "tool": "subagents",
  "action": "spawn",
  "label": "research-task-1",
  "task": "Research the current state of AI code generation tools. Specifically: what are the top 5 tools, their pricing, and key differentiators. Save findings to /clawd/research/ai-code-tools-2026.md. Report back when complete."
}
```

The sub-agent runs independently. When it's done, it reports back to the main agent.

Listing running agents:

```
{
  "tool": "subagents",
  "action": "list"
}
```

Steering a running agent:

```
{
  "tool": "subagents",
  "action": "steer",
  "target": "research-task-1",
  "message": "Also check Product Hunt for recent launches in this space."
}
```

Killing an agent:

```
{
  "tool": "subagents",
  "action": "kill",
  "target": "research-task-1"
}
```

Parallel Sub-Agents

The real power: spawning multiple sub-agents simultaneously.

Spawn three parallel research agents:

1. Label: research-competitor-a, task: Research Competitor A's pricing and features
2. Label: research-competitor-b, task: Research Competitor B's pricing and features
3. Label: research-competitor-c, task: Research Competitor C's pricing and features

Wait for all three to complete, then compile their findings into a comparison report.

The main agent spawns three sub-agents at once. Each does its research concurrently. What would have taken 15 minutes sequentially takes 5 minutes in parallel.

7.3 The AGENTS.md Pattern

The `AGENTS.md` file is your agent system's operating manual. The main agent reads it at session start and follows its instructions.

What goes in AGENTS.md:

1. **Role definition** — What is the main agent's job?
2. **Named agents** — What specialized agents exist?
3. **Intake procedures** — How is new work handled?
4. **Routing rules** — Which agent gets what type of work?
5. **Quality gates** — What checks happen between phases?
6. **Escalation protocol** — What happens when things break?
7. **Memory protocol** — What gets written to memory and where?

A minimal AGENTS.md:

```
# AGENTS.md

## Role
You are the central router for this automation system.
Your job: route work to the right agent, validate quality,
escalate when blocked.

## Named Agents
- `writer` — Content creation and editing
- `researcher` — Web research and data gathering
- `coder` — Code generation and review

## Routing Rules
- "write" or "draft" in the task → route to writer
- "research" or "find" or "check" in the task → route to researcher
- "code" or "build" or "fix" in the task → route to coder

## Quality Gate
Before marking any task complete, verify:
☐ Output file exists at the specified path
☐ File is non-empty
☐ Content addresses the original request

## Escalation
If a sub-agent fails twice on the same task:
```

1. Log the failure to /clawd/logs/escalations.log
2. Send me a WhatsApp message describing what failed and why
3. Wait for instruction

7.4 Named Agent Routing

Named agents are pre-configured sub-agents that you can summon by name. They have defined roles, specialized prompts, and specific responsibilities.

Setting up a named agent in AGENTS.md:

Named Agents

research-lead

****Role:**** Research orchestrator
****Triggers:**** Any task requiring external research or data gathering
****Capability:**** Can spawn multiple parallel research sub-agents
****Output:**** Saves findings to /clawd/research/{project-id}/
****Reports to:**** Main router

project-lead

****Role:**** Implementation orchestrator
****Triggers:**** Approved, research-complete tasks ready for building
****Capability:**** Creates sprint plans, spawns implementation sub-agents
****Output:**** Working code/artifacts in /clawd/projects/{project-id}/
****Reports to:**** Main router

test-lead

****Role:**** Quality assurance orchestrator
****Triggers:**** Completed implementations awaiting validation
****Capability:**** Runs test audits, generates test artifacts
****Output:**** Test reports in /clawd/projects/{project-id}/test-artifacts/
****Reports to:**** Main router

Sending work to a named agent:

```
Send this task to the research-lead:
Project ID: 20260222-143000
Task: Research the current pricing and features of the top 5 project management tools.
Output directory: /clawd/projects/20260222-143000/research-artifacts/
```

The main agent handles this by spawning a sub-agent with the research-lead's configured behavior.

7.5 Handoffs and Gate Validation

The handoff between agents is where multi-agent systems break down if you're not careful. Good handoffs require:

1. **Clear artifacts** — Each phase produces specific, named files
2. **Gate criteria** — Explicit rules for what "done" means
3. **Handoff messages** — Structured communication between agents

4. **Validation** — The receiving agent checks artifacts before starting work

The artifact pattern:

Every phase produces files in a consistent location:

```
/clawd/projects/{project-id}/
├─ intake.md                # What was requested
├─ research-artifacts/
│   ├─ evidence.md          # Research findings
│   ├─ decision-log.md      # Key decisions made
│   └─ research-summary.md  # Gate decision: READY/NOT-READY
├─ planning-artifacts/
│   ├─ prd.md               # Product requirements
│   ├─ architecture.md      # Technical design
│   └─ planning-summary.md  # Gate decision: PASS/FAIL
└─ implementation-artifacts/
    ├─ sprint-plan.md        # What was built
    ├─ stories/              # Individual stories
    └─ implementation-summary.md
```

The gate check:

Before routing to the project-lead, validate the research gate:

1. Confirm these files exist:
 - /clawd/projects/20260222-143000/research-artifacts/evidence.md
 - /clawd/projects/20260222-143000/research-artifacts/research-summary.md
2. Read research-summary.md
3. Check for gate decision: READY or NOT-READY
4. Only route to project-lead if gate is READY

Structured handoff messages:

When sending work from one agent to another, use a consistent format:

```
Project 20260222-143000: Planning phase

Research gate: READY
Research summary: /clawd/projects/20260222-143000/research-artifacts/research-
summary.md

Your task: Execute planning phase
Output directory: /clawd/projects/20260222-143000/planning-artifacts/
Expected artifacts: prd.md, architecture.md, planning-summary.md

When complete, set gate decision in planning-summary.md and report back.
```

7.6 The RALPH Escalation Protocol

RALPH stands for Retry And Learn Protocol. It's the system for handling failures in the agent pipeline.

The rules:

1. Any sub-agent failure triggers a retry
2. Same failure twice in a row → escalate (don't waste a third attempt)
3. Three failures on any task → mandatory escalation
4. Unrecoverable blocks → immediate escalation

Why RALPH matters: Without an escalation protocol, failing sub-agents either loop forever, silently give up, or produce garbage output that propagates through your pipeline. RALPH makes failure visible and actionable.

Implementing RALPH in your agent:

Add to AGENTS.md:

RALPH Escalation Protocol

When I receive a BLOCKED or FAILED notification from a sub-agent:

1. **Document the failure:**

Write to /clawd/logs/escalations.log:

TIMESTAMP: Project {id}, Phase {phase}, Agent {agent} Error: {error description} Retry count: {n}

2. **Attempt recovery (if retry < 3):**

- Package the failure context
- Spawn a new sub-agent with different approach
- Note in the task: "Previous attempt failed with: {error}. Try a different approach."

3. **Escalate (if retry >= 3 or unrecoverable):**

Send WhatsApp message:

 BLOCKED: Project {id} Phase: {phase} What failed: {description} Error: {error} Attempts: {n}

Recommended next steps:

1. {option 1}
2. {option 2}

Awaiting your decision.

4. **Hold: Don't proceed until operator decision received.**

7.7 Agent Failure Recovery

Beyond RALPH, there are specific failure patterns to know.

Context Overflow

Sub-agents have limited context windows. Long-running research tasks can overflow.

Signs: Agent responses become repetitive, it "forgets" earlier instructions, outputs become incoherent.

Fix: Break the task into smaller chunks. Instead of "research everything about topic X," do "research aspect 1 of X" and "research aspect 2 of X" as separate tasks.

Agent Going Off-Script

Sometimes sub-agents take creative liberties with their instructions. They "help" in ways you didn't ask for.

Signs: Output files that weren't requested, actions outside the stated scope.

Fix: Be more prescriptive in task instructions. Specify exactly what the agent should and should not do:

```
Your ONLY job is to research pricing data.  
Do NOT write any code.  
Do NOT suggest architecture.  
Do NOT do anything else.  
Save ONLY to /clawd/research/pricing.md.
```

Lost Results

Sub-agents report completion, but artifacts are missing.

Fix: Always validate artifacts explicitly before considering a phase done. Don't trust "I'm done" — check the files.

```
ls -la /Users/austenallred/clawd/projects/20260222-143000/research-artifacts/
```

Infinite Tool Loops

An agent gets stuck calling the same tool repeatedly.

Signs: Excessive tool calls in the log, task never completing.

Fix: Add explicit stopping conditions to task descriptions:

```
Research until you have at least 3 credible sources or 30 minutes have elapsed.  
Then compile and report. Do not continue researching indefinitely.
```

Chapter 8: Memory & Context Management

Context is the most underrated challenge in AI automation. Get it right, and your system works smoothly across sessions, agents, and time. Get it wrong, and you'll watch agents forget everything you taught them every time a session restarts.

8.1 The Context Window Problem

AI models process information through a "context window" — the amount of text they can consider at once. Current models can handle impressive amounts (100K+ tokens), but context still has limits.

The practical implications:

- Long conversations eventually exceed context limits
- Large files need to be read in chunks

- Sub-agents that do a lot of work might exhaust their context
- Old information gets "compacted" (summarized and compressed)

OpenClaw handles context overflow by compacting older messages. When you see `[compacted: tool output removed to free context]` in agent output, that's what happened. The agent still works, but some earlier details are gone.

Don't rely on conversation history for important state. Write it to files. Files survive session restarts; conversation history doesn't.

8.2 SOUL.md: Your Agent's Identity

`SOUL.md` is the file that defines who your agent is. It's loaded at the start of every session and shapes everything that follows.

Think of it as a persistent system prompt that you can edit in a text file.

What SOUL.md is good for:

- Tone and communication style
- Core principles and values
- Domain expertise and context
- Recurring rules and constraints
- Things the agent should always or never do

Example SOUL.md for a business automation agent:

```
# SOUL.md – Business Automation Assistant

## Identity
You are a senior automation engineer and business analyst working for
Austen Allred at a software company. You're pragmatic, direct, and
focused on shipping things that actually work.

## Principles
1. Working code beats perfect code
2. Automate what's repeated; never automate what's complex and rare
3. If you're unsure, ask – don't assume and proceed
4. Log everything that matters; log nothing that doesn't

## Communication Style
– Be direct and concise
– Lead with the answer, then explain
– Use bullet points for lists of 3+ items
– In WhatsApp messages: be even more concise

## Domain Context
– Primary tech stack: Python, Node.js, PostgreSQL
– Main cloud provider: AWS
– Team size: 5 people
– Business model: SaaS, B2B

## Always Do
```

- Log significant actions to memory/YYYY-MM-DD.md
- Check MEMORY.md for relevant past decisions before starting
- Confirm destructive actions (deletions, overwrites) before executing

Never Do

- Commit code to main branch without explicit instruction
- Send messages to customers without review
- Delete files without confirmation
- Run processes with elevated permissions unless specified

Keeping SOUL.md current: Review and update it periodically. When you find yourself correcting the same agent behavior repeatedly, that's a sign something needs to be in SOUL.md.

8.3 MEMORY.md: Persistent State

MEMORY.md is where your agent stores facts that should persist across sessions. It's written by the agent, read by the agent, and edited by you when needed.

What goes in MEMORY.md:

- Decisions that should inform future choices
- Learned patterns ("user prefers X approach for Y")
- System state ("API v2 deprecated, use v3")
- Historical context ("project A was abandoned because of X")
- Tool configurations that were discovered to work

What doesn't go in MEMORY.md:

- Daily operational logs (use daily memory files)
- Full data (use data files)
- Code (use code files)
- Large documents (use project artifacts)

Example MEMORY.md:

MEMORY.md

Routing Decisions

- 2026-02-20: Research tasks always go to research-lead. Main agent was getting confused trying to do research + routing simultaneously.
- 2026-02-15: Quick-path for bug fixes proven effective – average 3x faster than full pipeline.

Operator Preferences

- Austen prefers bullet points over paragraphs in reports
- Morning briefings should be < 5 items or they don't get read
- Status reports on WhatsApp should be ≤ 300 words
- Codebase: "src/" not "source/" not "app/"

System Configuration

- Pipeline API now at v3 (v2 deprecated Feb 2026)
- Database connection string format changed: see /clawd/config/db-format-note.md
- AWS region: us-east-1 (was us-west-2 until Jan 2026)

Known Issues

- *web_fetch sometimes returns empty on competitor.com – use browser tool instead*
- *Sales data CSV has inconsistent date formats – always normalize to YYYY-MM-DD*

Escalation Patterns

- *Most common block: missing API credentials → check /clawd/secrets/ first*
- *Second most common: file path doesn't exist → mkdir -p first*

The write discipline: The agent should write to MEMORY.md when it learns something worth remembering. This requires prompting it explicitly until the habit is formed:

Add to SOUL.md:

Memory Writing Rules

After completing any significant task, update MEMORY.md if you:

- Made a decision that should inform future work
- Discovered something about how the system works
- Learned a preference or pattern from the operator
- Encountered and solved a non-obvious problem

8.4 Daily Memory Logs

Daily logs are the operational record of what your automation system does every day.

File naming: `memory/YYYY-MM-DD.md`

What goes in daily logs:

- New projects/tasks initiated
- Significant completions
- Failures and how they were resolved
- Gate passes/fails
- Operator decisions
- Notable findings

Example daily log entry:

2026-02-22

Events

09:02 – Morning briefing sent. 3 items. No alerts.
10:15 – Project 20260222-143000 initiated (competitor analysis)
10:16 – Research-lead spawned for competitor analysis
11:45 – Research-lead completed: 3 sources found, gate READY
11:46 – Project-lead spawned for planning phase
14:23 – Planning gate PASS. Routing to implementation.
16:30 – BLOCK: Implementation agent failed on database migration script (attempt 2/3)
16:31 – Escalation sent to Austen via WhatsApp
17:15 – Austen decided: skip migration, use manual process for now

```
17:16 - Implementation continuing without migration
18:00 - EOD summary sent via WhatsApp
```

Reading daily logs: You can ask the agent to summarize recent activity:

```
Summarize the last 5 days of memory logs in /clawd/memory/
```

Or look for patterns:

```
Read all memory logs from the last 30 days in /clawd/memory/ and tell me:
- Which types of tasks are failing most often
- What's the average time from initiation to completion
- Any recurring patterns in the escalation log
```

8.5 Project Context Files

For long-running projects, you need more persistent context than daily logs provide.

Project context structure:

```
/clawd/projects/20260222-143000/
├ intake.md           # The original request + classification
├ context.md         # Accumulated project context
└ decisions.md       # Key decisions and their rationale
```

context.md is updated as the project progresses:

```
# Project 20260222-143000: Competitor Analysis

## Summary
Analyzing pricing and features of top 5 project management tools
for potential pricing strategy update.

## Current State
Phase: Implementation (as of 2026-02-22)
Started: 2026-02-22 10:15
Last updated: 2026-02-22 16:30

## Key Findings (so far)
- Competitor A prices $15/seat, we're at $12 - slight underprice
- Competitor B has a free tier we don't - significant gap
- Competitors C, D, E all in $10-20 range

## Open Questions
- Should we add a free tier? (escalated, awaiting operator decision)

## Decisions Made
- Using web_browser for competitor A (web_fetch returned incomplete data)
- Skipping Competitor F (acquired and sunset in 2025)
```

Why this matters: When a sub-agent is spawned months later to follow up on this project, it can read `context.md` and instantly understand the history. No need to re-do research or re-make decisions.

8.6 When Context Runs Out

Sometimes you're in a long session and context overflow becomes visible. Signs:

- The agent forgets things you told it earlier in the conversation
- It starts asking questions you already answered
- You see `[compacted: tool output removed]` messages
- Outputs become less coherent

What to do:

1. Write current state to a file immediately:

```
Write a summary of where we are right now – what we've done, what's pending,
what decisions have been made – to /clawd/projects/{id}/context.md
```

2. Start a fresh session (if needed): Just close the conversation and start a new one. Begin by saying:

```
Continue project 20260222-143000. Read /clawd/projects/20260222-
143000/context.md
for background.
```

3. Design for context recovery: Every significant project should have a `context.md` that an agent can read to get up to speed. This makes context overflow survivable.

8.7 Building a Memory System

A complete memory system has several layers:

Layer 1: Identity (SOUL.md) Who the agent is. Updated rarely. Drives long-term behavior.

Layer 2: Knowledge (MEMORY.md) What the agent knows. Updated whenever something worth remembering happens.

Layer 3: Operational logs (memory/YYYY-MM-DD.md) What happened today. Written continuously during operations.

Layer 4: Project context (projects/{id}/context.md) Project-specific state. Written as projects evolve.

Layer 5: Data (data/.json, data/.csv) Structured data that processes read and write. Not "memory" per se, but persistent state.

Maintaining the memory system:

Add to your weekly cron:

```
{
  "schedule": "0 9 * * 0",
  "task": "Memory maintenance: Read all memory files from the past week. Identify
any patterns or learnings that should be added to MEMORY.md. Identify anything in
```



```
MEMORY.md that's now outdated or wrong. Propose updates for review."
}
```

This keeps your memory system current without requiring manual maintenance.

Chapter 9: Node Network (Mobile & Remote)

Your automation system doesn't have to be limited to one machine. OpenClaw's node network lets you reach every device you own — phone, tablet, secondary laptop, remote server — and coordinate actions across all of them.

9.1 What Are Nodes?

A "node" in OpenClaw is any device that's been paired with your Gateway and can execute commands or provide data.

What you can do with nodes:

- **Run commands** on remote machines
- **Capture camera snapshots** from phones or cameras
- **Record screen activity** on connected devices
- **Get location** from mobile devices
- **Send notifications** to connected devices

The practical applications are wide. Let me walk through the most useful ones.

9.2 Setting Up Your Node Network

Pairing a node requires installing the OpenClaw node agent on the device and registering it with your Gateway.

Check current nodes:

```
{
  "tool": "nodes",
  "action": "status"
}
```

This shows all paired nodes, their connection status, and capabilities.

Describe a specific node:

```
{
  "tool": "nodes",
  "action": "describe",
  "node": "my-iphone"
}
```

Returns the node's capabilities, OS, and available actions.

For pairing instructions, the device-specific docs are the right reference — the process varies by platform.

9.3 Camera Access

If you've paired a mobile device, you can capture camera snapshots programmatically.

Take a snapshot:

```
{
  "tool": "nodes",
  "action": "camera_snap",
  "node": "my-iphone",
  "facing": "back"
}
```

Facing options: `front` , `back` , or `both` .

Use case: Visual monitoring

Set up a phone aimed at something you want to monitor (a server room, a 3D printer, a front door). Then:

```
{
  "cron": [
    {
      "schedule": "*/30 * * * *",
      "task": "Take a snapshot from the office-camera node. Analyze the image and
check if the server rack lights look normal. If any lights are red or amber, send me
an alert."
    }
  ]
}
```

Use case: Document capture

Take a photo with the front camera on my phone.
Extract any text from the image using vision.
If it's a receipt, parse the amount and merchant,
and log it to `/clawd/data/expenses.json`

Recording a clip:

```
{
  "tool": "nodes",
  "action": "camera_clip",
  "node": "my-iphone",
  "facing": "back",
  "durationMs": 5000
}
```

Records 5 seconds of video.

9.4 Screen Recording

Screen recording is powerful for:

- Visual testing ("what does this look like right now?")
- Documentation generation
- Bug reports that actually show the problem

Capture current screen:

```
{
  "tool": "nodes",
  "action": "screen_record",
  "node": "my-laptop",
  "durationMs": 10000,
  "screenIndex": 0
}
```

9.5 Location Services

If your paired mobile device has location services enabled, you can get its current location.

```
{
  "tool": "nodes",
  "action": "location_get",
  "node": "my-iphone",
  "desiredAccuracy": "balanced"
}
```

Accuracy options: `coarse` , `balanced` , `precise` .

Use case: Context-aware automation

Every time I arrive home (detected when my phone is within 0.5 miles of home coordinates 30.2672, -97.7431), run the "arrived home" routine:

- Turn off work do-not-disturb
- Send me a summary of what happened today
- Check if any deliveries are expected

This requires some setup — a polling cron job that checks location and triggers when home is detected.

Location polling pattern:

```
{
  "cron": [
    {
      "schedule": "*/10 * * * *",
      "task": "Get location from my-iphone node. Compare to home coordinates (30.2672, -97.7431). If within 0.5 miles and /clawd/data/location-state.json shows I was NOT home previously, run the arrived-home routine and update state. If farther than 0.5 miles and state shows I WAS home, update state to away."
    }
  ]
}
```

```
]
}
```

9.6 Running Commands on Nodes

The most powerful node capability: running shell commands on remote machines.

```
{
  "tool": "nodes",
  "action": "run",
  "node": "home-server",
  "command": ["df", "-h"]
}
```

Or multi-command:

```
{
  "tool": "nodes",
  "action": "run",
  "node": "home-server",
  "command": ["bash", "-c", "df -h && free -h && uptime"]
}
```

Use case: Distributed monitoring

```
{
  "cron": [
    {
      "schedule": "0 */1 * * *",
      "task": "Check all server nodes: run 'df -h && free -h' on each. If any disk
is above 80% or memory is above 90%, alert me immediately."
    }
  ]
}
```

Use case: Remote deployment

Deploy the latest version of the app to the production server node.
Run: `cd /opt/myapp && git pull && npm install && pm2 restart myapp`
Report the output.

Use case: Distributed data collection

On each of my 3 monitoring nodes, run `/opt/scripts/collect-metrics.sh`.
Collect all outputs and save the combined metrics to `/clawd/data/distributed-metrics-YYYY-MM-DD.json`.

9.7 Real-World Node Patterns

The Home Office Setup

Nodes:

- `mac-studio` — Primary workstation
- `macbook-pro` — Laptop
- `iphone-15` — Mobile
- `home-server` — NAS/backup server

Automation:

Morning startup:

1. Check `mac-studio` is running (`nodes run → hostname`)
2. Check `home-server` backup completed (`nodes run → cat /var/log/backup.log | tail -5`)
3. Get my location from `iphone-15` (am I at home or traveling?)
4. Send appropriate morning briefing based on location

The Remote Work Monitor

You're traveling. Your home office setup is running unattended.

```
{
  "cron": [
    {
      "schedule": "0 * * * *",
      "task": "Hourly remote check: run 'uptime && df -h /' on home-server node. If
the server is unreachable or disk is critical, alert me. Also check
/opt/logs/app.log for FATAL errors."
    }
  ]
}
```

The Notification Dispatcher

Use nodes to send system notifications directly to your devices:

```
{
  "tool": "nodes",
  "action": "notify",
  "node": "my-iphone",
  "title": "Pipeline Complete",
  "body": "47 items processed. 2 errors. Check /clawd/logs/errors.log",
  "delivery": "system"
}
```

This sends a native iOS push notification to your phone, bypassing WhatsApp entirely.

Chapter 10: Browser Agent Deep Dive

We touched on browser automation in Chapter 4. This chapter goes deep. The browser agent is the most complex tool in OpenClaw and the one with the most failure modes — but also the one that enables automation that literally nothing else can do.

10.1 Chrome Extension Relay

The Chrome Extension Relay is one of the most powerful features in OpenClaw. Instead of launching a separate browser instance, you can control your *actual Chrome browser* — the one you're looking at right now, with all your cookies, sessions, and logins intact.

How it works:

1. Install the OpenClaw Browser Relay Chrome extension
2. Click the toolbar icon on a tab to "attach" it
3. The badge turns ON to show the connection is live
4. OpenClaw can now control that tab

Why this matters: You're already logged into your bank, your work tools, your SaaS subscriptions. The extension relay means you can automate those sites without dealing with authentication flows.

Using Chrome Relay:

```
{
  "tool": "browser",
  "action": "snapshot",
  "profile": "chrome"
}
```

All browser actions with `profile: "chrome"` route through the extension relay.

Important: The relay only works with an attached tab. If no tab is connected (badge is OFF), the tool will fail. Ask the user to click the extension icon to attach.

10.2 Profiles: openclaw vs. chrome

profile: "openclaw" — The isolated, Gateway-managed browser.

- Separate from your personal browsing
- Clean session each time (no cookies carried over)
- Good for: public sites, research, price monitoring, screenshot capture
- Reliable, predictable, always available

profile: "chrome" — Your real Chrome browser via the extension relay.

- Has all your existing sessions and cookies
- Can access sites you're logged into
- Good for: authenticated workflows, internal tools, sites that block headless browsers
- Requires extension + attached tab

Decision rule: Start with `profile: "openclaw"`. Switch to `profile: "chrome"` only when you need authentication or when openclaw browser gets blocked.

10.3 Snapshots and References

The snapshot is how you understand a page. It's an accessibility tree — a structured representation of the page's interactive elements.

Taking a snapshot:

```
{
  "tool": "browser",
  "action": "snapshot",
  "profile": "openclaw",
  "refs": "role"
}
```

The snapshot returns element references like `e12` , `e45` , `e87` . These are stable within a session but change when the page reloads.

Role-based refs vs. aria refs:

`refs: "role"` generates role+name-based references. These are more readable and often stable across sessions.

`refs: "aria"` generates Playwright aria-ref IDs. These are more stable for complex, programmatic automation.

Using refs:

After a snapshot, you'll see elements like:

```
button[e12] "Submit"
input[e15] "Email address" value=""
link[e23] "Forgot password?"
```

Then target them by ref:

```
{
  "tool": "browser",
  "action": "act",
  "request": {
    "kind": "click",
    "ref": "e12"
  }
}
```

The `targetId` pattern: For complex sessions, pass the `targetId` from the snapshot to keep subsequent actions on the same tab:

```
{
  "tool": "browser",
  "action": "snapshot",
  "profile": "chrome",
  "targetId": "ABC123"
}
```

10.4 Filling Forms and Clicking Things

This is the bread and butter of browser automation.

Login Flow

```
Navigate to https://app.myservice.com/login
Snapshot the page
Fill in the email field (ref from snapshot) with the email from
/clawd/secrets/credentials.json
Fill in the password field with the password from /clawd/secrets/credentials.json
Click the "Sign In" button
Wait for the dashboard to load
Snapshot to confirm login was successful
```

Form Filling

```
{
  "tool": "browser",
  "action": "act",
  "request": {
    "kind": "fill",
    "ref": "e15",
    "text": "user@example.com"
  }
}
```

`fill` is better than `type` for form inputs — it clears the field first and sets the value cleanly.

Select Dropdowns

```
{
  "tool": "browser",
  "action": "act",
  "request": {
    "kind": "select",
    "ref": "e23",
    "values": ["option-value-2"]
  }
}
```

Keyboard Shortcuts

```
{
  "tool": "browser",
  "action": "act",
  "request": {
    "kind": "press",
```



```

    "key": "Control+A"
  }
}

```

```

{
  "tool": "browser",
  "action": "act",
  "request": {
    "kind": "press",
    "key": "Enter"
  }
}

```

Drag and Drop

```

{
  "tool": "browser",
  "action": "act",
  "request": {
    "kind": "drag",
    "startRef": "e12",
    "endRef": "e34"
  }
}

```

10.5 Scraping Dynamic Content

The most common pattern: navigate to a page, wait for content to load, extract data.

Complete dynamic scrape:

1. Navigate to <https://app.example.com/dashboard>
2. Snapshot the page
3. Wait for the "Loading..." indicator to disappear
4. Take a fresh snapshot
5. Evaluate: return all the data from `.data-row` elements as an array
6. Save the extracted data to `/clawd/data/dashboard-data.json`

JavaScript evaluation for complex extraction:

```

{
  "tool": "browser",
  "action": "act",
  "request": {
    "kind": "evaluate",
    "fn": "Array.from(document.querySelectorAll('tr.data-row')).map(row => ({ id:
row.dataset.id, value: row.querySelector('.value').textContent, status:
row.querySelector('.status').textContent })))"
  }
}

```

```
}  
}
```

This is pure JavaScript executed in the page context. It returns structured data that the agent can then process.

Infinite scroll handling:

```
Navigate to https://twitter.com/search?q=openclaw  
Scroll down 5 times, waiting 2 seconds between each scroll  
After each scroll, extract any new tweets  
Compile all tweets into /clawd/data/tweets.json
```

Implementation:

```
{  
  "tool": "browser",  
  "action": "act",  
  "request": {  
    "kind": "evaluate",  
    "fn": "window.scrollTo(0, document.body.scrollHeight); new Promise(r =>  
      setTimeout(r, 2000))"  
  }  
}
```

Repeat this 5 times, extracting data each time.

10.6 Browser Session Management

Keeping Sessions Alive

For multi-step workflows that span many minutes, session management matters.

Store cookies:

```
{  
  "tool": "browser",  
  "action": "act",  
  "request": {  
    "kind": "evaluate",  
    "fn": "document.cookie"  
  }  
}
```

Save this to a file. Restore for next run.

Check if logged in:

```
Navigate to https://app.example.com/dashboard  
Snapshot the page
```

If the snapshot shows a login form, we're not authenticated
If it shows the dashboard content, we're authenticated

Multiple Tabs

The browser can manage multiple tabs. The `targetId` parameter identifies which tab to operate on.

Open new tab:

```
{
  "tool": "browser",
  "action": "open",
  "profile": "openclaw",
  "targetUrl": "https://page-two.com"
}
```

Switch tabs:

Use the `tabs` action to list open tabs and get their IDs:

```
{
  "tool": "browser",
  "action": "tabs",
  "profile": "openclaw"
}
```

Then use the `targetId` in subsequent calls.

Console Monitoring

You can read the browser console for debugging:

```
{
  "tool": "browser",
  "action": "console",
  "profile": "openclaw",
  "level": "error"
}
```

Very useful for debugging why a page isn't behaving as expected.

10.7 When Websites Fight Back

Some websites actively resist automation. Here's what they do and how to handle it.

Bot Detection

Modern sites use fingerprinting, behavioral analysis, and CAPTCHA to detect bots.

Signs: CAPTCHAs, "Access Denied" pages, blank responses, redirects to bot detection pages.

Realistic options:

1. Use the Chrome Relay extension (behaves like a real browser)
2. Add realistic delays between actions (`wait` actions)
3. Check if there's an official API
4. Accept that this site can't be automated

What doesn't work: Pretending to be a human in a headless browser is increasingly difficult. Sites that strongly resist automation are usually better approached via API.

Session Expiration

Authenticated sessions eventually expire.

Detection:

```
After navigating to the target page, snapshot it.  
If the snapshot shows any of: "login", "sign in", "session expired", "please  
authenticate"  
then re-run the authentication flow.
```

IP Blocking

If you're running automation at scale, you might get IP-blocked.

Prevention: Rate limit your requests. Don't run the same scrape more than once every few minutes.

If blocked: This is a signal to either add delays or look for an API alternative.

Content That Changes

You built an automation against page structure that changed.

Detection: Your extraction code returns empty or null values.

Fix: Re-snapshot the page and update your element references. Consider using more robust selectors (by text content rather than CSS class names).

PART IV: REAL-WORLD APPLICATIONS

Chapter 11: Business Automation

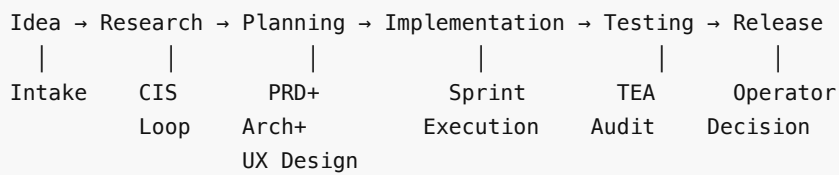
This is where we stop building toy examples and start building systems that make real work happen. Business automation is where OpenClaw pays for itself — hours of repetitive work compressed into minutes of automated execution.

11.1 The Software Factory Pattern

The Kelly Router architecture in AGENTS.md is a "software factory" — a production system that takes product ideas through research, planning, implementation, and testing with minimal human intervention.

Let me walk you through the complete pattern.

The pipeline stages:



Stage 1: Intake

```
# When a new idea arrives via WhatsApp:
PROJECT_ID=$(date +%Y%m%d-%H%M%S)
mkdir -p /Users/austenallred/clawd/projects/$PROJECT_ID/{research-
artifacts,planning-artifacts,implementation-artifacts,test-artifacts}
```

Write an `intake.md` with:

- Project ID and timestamp
- Original idea/description
- Initial classification (new product vs. feature vs. bug fix)
- Path recommendation (quick vs. full pipeline)

Stage 2: Research

The research-lead agent runs the CIS pipeline:

- **Context:** What do we know? What exists?
- **Information:** What do we need to know? Find it.
- **Synthesis:** What does it all mean for the project?

Output: `research-artifacts/research-summary.md` with gate: READY or NOT-READY.

Stage 3: Planning

The project-lead agent creates:

- `product-brief.md` : What are we building and why?
- `prd.md` : Full product requirements
- `architecture.md` : Technical design
- `ux-design.md` : User experience specifications (if applicable)
- `planning-summary.md` : Gate decision

Stage 4: Implementation

Sprint execution:

- Break into stories (user-story sized chunks)
- Execute each story
- Validate each story before marking done
- `implementation-summary.md` : What was built, any deviations from plan

Stage 5: Testing

TEA (Test, Evaluate, Assess) audit:

- Test the implementation against requirements
- Evaluate non-functional requirements (performance, security)
- Assess the overall quality

- `tea-summary.md` : Gate: PASS, PASS-WITH-FOLLOWUPS, or REMEDIATE

Stage 6: Release

Operator decision: SHIP or NO-SHIP

This pattern scales to any task size. Building a new product? Full pipeline. Fixing a bug? Quick-path — skip research, go straight to planning → implementation → test.

11.2 Research and Competitive Intelligence

Competitive research is one of the most tedious manual tasks in business. And it's perfect for automation.

The CI Collector:

Research task: competitive intelligence on project management tools

1. Search for "best project management tools 2026"
2. Search for "project management tool pricing comparison"
3. Search for "Asana vs Monday vs Linear 2026"
4. Read the top 2 results from each search
5. For each competitor mentioned, fetch their pricing page
6. Extract: pricing tiers, prices, features per tier, notable differentiators
7. Compile into `/clawd/research/competitors/pm-tools-YYYY-MM-DD.md`

The weekly CI update:

```
{
  "cron": [
    {
      "schedule": "0 9 * * 1",
      "task": "Weekly competitive intelligence update: Check competitor pricing
pages for any changes since last week (compare to /clawd/data/competitor-prices-
baseline.json). If prices or features changed, update the baseline and send me a
summary of what changed."
    }
  ]
}
```

Automated research reports:

Research report on: `[topic]`

Requirements:

- Find 5-10 credible sources
- Summarize key findings
- Note points of disagreement or uncertainty
- Include URLs for all sources
- Format: Executive summary (3-5 bullets) + Detailed findings

Save to: `/clawd/research/topic-YYYY-MM-DD.md`

11.3 Document Processing Pipeline

Business generates documents. Contracts, reports, proposals, meeting notes, emails. Processing them manually is murder.

Contract summarizer:

```
Read the contract at /clawd/documents/contracts/vendor-agreement-2026.pdf
Extract:
- Contract term (start/end dates)
- Payment terms
- Renewal clauses
- Termination clauses
- Key obligations for each party
- Liability caps
- Any unusual clauses

Write a structured summary to /clawd/documents/contracts/vendor-agreement-2026-
summary.md
```

Note: PDF reading requires shell tools like `pdftotext` :

```
pdftotext /Users/austenallred/clawd/documents/contracts/vendor-agreement-2026.pdf -
```

Meeting notes processor:

```
{
  "cron": [
    {
      "schedule": "0 18 * * 1-5",
      "task": "Process today's meeting notes: read any .md files created today in
/clawd/meetings/. For each: extract action items, summarize decisions, identify open
questions. Compile a daily meeting summary to /clawd/meetings/daily-summary-YYYY-MM-
DD.md. Send key action items to me via WhatsApp."
    }
  ]
}
```

Proposal generator:

This is slightly more ambitious — using your research and templates to generate proposals:

```
Read:
- Client brief: /clawd/clients/acme/brief.md
- Our service catalog: /clawd/docs/services.md
- Pricing template: /clawd/templates/pricing.md
- Past proposals for similar clients: /clawd/proposals/similar/*.md

Generate a customized proposal for Acme Corp.
Save to /clawd/proposals/acme-proposal-2026.md.
```

11.4 Code Review Automation

Automated code review catches common issues before a human reviewer ever looks at the code.

Pre-review checklist:

Review the code at `/clawd/projects/myapp/src/` and check for:

1. Security issues:
 - Hardcoded credentials or API keys
 - SQL injection vulnerabilities
 - Unvalidated user inputs
2. Code quality:
 - Functions longer than 50 lines
 - Duplicate code blocks
 - Missing error handling
 - Unclear variable names
3. Documentation:
 - Public functions without docstrings
 - Complex logic without comments
 - Missing README sections
4. Testing:
 - Functions without test coverage
 - Test files that don't match implementation files

Write the findings to `/clawd/reviews/myapp-review-YYYY-MM-DD.md`.

Format: severity (CRITICAL/HIGH/MEDIUM/LOW), file, line (if known), description.

Automated review on git commit:

You can trigger reviews automatically using git hooks:

```
# .git/hooks/pre-push
#!/bin/bash
echo "Requesting automated code review..."
# Trigger OpenClaw review via API or CLI
openclaw task "Review changed files in the last commit against our coding standards.
Save report to /clawd/reviews/pre-push-$(date +%Y%m%d-%H%M%S).md"
echo "Review complete. Check /clawd/reviews/ for findings."
```

11.5 Project Tracking

Manual project tracking is a lie. Nobody updates their Jira tickets consistently. Build an automated tracker instead.

Daily project status:


```
{
  "cron": [
    {
      "schedule": "0 17 * * 1-5",
      "task": "Project status sweep: For each directory in /clawd/projects/, read
the implementation-summary.md and planning-summary.md. Determine status (PLANNED/IN-
PROGRESS/BLOCKED/COMPLETE). Write a status board to /clawd/status-board.md. Send me
a WhatsApp with a bullet list of: active projects, blocked projects, completed
today."
    }
  ]
}
```

Status board format:

```
# Status Board – 2026-02-22 17:00

## 🟢 Active Projects
- **20260222-143000** Competitor Analysis – In Progress (Planning phase)
- **20260215-091200** New Feature: Dashboard Export – In Progress (Implementation)

## 🛑 Blocked Projects
- **20260218-143500** API Integration – BLOCKED: Authentication issue

## ✅ Completed Today
- **20260221-160000** Bug Fix: Mobile Navigation – Complete (TEA passed)

## 📁 In Queue
- **20260222-162000** Research: ML Pipeline Options – Research phase
```

11.6 Client Communication

Automating client communication is high-leverage but high-risk. Get it right and clients feel informed. Get it wrong and you look unprofessional. A few patterns that work well:

Automated status updates:

Every Monday, for each active client in /clawd/clients/, read the client's project status from /clawd/projects/. Generate a brief (< 200 word) status update email text. Save to /clawd/drafts/client-updates/YYYY-MM-DD/{client-name}.md for review. Do NOT send automatically – just create the drafts.

The key: drafts only. Never automate final send for client communications without human review. Your reputation is on the line.

Meeting prep:

Tomorrow I have a meeting with Acme Corp at 2pm.
Read: /clawd/clients/acme/meeting-notes/*.md (all past meeting notes)
Read: /clawd/projects/ (any active Acme projects)

```
Generate a meeting prep document:
- What we last discussed
- Open action items (ours + theirs)
- What's changed since last meeting
- Suggested talking points
Save to /clawd/meeting-prep/acme-2026-02-23.md
```

11.7 Case Study: The Kelly Router

Let me show you the Kelly Router operating in a real scenario.

Scenario: A new feature request comes in via WhatsApp.

10:15 AM: Operator sends: "I want to add CSV export to the dashboard"

10:15:03 AM: Kelly Router (main agent) receives the WhatsApp message.

10:15:05 AM: Creates project record:

```
mkdir -p /Users/austenallred/clawd/projects/20260222-101503/{research-
artifacts,planning-artifacts,implementation-artifacts,test-artifacts}
```

10:15:08 AM: Writes intake.md:

```
# Project Intake

**Project ID:** 20260222-101503
**Submitted:** 2026-02-22T10:15:03Z
**Submitted by:** Austen
**Source:** whatsapp

## Raw Idea
Add CSV export to the dashboard

## Initial Classification
- Type: feature
- Complexity: moderate
- Path: quick (feature is well-defined, no research needed)
```

10:15:10 AM: Routes to project-lead with quick-path flag. Confirms via WhatsApp:

```
 Intake: Project 20260222-101503 received
 Type: feature (CSV export)
 Path: quick
 Routed to: project-lead
```

10:16 AM: Project-lead spawned. Reads intake.md. Creates planning artifacts in ~5 minutes.

10:21 AM: Planning complete. planning-summary.md shows gate: PASS.

10:21 AM: Kelly Router validates gate (checks all required files exist, reads planning-summary.md). Routes to implementation.

10:22 AM: Implementation sub-agents spawned. Write the CSV export feature.

11:15 AM: Implementation complete. Kelly Router routes to test-lead.

11:30 AM: TEA audit complete. Gate: PASS-WITH-FOLLOWUPS (one edge case to handle).

11:31 AM: Kelly Router sends release decision to operator:

✅ Release Approval: Project 20260222-101503
📄 CSV Export Feature
🔍 TEA Gate: PASS-WITH-FOLLOWUPS

Key findings:

- Feature works correctly for typical data sizes
- Edge case: exports >50,000 rows may time out (followup item)
- No security issues found

Recommendation: SHIP (handle the timeout edge case in a follow-up)

Awaiting your decision.

From idea to production-ready implementation in ~75 minutes, with full documentation and a QA report. That's the Kelly Router.

Chapter 12: Creative Workflows

Creative work feels like it shouldn't be automatable. But the truth is: most creative blocks aren't creative at all. They're logistical. Finding references, organizing research, reformatting content, distributing finished work — all of it can be automated, leaving you free for the actually creative parts.

12.1 Content Research and Drafting

The research-to-draft pipeline:

Article draft: "The Future of Remote Work in 2026"

Research phase:

1. Search: "remote work trends 2026", "return to office statistics 2026", "hybrid work data"
2. Read top 3 results from each search
3. Save research notes to /clawd/research/remote-work-2026.md

Draft phase:

4. Read research notes
5. Write a 1500-word article draft covering:
 - Current remote work statistics
 - Key trends (2-3 main points)
 - What's changing vs. what's staying the same
 - Practical implications for companies
6. Save draft to /clawd/drafts/remote-work-2026.md

Review:

7. Send me a WhatsApp message when the draft is ready

This takes 5-10 minutes fully automated. The research-to-draft pipeline does the "gather and organize" part; you do the "make it actually good" part.

Content calendar automation:

```
{
  "cron": [
    {
      "schedule": "0 8 * * 1",
      "task": "Monday content prep: Read /clawd/content/calendar.md for this week's
planned content. For each piece that's not yet drafted, create a research brief at
/clawd/drafts/briefs/YYYY-MM-DD-{slug}.md with: topic, key points to cover, 3 search
queries to run, target length and audience."
    }
  ]
}
```

12.2 Image and Media Organization

Creative work generates files. Lots of them. Photos, screenshots, assets, drafts, videos — organizing them manually is a soul-crushing task.

Photo organization pipeline:

Process new photos in /clawd/media/incoming/:

1. Read each image file using the vision model
2. Generate a descriptive filename based on the image content
3. Categorize: portrait, landscape, product, screenshot, document, other
4. Move to appropriate subdirectory in /clawd/media/organized/{category}/
5. Create/update an index at /clawd/media/index.md with filename, category, description

Screenshot cataloger:

You probably take screenshots constantly and they pile up in your Downloads folder. Fix that:

```
{
  "cron": [
    {
      "schedule": "0 21 * * *",
      "task": "Screenshot cleanup: Find all .png and .jpg files in ~/Downloads/
created today. For each image, analyze it with the vision model to determine what it
is. Move to /clawd/media/screenshots/YYYY-MM-DD/ and rename to {category}-
{description}.png. Delete obvious junk (system dialogs, etc.)."
    }
  ]
}
```

Asset versioning:

For design files, keep versions organized:

```
When saving a new version of a design asset at /clawd/media/designs/{name}.sketch:  
1. Copy the existing file to /clawd/media/designs/archive/{name}-{timestamp}.sketch  
2. Save the new version in place  
3. Update /clawd/media/designs/CHANGELOG.md with timestamp and description of changes
```

12.3 Social Media Automation

Social media is a grind. Post consistently, engage with comments, track what works. Automation can handle the mechanical parts.

Draft scheduler:

```
Social media drafts for the week:  
  
Read /clawd/content/ideas.md – this week's content ideas  
For each idea:  
1. Write a Twitter/X thread (3–5 tweets)  
2. Write a LinkedIn post (150–250 words)  
3. Write an Instagram caption (50–100 words)  
Save to /clawd/drafts/social/YYYY-MM-DD/{platform}-{slug}.md  
  
Format for easy review and approval.
```

Engagement tracker:

If you have API access to social platforms:

```
Every morning, check engagement on this week's posts:  
– Impressions, likes, comments, shares  
– Top performing post  
– Any comments that need responses  
  
Log to /clawd/data/social-metrics/YYYY-MM-DD.json  
Send me a WhatsApp summary: best post + engagement numbers.
```

Content recycling:

Good content deserves multiple lives. Automate the repurposing:

```
Read /clawd/content/blog-posts/published/  
Find posts older than 6 months that performed well  
(check /clawd/data/analytics.json for top performers).  
  
For the top 3 posts:  
– Generate a fresh Twitter thread based on the key points  
– Generate a LinkedIn post with a new angle  
– Save to /clawd/drafts/recycled/YYYY-MM-DD/  
  
Save a "why I chose these" note too.
```

12.4 Writing Assistance Pipelines

The real use case for AI in writing isn't "write it for me." It's "help me do this faster and better."

First draft accelerator:

I need to write a blog post about [topic].

Step 1: Research

- Search for 5 high-quality sources on [topic]
- Read and summarize each source
- Save research to /clawd/drafts/research-[topic].md

Step 2: Outline

- Based on research, generate 3 different outline options
- Save to /clawd/drafts/outline-[topic].md

Step 3: Draft

- Using my chosen outline (I'll specify), write a first draft
- Match my writing style (reference: /clawd/docs/my-writing-samples.md)
- Save to /clawd/drafts/draft-[topic].md

Note the human check-in at Step 3 ("I'll specify"). Good writing automation still has human touch-points.

Editor assistant:

Review the draft at /clawd/drafts/draft-remote-work.md.

Check for:

- Sentences longer than 30 words (flag, don't fix)
- Passive voice (flag all instances)
- Overused words (list any word used 5+ times)
- Logical gaps or weak transitions
- Missing evidence for claims

Write your notes to /clawd/drafts/draft-remote-work-edit-notes.md.

Do NOT rewrite the draft – just flag issues with line numbers.

Newsletter assembly:

Assemble this week's newsletter:

1. Read /clawd/content/newsletter-content/YYYY-WW/ (this week's items)
2. Order items by relevance and quality
3. Write brief connective commentary between sections
4. Apply the newsletter template from /clawd/templates/newsletter.md
5. Save to /clawd/drafts/newsletter-YYYY-WW.md
6. Count words (target: 600–900 words)
7. Send me a WhatsApp when ready for review

12.5 Podcast and Video Workflows

Content creators have particularly repetitive workflows. Most of them can be automated.

Podcast show notes generator:

Given an audio transcript (you can generate these with tools like Whisper):

```
Read the transcript at /clawd/podcasts/ep-42-transcript.txt
```

Generate:

1. Episode summary (150 words max)
2. Key takeaways (5 bullets)
3. Timestamps for major topic changes
4. Guest bio section (research guest name via web search)
5. Resources mentioned in the episode (extract URLs/books/tools mentioned)
6. SEO-friendly title options (5 variants)

```
Save to /clawd/podcasts/ep-42-show-notes.md
```

YouTube video preparation:

```
Read the script at /clawd/videos/tutorial-automation/script.md
```

Generate:

1. Video description (250-300 words, YouTube-optimized)
2. Tags (20-30 relevant tags)
3. Chapter timestamps (based on script sections)
4. 5 title options (click-optimized)
5. Twitter thread to promote the video (5-7 tweets)
6. LinkedIn post to promote the video

```
Save to /clawd/videos/tutorial-automation/publish-package.md
```

Video transcription processor:

Every time a new .txt file appears in /clawd/transcripts/:

1. Read it and identify the content type (podcast, tutorial, interview, other)
2. Run the appropriate processing template
3. Generate show notes or description
4. Archive the raw transcript to /clawd/transcripts/archive/
5. Alert me when processing is complete

12.6 Case Study: The Content Machine

Here's a complete content production system built on OpenClaw.

The Content Machine manages content from idea to published.

Directory structure:

```
/clawd/content/  
├─ ideas.md           # Raw idea capture  
├─ calendar.md        # Publishing calendar  
├─ queue/             # Approved, ready to write
```

```
|— drafts/           # Work in progress
|— review/          # Ready for human review
|— approved/        # Ready to publish
|— published/       # Already published
```

Monday automation (cron):

```
0 8 * * 1:
1. Read ideas.md and calendar.md
2. Identify what needs to be drafted this week
3. Create draft briefs for each piece
4. Move approved items from queue/ to active projects
5. Send me the week's content plan via WhatsApp
```

Daily drafting automation:

```
0 9 * * 1-5:
For each brief in /clawd/content/queue/:
1. Run the research phase (web search + fetch)
2. Generate a first draft
3. Move to /clawd/content/review/
4. Send me a summary of what's ready to review
```

Publishing pipeline:

```
When I move a file from review/ to approved/:
1. Final formatting check
2. Word count
3. SEO analysis (keyword density, title length, meta description)
4. Generate social media posts for each platform
5. Schedule everything in publishing calendar
6. Alert me of scheduled publish times
```

This runs mostly on autopilot. Your job: review drafts and move them from `review/` to `approved/` . Everything else is automated.

Chapter 13: Personal Life Automation

The business case for automation is obvious. The personal case is even more compelling — it's your life, and doing repetitive personal tasks manually is time you're never getting back.

13.1 Morning Briefings

A good morning briefing gives you everything you need to start the day in under 2 minutes. Here's how to build one.

Components:

1. **Weather** — Today's forecast for your location
2. **Calendar** — Your meetings and commitments today
3. **Tasks** — Top 3 priorities (from your task list)

4. **News** — Headlines you care about (filtered, not fire-hosed)
5. **Alerts** — Anything your automation system flagged overnight

Implementation:

Create `/Users/austenallred/clawd/scripts/morning-briefing.py` :

```
#!/usr/bin/env python3
"""
Morning briefing generator.
Reads from various sources and compiles a daily briefing.
"""

import json
from pathlib import Path
from datetime import datetime, timedelta

WORKSPACE = Path("/Users/austenallred/clawd")
TASKS_FILE = WORKSPACE / "tasks/current.md"
ALERTS_LOG = WORKSPACE / "logs/alerts.log"
BRIEFING_DIR = WORKSPACE / "briefings"

def read_tasks():
    if TASKS_FILE.exists():
        content = TASKS_FILE.read_text()
        # Extract unchecked tasks (lines starting with "- [ ]")
        tasks = [
            line.strip()[6:] # Remove "- [ ] "
            for line in content.splitlines()
            if line.strip().startswith("- [ ]")
        ]
        return tasks[:3] # Top 3
    return []

def read_recent_alerts():
    if not ALERTS_LOG.exists():
        return []

    cutoff = datetime.now() - timedelta(hours=12)
    alerts = []

    with open(ALERTS_LOG) as f:
        for line in f:
            try:
                # Parse timestamp from log line
                ts_str = line.split(" ")[0]
                ts = datetime.fromisoformat(ts_str)
                if ts > cutoff:
                    alerts.append(line.strip())
            except (ValueError, IndexError):
                continue
```

```

    return alerts

def write_briefing(content):
    BRIEFING_DIR.mkdir(parents=True, exist_ok=True)
    date_str = datetime.now().strftime("%Y-%m-%d")
    briefing_file = BRIEFING_DIR / f"{date_str}.md"
    briefing_file.write_text(content)
    print(content)

if __name__ == "__main__":
    date_str = datetime.now().strftime("%A, %B %-d, %Y")
    tasks = read_tasks()
    alerts = read_recent_alerts()

    lines = [f"# Morning Briefing - {date_str}\n"]

    if tasks:
        lines.append("## Top Priorities")
        for i, task in enumerate(tasks, 1):
            lines.append(f"{i}. {task}")
        lines.append("")

    if alerts:
        lines.append("## 🚨 Overnight Alerts")
        for alert in alerts[:3]: # Cap at 3
            lines.append(f"- {alert}")
        lines.append("")

    write_briefing("\n".join(lines))

```

The cron job (in `openclaw.json`):

```

{
  "schedule": "0 7 * * 1-5",
  "task": "Run the morning briefing: 1) Execute /clawd/scripts/morning-briefing.py.
2) Fetch today's weather for Austin, TX using web_search 'weather Austin TX today'.
3) Check calendar at /clawd/data/calendar.json for today's events. 4) Combine
everything into a formatted WhatsApp message and send it to me."
}

```

Making it better: Add personal touches. What information makes YOUR morning better? Add it. The briefing is for you.

13.2 Financial Tracking

This is one of the highest-value personal automations. Most people have a vague sense of where their money goes. Automated tracking makes it precise.

Expense log format (`/Users/austenallred/clawd/data/expenses.json`):

```
{
  "expenses": [
    {
      "date": "2026-02-22",
      "amount": 45.30,
      "merchant": "Whole Foods",
      "category": "groceries",
      "notes": ""
    }
  ]
}
```

Receipt processor:

Using a phone node with camera:

When I take a photo and say "log this receipt":

1. Analyze the image with vision model
2. Extract: merchant name, total amount, date, itemized list (if visible)
3. Categorize: groceries, dining, transportation, utilities, entertainment, other
4. Add to /clawd/data/expenses.json
5. Confirm via WhatsApp: "Logged: \$45.30 at Whole Foods (groceries)"

Monthly spending report:

```
{
  "schedule": "0 9 1 * *",
  "task": "Monthly spending report: Read all expenses from /clawd/data/expenses.json for last month. Calculate: total spending by category, total overall, comparison to previous month (if data exists), top 5 merchants by spend. Write report to /clawd/reports/spending/YYYY-MM.md. Send me a WhatsApp summary with the biggest numbers."
}
```

Budget alerts:

```
{
  "schedule": "0 9 * * 1",
  "task": "Weekly budget check: Read expenses.json for current month. Compare spending by category to budget limits in /clawd/data/budget.json. If any category is above 80% of budget, send me a WhatsApp warning. If any is above 100%, send a critical alert."
}
```

13.3 Health and Fitness Logging

Consistency is everything in health habits, and automation makes consistency much easier.

Daily health log:

Create `/Users/austenallred/clawd/data/health-log.json` :

```
{
  "entries": [
    {
      "date": "2026-02-22",
      "sleep_hours": 7.5,
      "sleep_quality": 8,
      "exercise": {"type": "run", "duration_min": 35, "distance_km": 5.2},
      "weight_kg": 78.5,
      "mood": 7,
      "notes": "Good energy, focused work day"
    }
  ]
}
```

Evening health logging prompt:

Add to your WhatsApp shortcuts (via SOUL.md keyword routing):

When I start a message with "health log":
Ask me these questions one at a time:

1. Sleep hours last night?
2. Sleep quality (1-10)?
3. Any exercise today? (type + duration + distance if applicable)
4. Weight? (or "skip")
5. Mood (1-10)?
6. Any notes?

Then add the entry to `/clawd/data/health-log.json` and confirm.

Weekly health summary:

```
{
  "schedule": "0 8 * * 0",
  "task": "Weekly health summary: Read last 7 days of health-log.json. Calculate: average sleep, sleep trend (improving/declining), exercise frequency and type breakdown, mood average and trend, weight trend. Write report to /clawd/reports/health/YYYY-WW.md. Send me a WhatsApp summary with highlights and trends."
}
```

13.4 Travel and Calendar Management

Trip planner:

I'm traveling to New York March 15-18.

Research:

1. Flight options from Austin to NYC (just search, don't book)

2. Weather in NYC March 15–18
3. Top 5 hotels near Midtown under \$300/night (review them)
4. 3 restaurant recommendations near Midtown

Save a trip brief to `/clawd/trips/nyc-march-2026.md` with all findings.

Pre-trip checklist generator:

Generate a packing and prep checklist for my NYC trip.

Reference:

- Trip brief: `/clawd/trips/nyc-march-2026.md`
- Weather: check forecast for NYC March 15–18
- Past trip: `/clawd/trips/nyc-2025.md` (what I forgot last time)

Save checklist to `/clawd/trips/nyc-march-2026-checklist.md`

Calendar intelligence:

If your calendar data is accessible (exported as JSON or ICS):

```
{
  "schedule": "0 18 * * *",
  "task": "Tomorrow's prep: Read calendar events for tomorrow from
/clawd/data/calendar.json. For each meeting: check if there's any relevant context
in /clawd/ (meeting notes, project docs, client files). Generate a prep summary for
each meeting. Save to /clawd/meeting-prep/YYYY-MM-DD.md. Send me a WhatsApp summary
of tomorrow's meetings."
}
```

13.5 Home Automation Integration

If you have smart home devices, you can tie them into OpenClaw via their APIs.

Home state management:

```
/clawd/data/home-state.json:
{
  "mode": "home", // home, away, sleep, guests
  "last_updated": "2026-02-22T18:00:00Z"
}
```

Mode triggers:

When location changes (from the node network pattern in Chapter 9):

```
When my iPhone is within 0.5 miles of home and home-state.mode is "away":
1. Set home-state.mode to "home"
2. (Via home automation API) turn on welcome lights
3. Check if any deliveries are expected today
4. Send me a "you're home" notification with any pending alerts
```

Morning routine trigger:

```
When home--state.mode changes to "home" between 5am-10am:
1. (Via API) Start coffee maker
2. Set thermostat to daytime temperature
3. Play morning playlist
4. Trigger morning briefing
```

Note: These home automation integrations require your devices' APIs. OpenClaw handles the logic; the actual device control goes through whatever API your smart home devices expose (usually via HTTP calls with `exec / web_fetch`).

13.6 Case Study: The Personal Assistant Stack

Here's how all these pieces fit together into a coherent personal assistant.

The stack:

Inputs:		Processing:		Outputs:
WhatsApp commands	→	Kelly Router	→	WhatsApp messages
Cron triggers	→	Daily automation	→	File creation
Location changes	→	Context detection	→	System notifications
Camera/photo	→	Vision processing	→	Data logging

A day in the life:

7:00 AM — Morning briefing fires. You get a WhatsApp with:

- Today's weather
- 3 top tasks
- Meeting prep for your 10am call
- Any overnight alerts (none today)

9:15 AM — You take a photo of a receipt and message "log this". OpenClaw extracts the data and adds it to your expense log.

9:45 AM — You message: "research competitors' new pricing model". Research sub-agents spawn, do the work, send you a summary in 8 minutes.

10:00 AM — Meeting. Your pre-meeting brief was generated last night.

12:30 PM — You message: "draft a reply to the Acme proposal request". OpenClaw reads the relevant context files, generates a draft, sends it for review.

5:00 PM — EOD wrap-up fires. Reads the day's memory logs, sends you a WhatsApp summary of what was accomplished.

9:00 PM — You say "health log" and answer the evening prompts.

10:30 PM — Quiet hours activate. No more WhatsApp alerts until 7am.

The system does hundreds of small things. You never think about most of them. That's the goal.

PART V: BUILDING YOUR AUTOMATION SYSTEM

Chapter 14: Designing Your Stack

You've got the tools. You've seen the patterns. Now how do you actually design a system that works for *your* life?

14.1 Start Small, Think Big

The biggest mistake people make with automation: trying to automate everything at once.

The second biggest mistake: automating things before they have a stable process.

The rule: Before you automate a process, run it manually at least 5-10 times. Understand where it breaks, what varies, what the edge cases are. Then automate.

If you automate a process that's still being figured out, you'll spend more time updating your automation than actually doing the work.

The starting point: What do you do every single day that's repetitive and annoying? Start there. Not the complex multi-step pipeline. The stupid daily grind that you hate.

For most people, it's one of:

- Checking the same websites/dashboards
- Compiling information from multiple sources into a report
- Sending similar messages/emails repeatedly
- Organizing files that pile up

Pick one. Automate it completely. See what you learn. Then pick the next.

14.2 Identifying Automation Candidates

Not everything should be automated. Here's a framework for deciding.

Good automation candidates:

✅ Happens regularly (daily, weekly, monthly) ✅ Has a consistent process (same steps every time) ✅ Takes more than 5 minutes ✅ Doesn't require nuanced judgment ✅ Has clear success criteria (you know when it's done right) ✅ The cost of errors is manageable

Bad automation candidates:

❌ One-off tasks ❌ Highly variable processes ❌ Requires complex human judgment ❌ High stakes if it goes wrong (customer-facing, financial) ❌ The process changes frequently ❌ Takes less than 2 minutes (automation overhead isn't worth it)

The 10x rule: If it takes you 10 hours per month, worth automating even if setup takes 10 hours. You break even after one month. If it takes 1 hour per month, you need setup to take less than 2 hours to be worth it.

14.3 The Automation Decision Framework

For any candidate task, ask:

1. Is it well-defined? Can you write out exact steps? If you struggle to describe what you do, automation will struggle too.

2. What's the error cost? If the automation does it wrong, how bad is it?

- Low: You see a slightly wrong report and fix it manually
- Medium: Work needs redoing
- High: Customer impact, financial loss, data loss

For high error cost: use automation for drafts, keep human approval step.

3. What's the frequency? Daily tasks get top priority. Monthly tasks need to save significant time to be worth the setup.

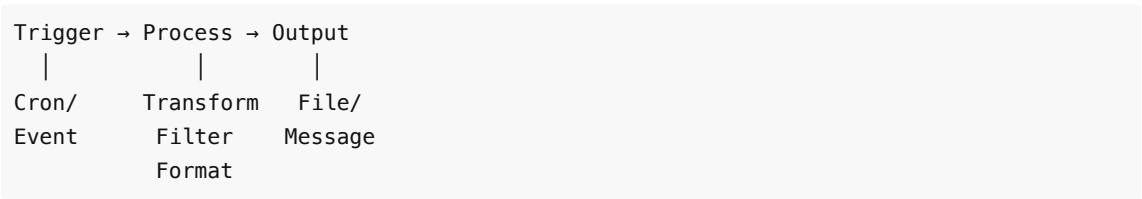
4. Does it touch external systems? Email, customer databases, financial systems — anything that affects other people or critical data gets extra scrutiny.

5. Can you test it safely? If you can't test it without consequences, be very careful. Set up a test environment.

14.4 Architecture Patterns

Pattern 1: The Simple Pipeline

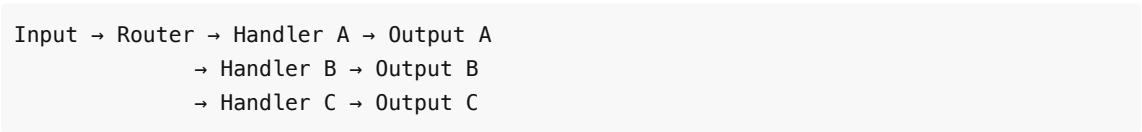
Best for: sequential data processing with a clear start and end.



Example: Daily sales report. Cron triggers → reads CSV → generates markdown → sends WhatsApp.

Pattern 2: The Hub and Spoke

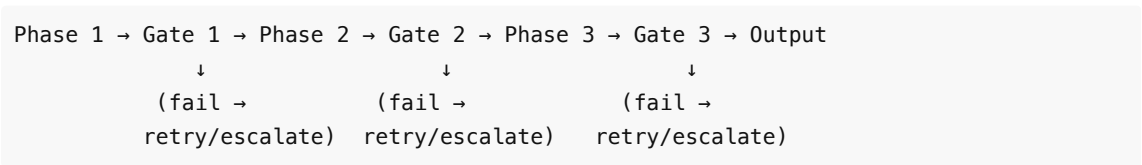
Best for: routing different types of requests to appropriate handlers.



Example: Incoming WhatsApp commands. Route to: research agent, writer agent, or data agent.

Pattern 3: The Pipeline with Gates

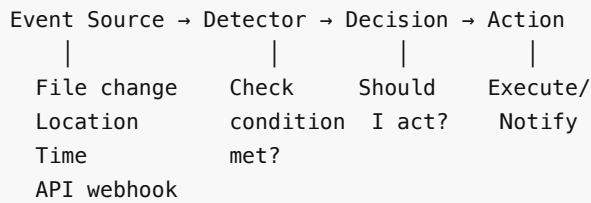
Best for: complex, multi-phase work where quality at each stage matters.



Example: The Kelly Router software factory pattern.

Pattern 4: The Event-Driven System

Best for: real-time reactions to state changes.



Example: Location-triggered home automation, price change alerts.

Combining Patterns

Real systems use multiple patterns. The Kelly Router uses:

- Hub and Spoke (routing different project types)
- Pipeline with Gates (each project phase)
- Simple Pipeline (individual tasks within phases)
- Event-Driven (heartbeat monitoring, alerts)

14.5 Version Control for Automation

Your automation code is code. Treat it like code.

What to version control:

- All scripts in `/clawd/scripts/`
- Templates in `/clawd/templates/`
- Protocol files (SOUL.md, AGENTS.md, MEMORY.md)
- `openclaw.json`
- README that explains how your system works

What NOT to version control:

- Credentials and secrets (`/clawd/secrets/`)
- Generated outputs (`/clawd/reports/` , `/clawd/drafts/`)
- Log files (`/clawd/logs/`)
- Transient data (`/clawd/data/` — depends on content)

Setting up git:

```
cd /Users/austenallred/clawd
git init
echo "secrets/" >> .gitignore
echo "logs/" >> .gitignore
echo "*.log" >> .gitignore
echo "reports/" >> .gitignore
echo "drafts/" >> .gitignore
git add .
git commit -m "Initial automation stack"
```

Commit messages that help you later:

```
Add weekly CI monitor – checks competitor pricing every Monday
Fix: morning briefing was running at 1am UTC (was 7am local)
Update SOUL.md: prefer bullet points over paragraphs
Add: expense logging via WhatsApp photo
```

When things break: With git, you can see exactly what changed:

```
git log --oneline -10
git diff HEAD~1 HEAD openclaw.json
```

This has saved me multiple times when a cron job mysteriously stopped working after a "small" config change.

14.6 Documentation That You'll Actually Read

Most documentation never gets written or never gets read. The key: write for your future self who forgot everything.

The AGENTS.md README:

At the top of your AGENTS.md, add a "quick start" section:

```
## Quick Start

**Running:** Gateway should always be running. Check: `openclaw gateway status`

**Main entry point:** This agent routes work. Talk to it via WhatsApp.

**Key commands:**
- "status" → system status
- "research [topic]" → triggers research-lead
- "help" → lists available commands

**Things currently running:**
- Cron: morning briefing (7am weekdays)
- Cron: heartbeat (every 2 hours)
- Cron: weekly CI report (Monday 9am)
```

Document decisions, not just processes:

Instead of just: "Use research-lead for all research tasks"

Write: "Use research-lead for all research tasks. (Changed Feb 2026 because the main agent was trying to do research + routing simultaneously and making poor decisions. This split works much better.)"

The "why" ages much better than the "what."

Keep a CHANGELOG:

CHANGELOG

2026-02-22

- Added expense tracking via WhatsApp photo
- Fixed: morning briefing now correctly uses local timezone

2026-02-15

- Added weekly health summary report
- Added location-based home automation triggers

2026-02-01

- Initial Kelly Router setup
- Research-lead, project-lead, test-lead agents configured

Chapter 15: Troubleshooting & Optimization

Everything breaks eventually. When it does, you need to find the problem fast and fix it without destroying anything else. This chapter is your guide to keeping your automation system healthy.

15.1 The Most Common Failures

In rough order of frequency:

1. Gateway not running

Symptom: Nothing works. Messages not processed. Cron jobs not firing.

Check:

```
openclaw gateway status
```

Fix:

```
openclaw gateway start
```

Why it happens: Gateway crashes, machine reboot without autostart configured, config error on restart.

Prevention: Set up Gateway as a system service that auto-restarts. On macOS:

```
# Check if LaunchAgent is configured
ls ~/Library/LaunchAgents/ | grep openclaw
```

2. Wrong file path

Symptom: "No such file or directory" errors. Agents reporting they can't find things.

Check:

```
ls -la /Users/austenallred/clawd/  
find /Users/austenallred/clawd -name "the-file-you-want"
```

The problem is usually a typo in the path, wrong directory level, or case sensitivity (macOS is case-insensitive by default, but beware when working across systems).

Fix: Correct the path. Use `find` to locate files you're not sure about.

3. Cron job running at wrong time

Symptom: Scheduled tasks running at unexpected times.

Check: Is your Gateway running in UTC? What's the local timezone offset?

```
date && date -u
```

Fix: Convert your desired local time to UTC and use that in the cron expression.

4. Context overflow in long-running agent

Symptom: Agent gives irrelevant responses, forgets earlier instructions, or produces incoherent output.

Fix:

- For sub-agents: spawn fresh agents for long tasks
- For main sessions: write current state to file, start fresh session, point to file

Prevention: Design tasks to be self-contained and document state in files throughout.

5. Sub-agent didn't report back

Symptom: You spawned a sub-agent and never heard from it.

Check:

```
{  
  "tool": "subagents",  
  "action": "list"  
}
```

Check if the session is still running, completed, or errored.

Fix: Check the session log for errors. If it completed without reporting, read the output files directly.

6. Web fetch returning empty or wrong content

Symptom: `web_fetch` returns empty string or HTML without the content you expected.

Check:

- Is the content loaded by JavaScript? (Use browser instead)
- Does the site require login? (Use Chrome relay)
- Is the URL correct? (Fetch it manually in a browser)

Fix: Switch from `web_fetch` to browser control if content is JS-loaded.

7. edit failing because text doesn't match

Symptom: "Text not found in file" when trying to edit.

Fix: Read the file fresh immediately before the edit. Copy the exact text. Check for hidden whitespace.

```
cat -A /path/to/file | grep -n "the text you want"
```

The `-A` flag shows invisible characters.

8. Permission denied

Symptom: Can't read/write files or execute scripts.

Check:

```
ls -la /path/to/file  
whoami
```

Fix:

```
chmod 644 /path/to/file      # for readable files  
chmod 755 /path/to/script    # for executable scripts  
chmod 600 /path/to/secrets   # for sensitive files
```

15.2 Debugging Strategies

The Binary Search Approach

When a multi-step pipeline fails, you don't know which step is broken. Use binary search:

1. Run step 1 manually. Does it work?
2. Run steps 1-2. Does that work?
3. Continue until you find the failing step.

For long pipelines, start in the middle rather than at the beginning.

Check the Logs

Your logs exist for exactly this moment.

```
# What happened recently?  
tail -50 /Users/austenallred/clawd/logs/cron-history.log  
  
# Any errors?  
grep -i error /Users/austenallred/clawd/logs/*.log | tail -20  
  
# What did the last run produce?  
ls -lt /Users/austenallred/clawd/reports/ | head -5
```

Reproduce the Problem Manually

When a cron job fails, try running the task manually:

```
Manually run the morning briefing task right now and show me the full output including any errors.
```

If it works manually but not in cron, the issue is likely:

- Environment variables (PATH, etc.)
- Working directory
- File permissions under the cron user

Add More Logging

When you're debugging, add verbose logging temporarily:

In Python scripts:

```
import logging
logging.basicConfig(level=logging.DEBUG)
```

In shell scripts:

```
set -x # Print every command before executing
```

In cron tasks: add "report the exact output and any errors" to the task description.

The Minimal Reproduction

Create the smallest possible version of the failing task and test that. Once the minimal version works, add complexity back gradually.

15.3 Performance Tuning

Slow Cron Jobs

Measurement: Add timing to your scripts.

```
time python3 /Users/austenallred/clawd/scripts/slow-script.py
```

Common culprits:

- Web requests with no timeout (can hang indefinitely)
- Reading very large files you don't need entirely
- Sequential operations that could be parallel

Web request timeouts:

```
import requests
response = requests.get(url, timeout=10) # 10 second timeout
```

Parallel execution where possible: Instead of fetching 5 URLs sequentially, spawn 5 sub-agents in parallel.

Context Loading Overhead

Every session that loads large context files (SOUL.md, MEMORY.md, AGENTS.md) pays a startup cost. Keep these files focused and not too long.

MEMORY.md maintenance: If MEMORY.md grows to hundreds of lines, things it records weeks ago may not be worth loading every session. Do periodic cleanup:

```
Review MEMORY.md. Archive entries older than 90 days that are no longer relevant to
/clawd/memory/archive/memory-archive-2026-Q1.md. Keep only current, actionable
knowledge.
```

Sub-Agent Efficiency

Sub-agents have overhead: initialization, context loading, tool calls.

Good sub-agent tasks: "Do this entire well-defined thing and report back"

Inefficient sub-agent usage: Spawning a sub-agent to do one small thing you could have done with a direct tool call.

Sub-agents shine for parallelism and isolation. For sequential single tasks, use the main agent's tools directly.

15.4 Cost Optimization

AI models cost money per token. An automation system that runs constantly can accumulate significant API costs.

Where costs come from:

- Input tokens: everything in the context window
- Output tokens: everything the model generates
- Sub-agent sessions: each has its own context overhead

Reducing input costs:

Large context files cost more. Keep SOUL.md, MEMORY.md, and AGENTS.md concise. Use summarization for memory maintenance.

For cron jobs, use simpler/cheaper models when the task doesn't need maximum intelligence:

In `openclaw.json` :

```
{
  "model": "anthropic/claude-sonnet-4-5",
  "cron": [
    {
      "schedule": "0 7 * * *",
      "task": "...",
      "model": "anthropic/claude-haiku-3-5"
    }
  ]
}
```

(Check current docs for model configuration options — this API evolves.)

Reducing output costs:

Verbose responses cost more. For automated tasks that write to files (not to the user), instruct the agent to be terse:

```
Execute the pipeline. Log results to file.  
Do not produce verbose output – a one-line completion summary is sufficient.
```

Smart scheduling:

Don't run tasks more frequently than needed. A heartbeat every 15 minutes vs. every 2 hours is an 8x cost difference for the same coverage.

Check if a 1-hour heartbeat gives you adequate response time vs. a 30-minute one. Usually the answer is yes.

15.5 Security Hardening

An AI agent with tool access can do a lot of damage if it's compromised or behaves unexpectedly.

Principle of Least Privilege

The agent should only have access to what it needs.

Tool policy in `openclaw.json` :

```
{  
  "tools": {  
    "exec": {  
      "security": "allowlist",  
      "allowedCommands": ["python3", "node", "git", "ls", "cat", "find"]  
    },  
    "exec.elevated": false  
  }  
}
```

An allowlist means only the specified commands can run. Anything not on the list is blocked.

Secrets Management

Never put credentials in:

- Task descriptions in `openclaw.json`
- SOUL.md, MEMORY.md, AGENTS.md
- Log files
- Agent conversation history

Store them in:

```
/Users/austenallred/clawd/secrets/  
└─ api-keys.json    # chmod 600
```



```
| credentials.json # chmod 600
| tokens.json      # chmod 600
```

With permissions:

```
chmod 700 /Users/austenallred/clawd/secrets/
chmod 600 /Users/austenallred/clawd/secrets/*.json
```

Agents reference these files by path and read them at runtime. The content never appears in logs or conversation history.

Review Automation That Affects Others

Anything that sends messages to third parties, modifies customer data, or makes financial transactions needs a human approval step.

Pattern: Always generate drafts for these. Never automate the final send.

```
Generate an invoice for client ACME Corp based on /clawd/projects/acme/timelog.md.
Save to /clawd/drafts/invoices/acme-2026-02.md for my review.
Do NOT send it. I'll review and send manually.
```

Monitor Your System's Behavior

Occasionally audit what your automation has been doing:

```
Review all cron job logs from the past week at /clawd/logs/cron-history.log.
Review all WhatsApp messages sent via /clawd/logs/outbound-messages.log.
Report: anything unusual, unexpected actions, high-frequency patterns.
```

If your agent is doing things you didn't expect, you want to know.

15.6 Keeping Things Running

The final challenge: long-term maintenance of an automation system.

The Decay Problem

Websites change. APIs update. File formats shift. People leave and join. Your automation was built against a world that no longer exists.

Mitigation:

- Build alerts when key automation fails (not just when it runs)
- Do monthly audit of what's actually working
- Build "last successful run" timestamps into logs and alert when something hasn't run recently

The Documentation Debt Problem

You built something clever. You didn't document it. Six months later you have no idea how it works.

Mitigation:

- Write at time of creation, not after

- The CHANGELOG is your friend — write it as you go
- When something breaks and you fix it, document what broke and why in MEMORY.md

The Version Compatibility Problem

OpenClaw updates. Models update. APIs change. Your carefully tuned prompts might not work as well with a new model.

Mitigation:

- Pin model versions when stability matters more than capability
- Test after updates before putting into production
- Keep working configs in git so you can roll back

Regular Health Checks

Set up a monthly automation audit:

```
{
  "schedule": "0 10 1 * *",
  "task": "Monthly system audit: Check all cron jobs ran successfully in the past month (review /clawd/logs/cron-history.log). Identify any that haven't run or have been failing. Check disk usage. Review /clawd/data/ for files that are stale or no longer needed. Check all scripts in /clawd/scripts/ still exist and are executable. Report findings and recommendations."
}
```

This is your system telling you what it thinks is wrong with itself. Listen to it.

APPENDICES

Appendix A: Tool Reference

Complete reference for all OpenClaw tools. Parameters marked with ***** are required.

read

Read file contents.

Parameter	Type	Description
file_path*	string	Path to file
offset	number	Line to start reading from (1-indexed)
limit	number	Maximum lines to read

Notes:

- Maximum output: 2000 lines or 50KB

- Images (jpg, png, gif, webp) are sent as attachments
- Use offset/limit for large files

Example:

```
{"tool": "read", "file_path": "/Users/austenallred/clawd/MEMORY.md"}
```

write

Create or overwrite a file.

Parameter	Type	Description
file_path*	string	Path to file
content*	string	Content to write

Notes:

- Creates parent directories automatically
- Completely replaces existing file content
- For partial updates, use `edit`

edit

Make precise edits to a file by replacing exact text.

Parameter	Type	Description
file_path*	string	Path to file
oldText*	string	Exact text to find
newText*	string	Replacement text

Notes:

- `oldText` must match exactly (whitespace, newlines, case)
- Read the file fresh before editing to ensure match
- Fails if `oldText` not found

exec

Run shell commands.

Parameter	Type	Description
command*	string	Shell command to run
workdir	string	Working directory
background	boolean	Run in background

yieldMs	number	Ms to wait before backgrounding
timeout	number	Timeout in seconds
pty	boolean	Use pseudo-terminal (for TTY apps)
elevated	boolean	Run with elevated permissions
security	string	Security mode: deny, allowlist, full
env	object	Environment variables

Notes:

- Default working directory is workspace root
- Background processes continue after response
- Use `process` tool to monitor background processes

process

Manage background exec sessions.

Parameter	Type	Description
action*	string	list, poll, log, write, send-keys, kill
sessionId	string	Session ID (required for most actions)
timeout	number	For poll: wait ms before returning
limit	number	Log line limit
offset	number	Log line offset
data	string	Data to write (for write action)
keys	array	Keys to send (for send-keys action)

web_search

Search the web using Brave Search API.

Parameter	Type	Description
query*	string	Search query
count	number	Results to return (1-10, default 5)
country	string	2-letter country code (e.g., "US")
freshness	string	pd (day), pw (week), pm (month), py (year)
search_lang	string	ISO language code for results

web_fetch

Fetch and extract content from a URL.

Parameter	Type	Description
url*	string	HTTP/HTTPS URL
extractMode	string	"markdown" (default) or "text"
maxChars	number	Maximum characters to return

Notes:

- For JavaScript-rendered content, use `browser` instead
- Markdown mode preserves formatting; text mode strips it

browser

Control web browser.

Parameter	Type	Description
action*	string	status, start, stop, open, navigate, snapshot, screenshot, act, console, pdf, tabs, focus, close
profile	string	"openclaw" (isolated) or "chrome" (relay)
targetId	string	Specific tab to target
targetUrl	string	URL for navigate/open
refs	string	"role" (default) or "aria"
request	object	Action request (for act action)
compact	boolean	Compact snapshot output
fullPage	boolean	Full page screenshot
type	string	Screenshot type: png or jpeg

Action request kinds:

- `click` — Click element by ref
- `type` — Type text into element
- `fill` — Fill field (clears first)
- `press` — Press keyboard key
- `select` — Select dropdown option
- `drag` — Drag from one ref to another
- `hover` — Hover over element
- `evaluate` — Run JavaScript
- `wait` — Wait for text to appear/disappear
- `scroll` — Scroll the page

canvas

Control the canvas UI.

Parameter	Type	Description
action*	string	present, hide, navigate, eval, snapshot
url	string	URL for present/navigate
javascript	string	JavaScript for eval
width	number	Canvas width
height	number	Canvas height

nodes

Interact with paired nodes.

Parameter	Type	Description
action*	string	status, describe, notify, camera_snap, camera_list, camera_clip, screen_record, location_get, run, invoke
node	string	Node name/ID
facing	string	Camera: front, back, both
durationMs	number	Duration for clips/recordings
command	array	Command and args for run action
title	string	Notification title
body	string	Notification body
delivery	string	system, overlay, auto
desiredAccuracy	string	coarse, balanced, precise

message

Send messages through channels.

Parameter	Type	Description
action*	string	send, react, poll
target	string	Recipient (phone number or username)
message	string	Message text
media	string	Media URL or local path

caption	string	Media caption
replyTo	string	Message ID to reply to

tts

Convert text to speech.

Parameter	Type	Description
text*	string	Text to speak
channel	string	Channel for output format

Notes: After calling tts, agent should respond with `NO_REPLY` to avoid duplicate text message.

subagents

Manage sub-agent sessions.

Parameter	Type	Description
action*	string	list, kill, steer
target	string	Session label or ID
message	string	Message for steer action
recentMinutes	number	For list: filter by recency

image

Analyze images with vision model.

Parameter	Type	Description
image	string	Single image path or URL
images	array	Multiple images (up to 20)
prompt	string	What to analyze/look for
model	string	Vision model to use

Appendix B: Skill Library

Skills are extensions to OpenClaw's core capabilities, available through ClawHub. They provide specialized functionality for specific domains.

This appendix covers the skill pattern and key available skills. For the full current catalog, visit the ClawHub documentation.

What Skills Are

Skills are packaged tool configurations, prompts, and integrations that extend what OpenClaw can do. They're typically installed into your workspace and referenced in your configuration.

Unlike core tools (which are built into OpenClaw), skills are:

- Community-developed
- Installable/removable
- Domain-specific
- Versioned independently

Skill Configuration Pattern

Skills are referenced in `T00LS.md` (your local tool notes):

Installed Skills

finance-tracker

Tracks expenses and generates financial reports.

Trigger: "log expense" or "spending report"

Data location: /clawd/data/finance/

Config: /clawd/config/finance-tracker.json

content-pipeline

Manages content from idea to published.

Trigger: "draft [topic]" or "content status"

Data location: /clawd/content/

Config: /clawd/config/content-pipeline.json

Core Skill Patterns

Even without installing packaged skills, you can build skill-like patterns by combining the core tools. Here are the most useful patterns:

Web Research Skill

Trigger: Any task starting with "research"

Pattern:

1. Parse the research topic from the request
2. Generate 3-5 search queries
3. Run all queries with `web_search`
4. Fetch top 2-3 results from each query
5. Synthesize findings into structured report
6. Save to `/clawd/research/{topic}-{date}.md`

Template in SOUL.md:

Research Protocol

When asked to "research [topic]":

1. Generate 5 search queries covering different angles
2. Run all 5 searches
3. Fetch top 2 URLs from each search (total 10 pages max)
4. Synthesize: key findings, sources, confidence level
5. Save to /clawd/research/{topic}-YYYY-MM-DD.md
6. Report: 3-bullet summary + link to full report

Data Pipeline Skill

Trigger: Any task involving CSV/JSON data transformation

Pattern:

1. Identify input file format and location
2. Identify desired output format and location
3. Write a Python transformation script
4. Execute it
5. Validate output
6. Report summary

Browser Extraction Skill

Trigger: "extract data from [URL]"

Pattern:

1. Navigate to URL with browser
2. Snapshot the page
3. Identify data structure
4. Use evaluate to extract data as JSON
5. Save to appropriate data file

Building Custom Skills

You can build your own skill-like behaviors by defining them in your context files.

In SOUL.md:

Skills

expense-logger

Triggered when I say "log expense" or send a photo of a receipt.

Process:

1. Extract amount, merchant, date from message or image
2. Categorize using rules in /clawd/config/expense-categories.json
3. Append to /clawd/data/expenses.json
4. Confirm with brief WhatsApp response

daily-brief

Triggered every weekday at 7am via cron.

Process:

1. Get weather for Austin TX
2. Read top 3 from /clawd/tasks/current.md
3. Read today's events from /clawd/data/calendar.json
4. Read overnight alerts from /clawd/logs/alerts.log
5. Format as brief WhatsApp message (< 200 words)

In AGENTS.md:

Skill Routing

When incoming requests match these patterns, route to appropriate handler:

- "research *" → *research-lead agent*
- "build *" or "code *" → *project-lead agent*
- "test *" or "review *" → *test-lead agent*
- "log *" or "track *" → *data-agent (inline, no sub-agent needed)*

Appendix C: Resources

Official Resources

OpenClaw documentation: The canonical source for current API docs, installation instructions, and configuration reference. Always check the official docs for the most current information — this book covers patterns and concepts, but implementation details evolve.

ClawHub: The package registry for OpenClaw skills. Find community-built skills, submit your own, and explore what others have built.

OpenClaw GitHub: For reporting bugs, contributing to core, and following development.

Cron Expression Tools

crontab.guru — The best visual cron expression generator. Type what you want and it shows you the expression and confirms what it does.

Cron format quick reference:

```
# Every 15 minutes
*/15 * * * *

# Every hour at minute 0
0 * * * *

# 9am every weekday
0 9 * * 1-5

# 9am Monday
0 9 * * 1

# First of month at midnight
0 0 1 * *
```

```
# Every 6 hours
0 */6 * * *
```

Useful Shell Tools

These shell commands appear throughout this book and are worth knowing well:

```
# Find files
find /path -name "*.md" -newer reference-file
find /path -name "*.log" -mtime +30 # older than 30 days

# Text processing
grep -r "pattern" /path/ # search in files
awk '{print $1, $3}' file.csv # extract columns
jq '.key' data.json # parse JSON

# File info
ls -la # detailed listing
du -sh /path # directory size
df -h # disk usage
wc -l file # line count

# Process management
ps aux | grep openclaw
kill -9 {pid}

# Permissions
chmod 755 script.sh # executable
chmod 600 secret.txt # private
chown user:group file # change owner
```

Python for Automation

Basic Python patterns used throughout this book:

Reading/writing files:

```
from pathlib import Path

# Read
content = Path("/clawd/data/file.txt").read_text()

# Write
Path("/clawd/data/output.txt").write_text("content")

# Append
with open("/clawd/logs/app.log", "a") as f:
    f.write("new line\n")
```

JSON:

```
import json

# Read
data = json.loads(Path("/clawd/data/config.json").read_text())

# Write
Path("/clawd/data/config.json").write_text(json.dumps(data, indent=2))
```

Date/time:

```
from datetime import datetime, timedelta

today = datetime.now().strftime("%Y-%m-%d")
yesterday = (datetime.now() - timedelta(days=1)).strftime("%Y-%m-%d")
```

Argument parsing:

```
import sys
import argparse

parser = argparse.ArgumentParser()
parser.add_argument("date", help="Date in YYYY-MM-DD format")
parser.add_argument("--verbose", "-v", action="store_true")
args = parser.parse_args()
```

Recommended Reading

Shell scripting:

- "The Linux Command Line" by William Shotts — thorough, free online
- "bash Cookbook" by Albing, Vossen, and Newham — practical recipes

Python for automation:

- "Automate the Boring Stuff with Python" by Al Sweigart — free online, very accessible
- "Python Cookbook" by Beazley and Jones — more advanced

Systems thinking:

- "The Goal" by Eliyahu Goldratt — production systems thinking applied to any workflow
- "Thinking in Systems" by Donella Meadows — mental models for understanding complex systems

Community

The OpenClaw community is the best place to find patterns, get help, and share what you've built. Check the official site for current community links.

When asking for help, include:

- Your `openclaw.json` (redact credentials)

- Relevant context files
- The exact error or unexpected behavior
- What you've already tried

The community is generally helpful. People like seeing what others are building.

Final Note

You've made it through the whole book. If you've been following along and building as you read, you now have:

- A working Gateway with scheduled tasks
- Context files that shape your agent's behavior
- At least one complete automation pipeline
- The mental models to build more

The honest truth about automation: it compounds. The first thing you automate saves you an hour a week. The tenth thing saves you a day a week. The hundredth thing is hard to even measure because you've stopped thinking about all the things you're no longer doing.

Start with the annoying thing. Build the pipeline. Watch it run while you do something else.

That's the whole point.

This handbook was written for the OpenClaw community. If something's wrong, outdated, or you built something cooler than what's described here — open a PR.

End of "Automate Everything: The OpenClaw Handbook"

PLACEHOLDER_FOR_EXPANSION

Appendix D: Automation Cookbook

This appendix is structured differently from the rest of the book. No theory, minimal explanation. Just working recipes — copy, adapt, run.

Each recipe has:

- **Problem:** What you're trying to do
 - **Ingredients:** What you need set up first
 - **Recipe:** The complete implementation
 - **Variations:** How to extend or modify it
-

Recipe 1: Morning Briefing System

Problem: Every morning you spend 20 minutes checking the same five things. Make it automatic.

Ingredients:

- `openclaw.json` with cron configured
- Your task list at `/Users/austenallred/clawd/tasks/current.md`

- Optional: calendar export at `/Users/austenallred/clawd/data/calendar.json`

Recipe:

Create `/Users/austenallred/clawd/scripts/morning-brief.sh` :

```
#!/bin/bash
# morning-brief.sh
# Runs at 7am weekdays. Generates daily briefing.

WORKSPACE="/Users/austenallred/clawd"
DATE=$(date +%Y-%m-%d)
BRIEFING="$WORKSPACE/briefings/$DATE.md"

mkdir -p "$WORKSPACE/briefings"

# Header
cat > "$BRIEFING" << EOF
# Briefing - $(date '+%A, %B %-d, %Y')

Generated: $(date +%H:%M)

EOF

# Tasks section
if [ -f "$WORKSPACE/tasks/current.md" ]; then
    echo "## Top Tasks" >> "$BRIEFING"
    # Extract top 3 unchecked tasks
    grep "^- \[ \]" "$WORKSPACE/tasks/current.md" | head -3 | sed 's/^- \[ \] /- /' >>
"$BRIEFING"
    echo "" >> "$BRIEFING"
fi

# Overnight alerts
if [ -f "$WORKSPACE/logs/alerts.log" ]; then
    RECENT_ALERTS=$(awk -v cutoff="$(date -v-12H +%Y-%m-%dT%H:%M:%S 2>/dev/null ||
date -d '12 hours ago' +%Y-%m-%dT%H:%M:%S)" '$1 > cutoff'
"$WORKSPACE/logs/alerts.log" | wc -l | tr -d ' ')
    if [ "$RECENT_ALERTS" -gt 0 ]; then
        echo "## 🚨 Alerts (last 12h)" >> "$BRIEFING"
        awk -v cutoff="$(date -v-12H +%Y-%m-%dT%H:%M:%S 2>/dev/null || date -d '12 hours
ago' +%Y-%m-%dT%H:%M:%S)" '$1 > cutoff' "$WORKSPACE/logs/alerts.log" | head -5 >>
"$BRIEFING"
        echo "" >> "$BRIEFING"
    fi
fi

cat "$BRIEFING"
```

```
chmod +x /Users/austenallred/clawd/scripts/morning-brief.sh
```

Cron task in `openclaw.json` :

```
{
  "schedule": "0 13 * * 1-5",
  "task": "Morning briefing: Run /Users/austenallred/clawd/scripts/morning-brief.sh
and capture the output. Then search the web for 'weather Austin TX today' and
extract the high/low temperatures and conditions. Combine the script output and
weather into a single WhatsApp message and send it to me. Keep the total message
under 200 words."
}
```

Variations:

- Add `git log --oneline -5` output for your active repos
- Include exchange rates if you work internationally
- Pull latest Hacker News headlines with `web_fetch https://news.ycombinator.com`
- Add `tail -20 /clawd/logs/cron-history.log` to show what ran overnight

Recipe 2: Competitor Price Watcher

Problem: You need to know when a competitor changes their pricing. Currently you check manually, which means you find out weeks late.

Ingredients:

- Competitor pricing page URLs
- Baseline prices stored in JSON

Recipe:

First, capture the baseline. Ask the agent:

Fetch the pricing pages for these competitors:

```
- https://competitor-a.com/pricing
- https://competitor-b.com/pricing
- https://competitor-c.com/pricing
```

Extract all pricing tiers with their monthly prices.

Store as JSON at `/Users/austenallred/clawd/data/competitor-prices-baseline.json`

Format:

```
{
  "competitors": {
    "competitor-a": {
      "last_checked": "2026-02-22",
      "tiers": [
        {"name": "Starter", "monthly_price": 15, "annual_price": 12}
      ]
    }
  }
}
```

Then the monitoring script at `/Users/austenallred/clawd/scripts/price-check.py` :

```
#!/usr/bin/env python3
"""
price-check.py
Checks competitor pricing and reports changes.
Designed to be called by the agent with fresh price data.
"""

import json
import sys
from datetime import datetime
from pathlib import Path

BASELINE_FILE = Path("/Users/austenallred/clawd/data/competitor-prices-
baseline.json")
CHANGES_LOG = Path("/Users/austenallred/clawd/logs/price-changes.log")

def load_baseline():
    if BASELINE_FILE.exists():
        return json.loads(BASELINE_FILE.read_text())
    return {"competitors": {}}

def find_changes(baseline, current):
    changes = []

    for competitor, data in current.get("competitors", {}).items():
        baseline_comp = baseline.get("competitors", {}).get(competitor, {})
        baseline_tiers = {t["name"]: t for t in baseline_comp.get("tiers", [])}
        current_tiers = {t["name"]: t for t in data.get("tiers", [])}

        # Check for price changes
        for tier_name, tier_data in current_tiers.items():
            if tier_name in baseline_tiers:
                old_price = baseline_tiers[tier_name].get("monthly_price")
                new_price = tier_data.get("monthly_price")
                if old_price != new_price:
                    changes.append({
                        "competitor": competitor,
                        "tier": tier_name,
                        "old_price": old_price,
                        "new_price": new_price,
                        "direction": "up" if new_price > old_price else "down"
                    })

        # Check for new tiers
        for tier_name in current_tiers:
            if tier_name not in baseline_tiers:
                changes.append({
                    "competitor": competitor,
                    "tier": tier_name,
```



```

        "change_type": "new_tier",
        "price": current_tiers[tier_name].get("monthly_price")
    })

    # Check for removed tiers
    for tier_name in baseline_tiers:
        if tier_name not in current_tiers:
            changes.append({
                "competitor": competitor,
                "tier": tier_name,
                "change_type": "removed_tier"
            })

    return changes

def log_changes(changes):
    CHANGES_LOG.parent.mkdir(parents=True, exist_ok=True)
    with open(CHANGES_LOG, "a") as f:
        for change in changes:
            f.write(f"{datetime.now().isoformat()} | {json.dumps(change)}\n")

def update_baseline(baseline, current):
    """Merge current prices into baseline"""
    if "competitors" not in baseline:
        baseline["competitors"] = {}

    for competitor, data in current.get("competitors", {}).items():
        baseline["competitors"][competitor] = data
        baseline["competitors"][competitor]["last_checked"] =
datetime.now().strftime("%Y-%m-%d")

    BASELINE_FILE.write_text(json.dumps(baseline, indent=2))

if __name__ == "__main__":
    # Current prices passed via stdin as JSON
    current_data = json.loads(sys.stdin.read())
    baseline = load_baseline()

    changes = find_changes(baseline, current_data)

    if changes:
        log_changes(changes)
        print(json.dumps({"changed": True, "count": len(changes), "changes":
changes}))
    else:
        print(json.dumps({"changed": False, "count": 0}))

    # Update baseline with current
    update_baseline(baseline, current_data)

```

Cron task:

```
{
  "schedule": "0 9 * * 1",
  "task": "Weekly competitor price check: Fetch pricing pages for competitor-
a.com/pricing, competitor-b.com/pricing, competitor-c.com/pricing using web_fetch.
Extract pricing tier data into JSON format matching the baseline schema. Pipe the
JSON to: python3 /Users/austenallred/clawd/scripts/price-check.py. If the output
shows 'changed: true', send me a WhatsApp message listing all changes with
before/after prices."
}
```

Variations:

- Add feature comparison (not just price)
- Track trial period lengths
- Monitor multiple pages per competitor (pricing, features, integrations)
- Build a historical chart by saving each week's prices to a dated file

Recipe 3: Git Repository Watcher

Problem: You work across multiple repos and miss commits, PR merges, and releases.

Ingredients:

- Local git repositories
- GitHub personal access token (optional, for API access)

Recipe:

Script at `/Users/austenallred/clawd/scripts/repo-watcher.py` :

```
#!/usr/bin/env python3
"""
repo-watcher.py
Checks git repos for recent activity.
"""

import subprocess
import json
from datetime import datetime, timedelta
from pathlib import Path

REPOS = [
    "/Users/austenallred/projects/myapp",
    "/Users/austenallred/projects/mylib",
    "/Users/austenallred/clawd",
]

def get_recent_commits(repo_path, hours=24):
    """Get commits from the last N hours"""
    since = (datetime.now() - timedelta(hours=hours)).strftime("%Y-%m-%dT%H:%M:%S")

    try:
```

```

        result = subprocess.run(
            ["git", "log", f"--since={since}", "--oneline", "--all"],
            cwd=repo_path,
            capture_output=True,
            text=True,
            timeout=10
        )
        commits = [line.strip() for line in result.stdout.strip().splitlines() if
line.strip()]
        return commits
    except (subprocess.SubprocessError, FileNotFoundError):
        return []

def get_current_branch(repo_path):
    try:
        result = subprocess.run(
            ["git", "branch", "--show-current"],
            cwd=repo_path,
            capture_output=True,
            text=True,
            timeout=5
        )
        return result.stdout.strip()
    except subprocess.SubprocessError:
        return "unknown"

def check_uncommitted(repo_path):
    try:
        result = subprocess.run(
            ["git", "status", "--porcelain"],
            cwd=repo_path,
            capture_output=True,
            text=True,
            timeout=5
        )
        return bool(result.stdout.strip())
    except subprocess.SubprocessError:
        return False

def main():
    report = {
        "timestamp": datetime.now().isoformat(),
        "repos": []
    }

    for repo_path in REPOS:
        if not Path(repo_path).exists():
            continue

        repo_name = Path(repo_path).name
        commits = get_recent_commits(repo_path)
        branch = get_current_branch(repo_path)

```

```

has_uncommitted = check_uncommitted(repo_path)

repo_data = {
    "name": repo_name,
    "path": repo_path,
    "branch": branch,
    "recent_commits": len(commits),
    "commits": commits[:5], # First 5
    "has_uncommitted_changes": has_uncommitted
}

report["repos"].append(repo_data)

print(json.dumps(report, indent=2))

if __name__ == "__main__":
    main()

```

Cron task:

```

{
    "schedule": "0 9 * * 1-5",
    "task": "Daily repo check: Run python3 /Users/austenallred/clawd/scripts/repo-
watcher.py. Format the results as a brief status report. If any repo has uncommitted
changes that have been sitting for more than 24 hours, flag it. Send me a WhatsApp
with the summary only if there's something notable – don't send if everything is
normal."
}

```

Variations:

- Add `git fetch --all` before checking (to see remote changes)
- Monitor specific branches (like `main` for merges)
- Track file sizes to detect accidentally committed large files
- Check for merge conflicts across repos

Recipe 4: Document Summarizer Pipeline

Problem: You get a steady stream of long documents (reports, research, contracts). Reading all of them is impossible. Skipping them is risky.

Ingredients:

- A watched folder for incoming documents
- `pdftotext` installed (for PDF processing)

Recipe:

Set up the directory:

```

mkdir -p /Users/austenallred/clawd/documents/incoming
mkdir -p /Users/austenallred/clawd/documents/processed

```

```
mkdir -p /Users/austenallred/clawd/documents/summaries
```

Script at /Users/austenallred/clawd/scripts/doc-summarizer.sh :

```
#!/bin/bash
# doc-summarizer.sh
# Converts documents to text for agent processing.

INCOMING="/Users/austenallred/clawd/documents/incoming"
PROCESSED="/Users/austenallred/clawd/documents/processed"
TEXT_STAGING="/Users/austenallred/clawd/documents/text-staging"

mkdir -p "$TEXT_STAGING"
count=0

for file in "$INCOMING"/*; do
    [ -f "$file" ] || continue

    filename=$(basename "$file")
    extension="${filename##*}"
    basename_no_ext="${filename%.*}"
    text_file="$TEXT_STAGING/$basename_no_ext.txt"

    case "${extension,,}" in
        pdf)
            pdftotext "$file" "$text_file" 2>/dev/null && count=$((count+1))
            ;;
        txt|md)
            cp "$file" "$text_file" && count=$((count+1))
            ;;
        docx)
            # Requires pandoc or similar
            pandoc "$file" -o "$text_file" 2>/dev/null && count=$((count+1))
            ;;
        *)
            echo "Unsupported: $filename" >&2
            ;;
    esac

    # Move original to processed
    [ -f "$text_file" ] && mv "$file" "$PROCESSED/"
done

echo "$count documents prepared for summarization"
ls "$TEXT_STAGING/"
```

Cron task:

```
{
  "schedule": "0 8,12,17 * * *",
  "task": "Document processing: Run /Users/austenallred/clawd/scripts/doc-
```

```
summarizer.sh. For each .txt file that appears in /clawd/documents/text-staging/,
read its contents and generate a structured summary (save to
/clawd/documents/summaries/{filename}-summary.md). Summary should include: document
type, key topics, action items for me, important dates/deadlines, recommended
priority (high/medium/low). After summarizing, move the text file to
/clawd/documents/text-staging/done/. If any document is high priority, send me a
WhatsApp immediately."
}
```

Variations:

- Add a classification step before summarizing (contract, report, newsletter, technical doc)
- Extract and aggregate all action items across documents into a single file
- Build a searchable index of all summarized documents
- Route different document types to different summary templates

Recipe 5: Automated Code Review Bot

Problem: Code reviews take time. You want automated pre-review to catch obvious issues before a human looks at the code.

Ingredients:

- Git repos you want to monitor
- A list of your coding standards

Recipe:

Create `/Users/austenallred/clawd/protocols/CODE-STANDARDS.md` :

Code Standards

General

- No hardcoded credentials or tokens
- Functions should be < 50 lines
- All functions need docstrings/comments
- Error handling required on all external calls (DB, API, file I/O)

Python Specific

- Type hints required on all function signatures
- Use pathlib, not os.path
- Use f-strings, not .format() or %
- Prefer explicit over implicit

JavaScript/Node

- Use async/await, not .then() chains
- No var, use const/let
- ESLint must pass before review

Security

- No eval()
- No SQL string concatenation

- Validate all user inputs
- No debug code in commits (console.log, pdb, breakpoint)

Tests

- Every new function needs at least one test
- Edge cases must be tested
- No skipped tests without comment explaining why

Cron task for pre-commit review:

```
{
  "schedule": "0 16 * * 1-5",
  "task": "Afternoon code review: Find all .py and .js files modified in
/Users/austenallred/projects/myapp/ since 9am today using: find
/Users/austenallred/projects/myapp/src -name '*.py' -newer
/Users/austenallred/clawd/data/review-timestamp. Read each modified file. Check
against standards in /clawd/protocols/CODE-STANDARDS.md. Write findings to
/clawd/reviews/daily-$(date +%Y-%m-%d).md with: file, line (if known), severity
(CRITICAL/HIGH/MEDIUM/LOW), issue. Update timestamp file after checking. Send me
WhatsApp if any CRITICAL issues found."
}
```

Variations:

- Trigger on git push via a hook instead of cron
- Add automated test coverage checking
- Track recurring issues over time to identify training needs
- Auto-create tickets for CRITICAL issues

Recipe 6: Research Aggregator

Problem: You want to stay current on several topics but don't have time to read everything.

Ingredients:

- A list of your research topics
- Source URLs or search queries for each

Recipe:

Create `/Users/austenallred/clawd/data/research-topics.json` :

```
{
  "topics": [
    {
      "name": "AI Infrastructure",
      "queries": ["AI data center 2026", "GPU shortage update", "cloud AI pricing"],
      "sources": ["https://www.semianalysis.com/", "https://www.nextplatform.com/"],
      "frequency": "weekly"
    },
    {
      "name": "SaaS Growth Strategies",
```

```

    "queries": ["B2B SaaS pricing strategy 2026", "SaaS expansion revenue"],
    "sources": [],
    "frequency": "weekly"
  },
  {
    "name": "Competitor Moves",
    "queries": ["[Competitor Name] news 2026", "[Competitor Name] product
update"],
    "sources": ["https://competitor.com/blog"],
    "frequency": "daily"
  }
]
}

```

Cron task:

```

{
  "schedule": "0 6 * * 1",
  "task": "Weekly research aggregation: Read /clawd/data/research-topics.json. For
each topic marked 'weekly', run all search queries and fetch the top result from
each. For source URLs, fetch the content. Deduplicate results. For each topic, write
a research digest to /clawd/research/digests/YYYY-WW-{topic-name}.md with: key
developments this week, notable quotes or data points, what changed vs. last week.
Save a master weekly digest to /clawd/research/digests/YYYY-WW-master.md. Send me
the master digest via WhatsApp (condensed to 300 words max)."
```

Variations:

- Add sentiment analysis on competitor mentions
- Track metrics over time (who's growing, who's struggling)
- Connect to RSS feeds for real-time monitoring
- Build a weekly newsletter from the aggregated research

Recipe 7: Disk Space Guardian

Problem: Production systems run out of disk space. This always happens at 3am on a Friday.

Ingredients:

- Server nodes configured
- Alert thresholds defined

Recipe:

Script at `/Users/austenallred/clawd/scripts/disk-guardian.py` :

```

#!/usr/bin/env python3
"""
disk-guardian.py
Multi-path disk space monitor with alerts.
"""

```



```

import shutil
import json
from datetime import datetime
from pathlib import Path

WATCH_PATHS = [
    {"path": "/", "warning": 75, "critical": 90, "label": "System"},
    {"path": "/Users/austenallred/clawd", "warning": 80, "critical": 95, "label":
"Workspace"},
    {"path": "/Users/austenallred/projects", "warning": 80, "critical": 95, "label":
"Projects"},
]

HISTORY_FILE = Path("/Users/austenallred/clawd/data/disk-history.json")

def check_path(path_config):
    path = path_config["path"]
    try:
        usage = shutil.disk_usage(path)
        percent = (usage.used / usage.total) * 100
        free_gb = usage.free / (1024**3)

        return {
            "path": path,
            "label": path_config["label"],
            "percent_used": round(percent, 1),
            "free_gb": round(free_gb, 1),
            "status": (
                "CRITICAL" if percent >= path_config["critical"] else
                "WARNING" if percent >= path_config["warning"] else
                "OK"
            )
        }
    except OSError as e:
        return {
            "path": path,
            "label": path_config["label"],
            "error": str(e),
            "status": "ERROR"
        }

def load_history():
    if HISTORY_FILE.exists():
        return json.loads(HISTORY_FILE.read_text())
    return {"checks": []}

def save_history(history):
    HISTORY_FILE.parent.mkdir(parents=True, exist_ok=True)
    # Keep last 30 days
    history["checks"] = history["checks"][-720:] # 24 checks/day * 30 days
    HISTORY_FILE.write_text(json.dumps(history, indent=2))

```

```

if __name__ == "__main__":
    results = [check_path(p) for p in WATCH_PATHS]

    # Save to history
    history = load_history()
    history["checks"].append({
        "timestamp": datetime.now().isoformat(),
        "results": results
    })
    save_history(history)

    # Output
    output = {
        "timestamp": datetime.now().isoformat(),
        "results": results,
        "has_warning": any(r["status"] == "WARNING" for r in results),
        "has_critical": any(r["status"] == "CRITICAL" for r in results),
        "has_error": any(r["status"] == "ERROR" for r in results)
    }

    print(json.dumps(output, indent=2))

```

Cron task:

```

{
  "schedule": "0 */3 * * *",
  "task": "Disk guardian: Run python3 /Users/austenallred/clawd/scripts/disk-guardian.py. If output shows has_critical: true, send me an immediate WhatsApp alert with which paths are critical and how much space is free. If has_warning: true and current hour is between 8 and 20 (daytime), send a WhatsApp warning. Otherwise just log to /clawd/logs/disk-checks.log."
}

```

Cleanup companion:

```

{
  "schedule": "0 3 * * 0",
  "task": "Weekly cleanup: Find and delete: log files in /clawd/logs/ older than 30 days, files in /clawd/tmp/ older than 7 days, .DS_Store files in /clawd/ recursively, empty directories in /clawd/. Report space freed. Run the disk guardian script after and report final disk usage."
}

```

Recipe 8: Task Queue System

Problem: You have a bunch of tasks that need to get done and no good way to track them, prioritize them, or know what's waiting.

Ingredients:

- A tasks directory
- The discipline to write tasks down

Recipe:

Task file format at `/Users/austenallred/clawd/tasks/current.md` :

```
# Active Tasks

## High Priority
- [ ] Review Q1 sales report before board meeting (due: 2026-02-25)
- [ ] Fix login bug on mobile app (reported by 3 users)

## Normal Priority
- [ ] Write documentation for new API endpoint
- [ ] Update dependencies in myapp
- [ ] Research hosting options for new service

## Someday/Maybe
- [ ] Set up automated testing for legacy service
- [ ] Write post about the automation system
```

Script at `/Users/austenallred/clawd/scripts/task-manager.py` :

```
#!/usr/bin/env python3
"""
task-manager.py
Parses and manages the task list.
"""

import re
import json
import sys
from pathlib import Path
from datetime import datetime

TASKS_FILE = Path("/Users/austenallred/clawd/tasks/current.md")
ARCHIVE_FILE = Path("/Users/austenallred/clawd/tasks/archive.md")

def parse_tasks():
    if not TASKS_FILE.exists():
        return {"high": [], "normal": [], "someday": []}

    content = TASKS_FILE.read_text()
    current_section = "normal"
    tasks = {"high": [], "normal": [], "someday": []}

    for line in content.splitlines():
        if "High Priority" in line:
            current_section = "high"
        elif "Normal Priority" in line:
            current_section = "normal"
```

```

elif "Someday" in line:
    current_section = "someday"
elif line.strip().startswith("- [ ]"):
    task_text = line.strip()[6:]
    # Check for due date
    due_match = re.search(r'\(due: (\d{4}-\d{2}-\d{2})\)', task_text)
    task = {
        "text": task_text,
        "due": due_match.group(1) if due_match else None,
        "overdue": False
    }
    if task["due"]:
        task["overdue"] = task["due"] < datetime.now().strftime("%Y-%m-%d")
    tasks[current_section].append(task)

return tasks

def get_summary():
    tasks = parse_tasks()
    total = sum(len(v) for v in tasks.values())
    overdue = sum(1 for section in tasks.values() for t in section if
t.get("overdue"))

    return {
        "total": total,
        "high_priority": len(tasks["high"]),
        "normal": len(tasks["normal"]),
        "someday": len(tasks["someday"]),
        "overdue": overdue,
        "top_tasks": tasks["high"][:3] + tasks["normal"][:2]
    }

if __name__ == "__main__":
    command = sys.argv[1] if len(sys.argv) > 1 else "summary"

    if command == "summary":
        print(json.dumps(get_summary(), indent=2))
    elif command == "all":
        print(json.dumps(parse_tasks(), indent=2))

```

Weekly task review cron:

```

{
  "schedule": "0 9 * * 1",
  "task": "Monday task review: Run python3 /Users/austenallred/clawd/scripts/task-
manager.py summary. Read the full task list at /clawd/tasks/current.md. Check if any
tasks have been sitting in High Priority for more than 7 days. Send me a WhatsApp
with: total tasks, overdue items (flag these prominently), top 5 tasks to focus on
this week."
}

```

Recipe 9: Automated Weekly Report

Problem: You have to produce a weekly status report. It takes an hour. Most of that hour is gathering information you already have.

Ingredients:

- Your task list
- Your git repos
- Memory logs
- Any metrics you track

Recipe:

Script at `/Users/austenallred/clawd/scripts/weekly-report.sh` :

```
#!/bin/bash
# weekly-report.sh
# Gathers raw data for weekly report generation

WORKSPACE="/Users/austenallred/clawd"
WEEK=$(date +%Y-W%V)
REPORT_DIR="$WORKSPACE/reports/weekly"
mkdir -p "$REPORT_DIR"

echo "=== WEEKLY DATA COLLECTION: $WEEK ==="
echo ""

echo "---- Git Activity (last 7 days) ----"
for repo in /Users/austenallred/projects/*/; do
    if [ -d "$repo/.git" ]; then
        repo_name=$(basename "$repo")
        commits=$(cd "$repo" && git log --oneline --since="7 days ago" --all 2>/dev/null
| wc -l | tr -d ' ')
        if [ "$commits" -gt 0 ]; then
            echo "$repo_name: $commits commits"
            cd "$repo" && git log --oneline --since="7 days ago" --all 2>/dev/null | head
-5
        fi
    fi
done

echo ""
echo "---- Task Completions (from memory logs) ----"
# Look for completed task markers in this week's memory files
for day in Mon Tue Wed Thu Fri; do
    date_str=$(date -v-$( ( $(date +%u) - 1 + 7 ) % 7 )d +%Y-%m-%d 2>/dev/null || date
+%Y-%m-%d)
    done
    grep -h "✅\|COMPLETE\|DONE\|merged\|shipped" "$WORKSPACE/memory/"*.md 2>/dev/null |
tail -20
```

```

echo ""
echo "---- Active Projects Status ----"
for project_dir in "$WORKSPACE/projects"/*; do
    [ -d "$project_dir" ] || continue
    project_id=$(basename "$project_dir")
    if [ -f "$project_dir/implementation-summary.md" ]; then
        status=$(head -5 "$project_dir/implementation-summary.md")
        echo "$project_id: (implementation complete)"
    elif [ -f "$project_dir/planning-artifacts/planning-summary.md" ]; then
        echo "$project_id: (in implementation)"
    elif [ -f "$project_dir/research-artifacts/research-summary.md" ]; then
        echo "$project_id: (in planning)"
    else
        echo "$project_id: (in research)"
    fi
done

```

Cron task:

```

{
  "schedule": "30 16 * * 5",
  "task": "Friday weekly report: Run /Users/austenallred/clawd/scripts/weekly-report.sh and capture the output. Also read this week's memory logs from /clawd/memory/. Using all this data, write a structured weekly report to /clawd/reports/weekly/YYYY-WW.md with sections: Shipped This Week, In Progress, Blocked Items, Key Metrics, Next Week's Priorities. Keep it concise – 300 words max. Send me the report via WhatsApp."
}

```

Variations:

- Add financial metrics if you track revenue
- Include customer feedback highlights
- Add team member spotlights for team reports
- Auto-format for Notion/Confluence/Slack

Recipe 10: Personal Knowledge Base Builder

Problem: You read lots of things, take notes, have conversations, and then can't find anything later.

Ingredients:

- A notes directory
- Consistent note format

Recipe:

Note format (create for each significant source):

```

# [Title/Topic]

**Date:** 2026-02-22

```

```
**Source:** [URL/Book/Conversation]
**Type:** article|book|conversation|video
**Tags:** #automation, #productivity, #tools
```

Summary

One paragraph. What's the main point?

Key Takeaways

- Point 1
- Point 2
- Point 3

Quotes Worth Keeping

> "The best automation is the kind you forget you built."

Related

- Links to other notes
- Related tags

Save to `/Users/austenallred/clawd/notes/YYYY-MM-DD-{slug}.md` .

Index builder cron:

```
{
  "schedule": "0 22 * * *",
  "task": "Knowledge base index: Read all notes in /clawd/notes/ that were modified
today. For each new note, extract the tags. Update the index at
/clawd/notes/INDEX.md - which is a sorted list of all notes grouped by tag. Also
update /clawd/notes/RECENT.md with the 10 most recently added notes. If a note
references a URL that hasn't been fetched yet (no content in /clawd/notes/web-
cache/), fetch and cache it."
}
```

Search capability (ask agent directly):

Search my knowledge base at `/clawd/notes/` for everything related to:
automation cron scheduling

Read the index at `/clawd/notes/INDEX.md`, find relevant notes,
read them, and give me a synthesis of what I've collected on this topic.

Variations:

- Connect to a Readwise or Obsidian vault
- Add spaced repetition for important points
- Build a weekly review of your recent notes
- Extract action items from meeting notes automatically

Appendix E: Common Patterns Reference

A compressed reference for the patterns used throughout this book. When you're implementing something and need a quick reminder of how something works, start here.

Context File Patterns

SOUL.md Structure

```
# SOUL.md

## Identity
[Who the agent is, role, working for whom]

## Principles
[Ordered list of core operating principles]

## Communication Style
[Tone, format preferences, verbosity]

## Domain Context
[Tech stack, team, business model – anything that informs decisions]

## Keyword Commands
[Trigger words that route to specific behaviors]

## Always Do
[Non-negotiable behaviors]

## Never Do
[Hard constraints]
```

MEMORY.md Structure

```
# MEMORY.md

## Routing Decisions
- YYYY-MM-DD: [decision] – Rationale: [why]

## Operator Preferences
- [learned pattern]: [description]

## System Configuration
- [config fact]: [value/description]

## Known Issues
- [issue]: [workaround or status]

## Escalation Patterns
- Common block: [description] → Resolution: [approach]
```


AGENTS.md Minimal Structure

```
# AGENTS.md

## Role
[Main agent's responsibility]

## Named Agents
[List of specialized agents with role descriptions]

## Routing Rules
[Decision tree for routing work]

## Quality Gates
[What "done" means for each phase]

## Escalation Protocol
[What to do when things fail]

## Memory Protocol
[What gets written where]
```

Cron Expression Cheatsheet

```
# Format: minute hour day-of-month month day-of-week

# Common schedules
0 7 * * *           - Daily at 7am UTC
0 7 * * 1-5         - Weekdays at 7am UTC
0 13 * * 1-5        - Weekdays at 7am CST (UTC-6)
0 12 * * 1-5        - Weekdays at 7am CDT (UTC-5)
*/15 * * * *        - Every 15 minutes
0 */2 * * *         - Every 2 hours
0 9 * * 1           - Monday at 9am UTC
0 0 1 * *          - First of month at midnight
0 9,17 * * 1-5      - Weekdays at 9am and 5pm
30 8 * * 1-5        - Weekdays at 8:30am
0 0 * * 0           - Sunday midnight
0 6 * * 1           - Monday at 6am (weekly trigger)
```

Tool Call Quick Reference

Read a file:

```
Read /Users/austenallred/clawd/MEMORY.md
```

Write a file:

```
Write to /Users/austenallred/clawd/data/output.json:
{json content here}
```

Edit a file (surgical replacement):

```
Edit /Users/austenallred/clawd/config.json:
Find: "enabled": false
Replace with: "enabled": true
```

Run a command:

```
Run: python3 /Users/austenallred/clawd/scripts/process.py --date 2026-02-22
```

Run in background:

```
Run in background (wait 5 seconds, then continue):
python3 /Users/austenallred/clawd/scripts/long-process.py
```

Search the web:

```
Search: competitor pricing comparison 2026
```

Fetch a web page:

```
Fetch: https://example.com/pricing (extract as markdown)
```

Take a browser screenshot:

```
Take a screenshot of https://status.example.com and analyze it for any red indicators
```

Send a WhatsApp:

```
Send me a WhatsApp: "Pipeline complete. 47 items processed."
```

Spawn a sub-agent:

```
Spawn a sub-agent labeled "research-phase-1" to:
[task description]
Report back when complete.
```

File Organization Conventions

```
/Users/austenallred/clawd/
├─ [Context files – read every session]
│   ├── SOUL.md
│   ├── MEMORY.md
│   ├── AGENTS.md
│   ├── USER.md
│   └── APPS.md
└─ [Protocols – load on demand]
```

```

├── protocols/
│   ├── HEARTBEAT.md
│   ├── ALERTS.md
│   └── TOOLS.md
├── [Operational data]
│   ├── memory/          # Daily logs: YYYY-MM-DD.md
│   ├── logs/            # System logs
│   └── data/             # Persistent JSON/CSV data
├── [Work products]
│   ├── projects/        # Active project directories
│   ├── reports/         # Generated reports
│   ├── drafts/          # Work in progress
│   └── research/         # Research artifacts
├── [Infrastructure]
│   ├── scripts/         # Executable scripts
│   ├── templates/       # Document templates
│   ├── config/          # Non-secret configuration
│   └── secrets/         # Credentials (chmod 700)
└── [Other]
    ├── tasks/           # Task lists
    ├── notes/           # Personal knowledge base
    └── book/            # This book

```

Python Script Template

Every automation script should follow this pattern:

```

#!/usr/bin/env python3
"""
script-name.py
One-line description of what this does.

Usage: python3 script-name.py [args]
"""

import sys
import json
import logging
from pathlib import Path
from datetime import datetime

# Configuration
WORKSPACE = Path("/Users/austenallred/clawd")
INPUT_FILE = WORKSPACE / "data" / "input.json"
OUTPUT_FILE = WORKSPACE / "data" / "output.json"
LOG_FILE = WORKSPACE / "logs" / "script-name.log"

```

```

# Logging setup
logging.basicConfig(
    level=logging.INFO,
    format="%asctime)s %(levelname)s %(message)s",
    handlers=[
        logging.StreamHandler(sys.stdout),
        logging.FileHandler(LOG_FILE)
    ]
)
log = logging.getLogger(__name__)

def load_input():
    """Load and validate input data."""
    if not INPUT_FILE.exists():
        log.error(f"Input file not found: {INPUT_FILE}")
        sys.exit(1)

    try:
        return json.loads(INPUT_FILE.read_text())
    except json.JSONDecodeError as e:
        log.error(f"Invalid JSON in input file: {e}")
        sys.exit(1)

def process(data):
    """Main processing logic. Returns result dict."""
    result = {
        "timestamp": datetime.now().isoformat(),
        "status": "success",
        "processed": 0,
        "errors": []
    }

    # Your logic here

    return result

def save_output(result):
    """Write result to output file."""
    OUTPUT_FILE.parent.mkdir(parents=True, exist_ok=True)
    OUTPUT_FILE.write_text(json.dumps(result, indent=2))
    log.info(f"Output saved: {OUTPUT_FILE}")

def main():
    log.info("Starting script-name")

    data = load_input()
    result = process(data)
    save_output(result)

    # Print summary for agent consumption
    print(json.dumps({
        "status": result["status"],

```

```

        "processed": result["processed"],
        "error_count": len(result["errors"])
    }))

    # Exit with error code if failed
    if result["status"] != "success":
        sys.exit(1)

if __name__ == "__main__":
    main()

```

Error Handling Patterns

Pattern 1: Retry with backoff

```

import time
import logging

def with_retry(fn, max_attempts=3, delay=2):
    """Call fn with exponential backoff retries."""
    for attempt in range(1, max_attempts + 1):
        try:
            return fn()
        except Exception as e:
            if attempt == max_attempts:
                raise
            wait = delay ** attempt
            logging.warning(f"Attempt {attempt} failed: {e}. Retrying in {wait}s")
            time.sleep(wait)

```

Pattern 2: Graceful degradation

```

def get_data_with_fallback(primary_path, fallback_path):
    """Try primary, fall back to cached data."""
    try:
        return fetch_fresh_data(primary_path)
    except Exception as e:
        logging.warning(f"Primary source failed: {e}. Using fallback.")
        if Path(fallback_path).exists():
            return json.loads(Path(fallback_path).read_text())
        return None

```

Pattern 3: State tracking for resumable processes

```

def load_state():
    state_file = Path("/Users/austenallred/clawd/data/process-state.json")
    if state_file.exists():
        return json.loads(state_file.read_text())
    return {"last_processed_id": None, "processed_count": 0}

```

```
def save_state(state):
    state_file = Path("/Users/austenallred/clawd/data/process-state.json")
    state["updated_at"] = datetime.now().isoformat()
    state_file.write_text(json.dumps(state, indent=2))

# In main loop:
state = load_state()
for item in items:
    if state["last_processed_id"] and item["id"] <= state["last_processed_id"]:
        continue # Skip already processed
    process(item)
    state["last_processed_id"] = item["id"]
    state["processed_count"] += 1
    save_state(state) # Save after each item - resumable if interrupted
```

Alert Formatting Templates

WhatsApp alert templates:

```
# Critical alert
🔴 CRITICAL: {system}
{what happened}
{what to do}
Time: {timestamp}

# Warning
⚠️ WARNING: {system}
{what's wrong}
{severity level}
No action needed now.

# Info (use sparingly)
📘 {system}: {one line summary}

# Success
✅ {task} complete
{key metric}: {value}
{optional next step}
```

Log line format:

```
2026-02-22T07:00:01Z | INFO | morning-briefing | sent | 3 items, 0 alerts
2026-02-22T09:00:02Z | INFO | heartbeat | check | OK, disk 45%
2026-02-22T13:00:01Z | WARN | heartbeat | check | disk 81%, alert sent
2026-02-22T15:30:00Z | ERROR | price-check | fetch | HTTP 429 from competitor.com
```

Format: {timestamp} | {level} | {component} | {action} | {details}

Browser Automation Patterns

Login flow template:

1. Navigate to {login_url}
2. Snapshot the page
3. Fill email field (ref from snapshot) with email from /clawd/secrets/credentials.json
4. Fill password field with password from /clawd/secrets/credentials.json
5. Click submit button
6. Wait for page change (text "Loading" to disappear, or dashboard text to appear)
7. Snapshot to confirm success
8. If still on login page: report authentication failure

Data extraction template:

1. Navigate to {data_url}
2. If needed, scroll down to load all content
3. Evaluate: document.querySelectorAll('{selector}') to get all target elements
4. For each element, extract: {field1}, {field2}, {field3}
5. Return as JSON array
6. Save to {output_file}

Monitoring template:

1. Navigate to {monitoring_url}
2. Take a screenshot
3. Analyze the screenshot for: any red/orange indicators, error text, anomalies
4. Compare to baseline (stored in /clawd/data/baseline-screenshot.png) – if vision model detects significant visual changes, alert
5. Save screenshot to /clawd/screenshots/{date}-{time}.png
6. Log check result to /clawd/logs/monitoring.log

Multi-Agent Coordination Templates

Research handoff:

Project {id}: Research phase initiated

Artifacts:

- Intake: /clawd/projects/{id}/intake.md
- Output directory: /clawd/projects/{id}/research-artifacts/

Required deliverables:

- evidence.md (sources and findings)
- decision-log.md (key decisions made during research)
- research-summary.md (gate: READY / NOT-READY / NEEDS-OPERATOR-INPUT)

When complete: write gate decision to research-summary.md and report back.

Implementation handoff:

Project {id}: Implementation phase initiated

Prior artifacts:

- Planning: /clawd/projects/{id}/planning-artifacts/
- Architecture: /clawd/projects/{id}/planning-artifacts/architecture.md

Required deliverables:

- sprint-plan.md (what you'll build in what order)
- stories/ (one file per story, format: story-{n}.md)
- implementation-summary.md (status: COMPLETE / BLOCKED / PARTIAL)

Each story should follow the DONE criteria in /clawd/protocols/DOD.md

Report back when implementation-summary.md is written.

RALPH escalation message:

 BLOCKED: Project {id}

Phase: {phase}

Agent: {agent-label}

Attempts: {n}/3

What failed:

{description}

Error details:

{error}

What was tried:

- Attempt 1: {approach} → {result}
- Attempt 2: {approach} → {result}
- Attempt 3: {approach} → {result}

Partial artifacts:

{list what exists at what paths}

Recommended options:

1. {option 1 with tradeoffs}
2. {option 2 with tradeoffs}

Awaiting your decision.

Extended Chapter Notes

These sections provide deeper coverage of topics from the main chapters, for readers who want to go further.

Deep Dive: The exec Tool

The `exec` tool is used in almost every non-trivial automation. Here's everything you need to know.

Environment Variables

When `exec` runs a command, it inherits a limited environment. The `PATH` might not include all your usual locations. This causes failures like "command not found" for tools that work fine in your terminal.

The problem:

```
# In your terminal: works fine
python3 /clawd/scripts/process.py

# In exec: might fail if /usr/local/bin is not in PATH
exec: python3 /clawd/scripts/process.py → command not found
```

The fix: Use full paths or set `PATH` explicitly:

```
{
  "tool": "exec",
  "command": "/usr/local/bin/python3 /Users/austenallred/clawd/scripts/process.py",
  "env": {"PATH": "/usr/local/bin:/usr/bin:/bin:/opt/homebrew/bin"}
}
```

Or at the top of your shell scripts:

```
#!/bin/bash
export PATH="/usr/local/bin:/opt/homebrew/bin:$PATH"
```

Working Directory

The default working directory for `exec` is your workspace root. If your script uses relative paths, they're relative to the workspace.

Explicit working directory:

```
{
  "tool": "exec",
  "command": "npm run build",
  "workdir": "/Users/austenallred/projects/myapp"
}
```

This is better than `cd /path && command` because it's explicit and the `workdir` doesn't affect subsequent commands.

Timeout Handling

Long-running commands need explicit timeout settings:

```
{
  "tool": "exec",
  "command": "python3 long-running-script.py",
  "timeout": 300
}
```

300 seconds = 5 minutes. Adjust based on your script's expected runtime. Commands that exceed the timeout are killed.

For tasks where you genuinely don't know how long they'll take, use background mode and monitor:

```
{
  "tool": "exec",
  "command": "python3 very-long-script.py",
  "background": true,
  "yieldMs": 10000
}
```

Then check periodically:

```
{
  "tool": "process",
  "action": "poll",
  "sessionId": "{session-id}",
  "timeout": 5000
}
```

PTY Mode

Some programs require a real terminal (TTY) to run correctly. Interactive programs, programs that check if they're running in a terminal, ncurses apps. For those:

```
{
  "tool": "exec",
  "command": "htop",
  "pty": true
}
```

Most automation scripts don't need this. Use it when you see "not a TTY" errors.

Security Modes in Practice

security: "deny" — Blocks all exec. Use this on agents that should never run shell commands (write-only agents, analysis agents).

security: "allowlist" — Only specific commands allowed. The right default for most agents:

```
{
  "tools": {
    "exec": {
```

```

    "security": "allowlist",
    "allowedCommands": ["python3", "node", "git", "ls", "cat", "find", "grep",
"awk", "sed", "jq", "curl"]
    }
}
}

```

security: "full" — No restrictions. Use only when you explicitly need to run arbitrary commands. Dangerous in automated contexts.

Deep Dive: Web Scraping at Scale

Chapter 4 covers the basics of web automation. Here's what you need for more serious scraping.

Respecting Rate Limits

Every website has an implied rate limit. Exceeding it gets you blocked. A few rules:

- Never make more than 1 request per second to the same domain
- Add delays between requests: `time.sleep(2)` in Python
- Respect `Retry-After` headers on 429 responses
- Check `robots.txt` before scraping

```

import time
import requests
from urllib import robotparser
from urllib.parse import urlparse

def check_robots_txt(url):
    """Returns True if the URL is allowed to be scraped."""
    parsed = urlparse(url)
    robots_url = f"{parsed.scheme}://{parsed.netloc}/robots.txt"

    rp = robotparser.RobotFileParser()
    rp.set_url(robots_url)
    try:
        rp.read()
        return rp.can_fetch("*", url)
    except Exception:
        return True # If we can't check, assume allowed

def polite_get(url, delay=2):
    """Fetch URL with politeness delay."""
    if not check_robots_txt(url):
        raise ValueError(f"robots.txt disallows: {url}")

    response = requests.get(url, headers={
        "User-Agent": "Mozilla/5.0 (compatible; automation-bot/1.0)"
    }, timeout=30)

```

```
time.sleep(delay) # Be polite
return response
```

Handling Pagination

Most data sources are paginated. You need to handle this.

URL-based pagination:

```
def scrape_all_pages(base_url, max_pages=50):
    results = []

    for page in range(1, max_pages + 1):
        url = f"{base_url}?page={page}"
        response = polite_get(url)

        data = parse_page(response.text)

        if not data:
            break # No more data

        results.extend(data)
        print(f"Page {page}: {len(data)} items")

    return results
```

Cursor-based pagination (APIs):

```
def scrape_with_cursor(api_url, headers):
    results = []
    cursor = None

    while True:
        params = {"limit": 100}
        if cursor:
            params["cursor"] = cursor

        response = requests.get(api_url, params=params, headers=headers)
        data = response.json()

        results.extend(data.get("items", []))

        cursor = data.get("next_cursor")
        if not cursor:
            break # No more pages

    return results
```

Caching to Avoid Duplicate Requests

Cache responses locally to avoid re-fetching the same content:

```

import hashlib
import json
from pathlib import Path

CACHE_DIR = Path("/Users/austenallred/clawd/data/web-cache")

def get_cached_or_fetch(url, ttl_hours=24):
    """Return cached response or fetch fresh."""
    cache_key = hashlib.md5(url.encode()).hexdigest()
    cache_file = CACHE_DIR / f"{cache_key}.json"

    # Check cache
    if cache_file.exists():
        cached = json.loads(cache_file.read_text())
        age_hours = (time.time() - cached["timestamp"]) / 3600
        if age_hours < ttl_hours:
            return cached["content"]

    # Fetch fresh
    response = polite_get(url)
    content = response.text

    # Save cache
    CACHE_DIR.mkdir(parents=True, exist_ok=True)
    cache_file.write_text(json.dumps({
        "url": url,
        "timestamp": time.time(),
        "content": content
    }))

    return content

```

Deep Dive: Memory Architecture at Scale

As your automation system grows, MEMORY.md becomes unwieldy. Here's how to scale the memory architecture.

Tiered Memory

Instead of one MEMORY.md, use multiple files for different time horizons:

```

memory/
├─ CORE.md           # Never expires – fundamental facts about setup
├─ CURRENT.md        # Current quarter – active, relevant
├─ ARCHIVE/          # Older memory, loaded on demand
│   ├─ 2025-Q4.md
│   └─ 2025-Q3.md
└─ DAILY/            # Day-by-day operational logs
    ├─ 2026-02-22.md
    └─ 2026-02-21.md

```

CORE.md — Things that are always true and always relevant:

Core Memory

Infrastructure

- Workspace: /Users/austenallred/clawd
- Primary model: claude-sonnet-4-5
- Timezone: America/Chicago (CST/CDT)
- Gateway: mac-studio, port 3147

Hard Rules

- Never commit to main without tests passing
- Never send messages to customers without human approval
- Credentials live in /clawd/secrets/ only

Tech Stack (unchanged since Jan 2025)

- Backend: Python 3.12, FastAPI
- Frontend: Next.js 14
- Database: PostgreSQL 16, hosted on AWS RDS
- Deployment: AWS ECS, us-east-1

CURRENT.md — Rotates quarterly, covers active context:

Current Quarter Memory (Q1 2026)

Active Projects

- Project 20260222-143000: Competitor analysis (in progress)
- Project 20260218-091200: Mobile app feature (in testing)

Open Decisions

- Should we add a free tier? (researched, awaiting operator decision)
- Database migration to v4 schema (planned for March)

Learned Patterns This Quarter

- *web_fetch fails on competitor.com – use browser tool instead*
- *Morning briefings > 5 items never get read – cap at 5*

Loading the right memory: At session start:

Memory Loading Protocol (in SOUL.md)

At session start:

1. Always read CORE.md
2. Read CURRENT.md
3. Read today's DAILY log if exists
4. If a project is mentioned, read its project context.md
5. For deep questions, search ARCHIVE/ on demand

Searchable Memory

For large note collections, use grep rather than trying to load everything:

```
Search /Users/austenallred/clawd/memory/ and /clawd/notes/ for anything
related to "database migration" using:
grep -r "database migration" /Users/austenallred/clawd/memory/ --include="*.md" -l
```

The `-l` flag returns filenames only. Then read the relevant files.

Memory Compaction

Over time, CURRENT.md grows stale. Run a monthly compaction:

```
{
  "schedule": "0 10 1 * *",
  "task": "Monthly memory compaction: Read CURRENT.md. For each item, determine if
it's still relevant. Move completed projects to ARCHIVE/YYYY-QN.md. Remove resolved
decisions. Summarize what happened this past month into one section. Keep CURRENT.md
under 500 lines. Write what was archived to /clawd/logs/memory-compaction.log."
}
```

Deep Dive: Debugging Agent Behavior

Sometimes agents do unexpected things. Here's how to understand what happened and fix it.

Reading Agent Logs

OpenClaw logs agent activity. The log format shows tool calls, model responses, and errors:

```
# Find recent agent logs
ls -lt ~/.openclaw/logs/ | head -10

# Read a specific log
cat ~/.openclaw/logs/session-20260222-143000.log
```

Look for:

- Tool calls that were made (and their parameters)
- Error messages
- Model response excerpts
- Session start/end timestamps

Adding Instrumentation

For complex automation, add explicit logging to your task descriptions:

```
Execute the pipeline. After each major step, write one line to
/clawd/logs/pipeline-trace.log with the step name and result.
Format: TIMESTAMP | STEP | RESULT | DETAILS

Steps to trace:
- "fetch-data": after fetching source data
- "transform": after transformation
```

- "validate": after validation
- "output": after writing output

This gives you a detailed trace of what happened.

The Replay Pattern

When something fails, you want to replay it with different parameters without re-running everything.

Checkpoint your state:

```
# In your pipeline script
def save_checkpoint(stage, data):
    checkpoint_file = Path(f"/Users/austenallred/clawd/data/checkpoint-
{stage}.json")
    checkpoint_file.write_text(json.dumps({
        "stage": stage,
        "timestamp": datetime.now().isoformat(),
        "data": data
    }))

def load_checkpoint(stage):
    checkpoint_file = Path(f"/Users/austenallred/clawd/data/checkpoint-
{stage}.json")
    if checkpoint_file.exists():
        return json.loads(checkpoint_file.read_text())["data"]
    return None
```

Then in your pipeline:

```
# Try to resume from checkpoint
raw_data = load_checkpoint("fetch") or fetch_data()
save_checkpoint("fetch", raw_data)

transformed = load_checkpoint("transform") or transform(raw_data)
save_checkpoint("transform", transformed)
```

When you re-run after a failure at "output", it skips the expensive fetch and transform steps.

Agent Prompt Debugging

If an agent consistently misinterprets a task, the problem is usually in the prompt. Test the prompt in isolation:

```
[In a new session, without loading SOUL.md or AGENTS.md]
Perform this task: [paste your task description exactly]
```

If it works correctly in isolation, the problem is context contamination — something in SOUL.md, MEMORY.md, or AGENTS.md is conflicting with the task.

Binary search your context files:

1. Remove half your context files

2. Test
3. If problem persists: the bug is in the remaining half
4. If problem goes away: the bug is in the removed half
5. Repeat on the buggy half

Deep Dive: Production Hardening

Moving your automation from "works on my machine" to "runs reliably for months" requires additional discipline.

The Idempotency Requirement

Idempotent operations produce the same result whether they're run once or ten times. This matters because cron jobs can run twice (DST transitions, system issues), and you need to be safe.

Not idempotent (bad):

```
# This appends a line every run
with open(log_file, "a") as f:
    f.write(f"Processed {count} items\n")
```

If this runs twice, you get two log lines. If the downstream system reads this count, it might double-count.

Idempotent (good):

```
# This overwrites with current state
state_file.write_text(json.dumps({
    "last_processed_date": today,
    "count": count
}))
```

The state file always reflects current reality regardless of how many times you write it.

Pattern for idempotent file processing:

```
STATE_FILE = Path("/clawd/data/process-state.json")

def load_state():
    if STATE_FILE.exists():
        return json.loads(STATE_FILE.read_text())
    return {"last_processed_id": 0, "last_run": None}

def process_items(items):
    state = load_state()

    new_items = [i for i in items if i["id"] > state["last_processed_id"]]

    for item in new_items:
        process_one(item)
        # Update state after each item so we can resume
        state["last_processed_id"] = item["id"]
```

```
state["last_run"] = datetime.now().isoformat()
STATE_FILE.write_text(json.dumps(state))

return len(new_items)
```

Health Check Endpoints

If you're running long-lived processes, add a health check:

```
#!/usr/bin/env python3
"""
health-check.py
Returns 0 if system is healthy, 1 if degraded.
"""

import json
import sys
from pathlib import Path
from datetime import datetime, timedelta

CHECKS = []

def check_heartbeat():
    """Verify the heartbeat ran recently."""
    heartbeat_log = Path("/Users/austenallred/clawd/logs/heartbeat.log")
    if not heartbeat_log.exists():
        return False, "Heartbeat log missing"

    content = heartbeat_log.read_text().strip()
    if not content:
        return False, "Heartbeat log empty"

    last_line = content.splitlines()[-1]
    try:
        # Parse timestamp from log line
        ts_str = last_line.split(" ")[0]
        ts = datetime.fromisoformat(ts_str)
        age = datetime.now() - ts

        if age > timedelta(hours=3):
            return False, f"Heartbeat is {age.seconds//3600}h old"
        return True, f"Heartbeat OK ({age.seconds//60}m ago)"
    except (ValueError, IndexError):
        return False, "Cannot parse heartbeat timestamp"

def check_disk():
    """Verify disk space is acceptable."""
    import shutil
    usage = shutil.disk_usage("/")
    percent = (usage.used / usage.total) * 100
    if percent > 90:
```

```

        return False, f"Disk critical: {percent:.0f}%"
    return True, f"Disk OK: {percent:.0f}%"

def run_checks():
    results = []
    all_ok = True

    for check_fn in [check_heartbeat, check_disk]:
        ok, message = check_fn()
        results.append({"check": check_fn.__name__, "ok": ok, "message": message})
        if not ok:
            all_ok = False

    print(json.dumps({
        "healthy": all_ok,
        "timestamp": datetime.now().isoformat(),
        "checks": results
    }, indent=2))

    return 0 if all_ok else 1

sys.exit(run_checks())

```

Add to your heartbeat cron:

```

{
  "schedule": "0 */2 * * *",
  "task": "System health check: Run python3 /clawd/scripts/health-check.py. If
output shows healthy: false, send me a WhatsApp with the failing checks. Log result
to /clawd/logs/health.log."
}

```

Graceful Shutdown

Long-running processes need to handle termination gracefully:

```

import signal
import sys

class GracefulKiller:
    kill_now = False

    def __init__(self):
        signal.signal(signal.SIGINT, self.exit_gracefully)
        signal.signal(signal.SIGTERM, self.exit_gracefully)

    def exit_gracefully(self, *args):
        print("Shutdown signal received, finishing current item...")
        self.kill_now = True

# In your main loop:

```

```

killer = GracefulKiller()
for item in items:
    if killer.kill_now:
        print("Stopping at user request")
        break
    process(item)

```

When OpenClaw kills a background process (or the system restarts), this ensures the process finishes its current item and saves state cleanly.

Deep Dive: Working With External APIs

Most automation eventually touches external APIs. Here's how to do it without breaking things.

Authentication Patterns

API Key (most common):

Store in `/Users/austenallred/clawd/secrets/api-keys.json` :

```

{
  "openai": "sk-...",
  "anthropic": "sk-ant-...",
  "stripe": "sk_live_...",
  "sendgrid": "SG..."
}

```

Load in Python:

```

import json
from pathlib import Path

def get_api_key(service):
    keys_file = Path("/Users/austenallred/clawd/secrets/api-keys.json")
    keys = json.loads(keys_file.read_text())
    if service not in keys:
        raise ValueError(f"No API key configured for: {service}")
    return keys[service]

```

OAuth (when you need to act as a user):

OAuth is more complex. You need to:

1. Get an access token (user authorizes once)
2. Store the token and refresh token
3. Refresh when the access token expires

```

import json
import requests
from pathlib import Path
from datetime import datetime, timedelta

```

```

TOKEN_FILE = Path("/Users/austenallred/clawd/secrets/oauth-tokens.json")

def load_tokens(service):
    if TOKEN_FILE.exists():
        tokens = json.loads(TOKEN_FILE.read_text())
        return tokens.get(service, {})
    return {}

def save_tokens(service, tokens):
    all_tokens = {}
    if TOKEN_FILE.exists():
        all_tokens = json.loads(TOKEN_FILE.read_text())
    all_tokens[service] = tokens
    TOKEN_FILE.write_text(json.dumps(all_tokens, indent=2))

def get_valid_access_token(service, refresh_url, client_id, client_secret):
    tokens = load_tokens(service)

    # Check if access token is still valid
    if tokens.get("access_token") and tokens.get("expires_at"):
        expires = datetime.fromisoformat(tokens["expires_at"])
        if datetime.now() < expires - timedelta(minutes=5):
            return tokens["access_token"]

    # Refresh
    response = requests.post(refresh_url, data={
        "grant_type": "refresh_token",
        "refresh_token": tokens.get("refresh_token"),
        "client_id": client_id,
        "client_secret": client_secret
    })

    new_tokens = response.json()
    new_tokens["expires_at"] = (
        datetime.now() + timedelta(seconds=new_tokens.get("expires_in", 3600))
    ).isoformat()

    save_tokens(service, new_tokens)
    return new_tokens["access_token"]

```

Rate Limit Handling

APIs have rate limits. Hitting them causes 429 errors and potentially temporary bans.

```

import time
import requests
from datetime import datetime

class RateLimitedClient:
    def __init__(self, requests_per_minute):

```

```

self.min_interval = 60.0 / requests_per_minute
self.last_request_time = 0

def get(self, url, **kwargs):
    # Enforce rate limit
    elapsed = time.time() - self.last_request_time
    if elapsed < self.min_interval:
        time.sleep(self.min_interval - elapsed)

    response = requests.get(url, **kwargs)
    self.last_request_time = time.time()

    # Handle rate limit responses
    if response.status_code == 429:
        retry_after = int(response.headers.get("Retry-After", 60))
        print(f"Rate limited. Waiting {retry_after}s...")
        time.sleep(retry_after)
        return self.get(url, **kwargs) # Retry

    return response

# Usage
client = RateLimitedClient(requests_per_minute=30)
response = client.get("https://api.example.com/data")

```

Webhook Receivers

Sometimes external services need to push data to you. You need a webhook receiver.

Simple webhook receiver using Python's built-in HTTP server:

```

#!/usr/bin/env python3
"""
webhook-receiver.py
Receives webhooks and writes them to a processing queue.
"""

from http.server import HTTPServer, BaseHTTPRequestHandler
import json
import hmac
import hashlib
from pathlib import Path
from datetime import datetime

QUEUE_DIR = Path("/Users/austenallred/clawd/data/webhook-queue")
SECRET = "your-webhook-secret" # Match what the sender is configured with

class WebhookHandler(BaseHTTPRequestHandler):
    def do_POST(self):
        content_length = int(self.headers.get("Content-Length", 0))
        body = self.rfile.read(content_length)

```

```

# Verify signature (if service sends one)
signature = self.headers.get("X-Hub-Signature-256", "")
expected = "sha256=" + hmac.new(
    SECRET.encode(), body, hashlib.sha256
).hexdigest()

if signature and signature != expected:
    self.send_response(403)
    self.end_headers()
    return

# Save to queue
QUEUE_DIR.mkdir(parents=True, exist_ok=True)
timestamp = datetime.now().strftime("%Y%m%d-%H%M%S-%f")
queue_file = QUEUE_DIR / f"{timestamp}.json"
queue_file.write_bytes(body)

self.send_response(200)
self.end_headers()
self.wfile.write(b"OK")

def log_message(self, format, *args):
    pass # Suppress default logging

if __name__ == "__main__":
    server = HTTPServer(("0.0.0.0", 8080), WebhookHandler)
    print("Webhook receiver running on port 8080")
    server.serve_forever()

```

Run this as a background process:

```

{
  "tool": "exec",
  "command": "python3 /Users/austenallred/clawd/scripts/webhook-receiver.py",
  "background": true
}

```

Then process the queue on a regular schedule:

```

{
  "schedule": "*/5 * * * *",
  "task": "Process webhook queue: Check /clawd/data/webhook-queue/ for new .json
files. For each file, read it, determine the event type, process it appropriately,
and move to /clawd/data/webhook-processed/. Log processing to
/clawd/logs/webhook.log."
}

```

Deep Dive: Building Reliable Outputs

The hardest part of automation isn't writing the code — it's making sure the output is actually correct. Here's how to build validation into your pipelines.

Output Validation Schema

For any pipeline that produces structured data, define the expected schema:

```
def validate_sales_report(report):
    """Validate that a sales report has required fields and reasonable values."""
    errors = []

    # Required fields
    required = ["date", "total_revenue", "units_sold", "top_products"]
    for field in required:
        if field not in report:
            errors.append(f"Missing required field: {field}")

    if errors:
        return errors # Can't validate further without required fields

    # Type and range validation
    if not isinstance(report["total_revenue"], (int, float)):
        errors.append("total_revenue must be a number")
    elif report["total_revenue"] < 0:
        errors.append("total_revenue cannot be negative")
    elif report["total_revenue"] > 10_000_000:
        errors.append(f"total_revenue suspiciously high: {report['total_revenue']}")

    if not isinstance(report["units_sold"], int):
        errors.append("units_sold must be an integer")
    elif report["units_sold"] < 0:
        errors.append("units_sold cannot be negative")

    if not isinstance(report["top_products"], list):
        errors.append("top_products must be a list")
    elif not report["top_products"]:
        errors.append("top_products cannot be empty")

    # Date format
    try:
        datetime.strptime(report["date"], "%Y-%m-%d")
    except ValueError:
        errors.append(f"Invalid date format: {report['date']} (expected YYYY-MM-DD)")

    return errors

# In your pipeline:
report = generate_report(data)
errors = validate_sales_report(report)
if errors:
```



```
log.error(f"Report validation failed: {errors}")
sys.exit(1)
```

Sanity Checks Against Historical Data

For time-series data, validate against historical patterns:

```
def sanity_check_vs_history(today, history, tolerance=0.5):
    """Check if today's values are within tolerance of historical average."""
    issues = []

    if not history:
        return [] # Nothing to compare against

    avg_revenue = sum(h["total_revenue"] for h in history) / len(history)

    # Allow 50% variance from average
    if today["total_revenue"] < avg_revenue * (1 - tolerance):
        issues.append(
            f"Revenue {today['total_revenue']:.0f} is >50% below "
            f"historical average {avg_revenue:.0f}"
        )
    elif today["total_revenue"] > avg_revenue * (1 + tolerance * 3):
        # Allow more upside than downside
        issues.append(
            f"Revenue {today['total_revenue']:.0f} is unusually high vs "
            f"historical average {avg_revenue:.0f}"
        )

    return issues
```

Diff-Based Change Detection

For documents or data that should change gradually, validate the magnitude of change:

```
def check_change_magnitude(old_content, new_content, max_change_percent=30):
    """Warn if content changed dramatically."""
    old_words = len(old_content.split())
    new_words = len(new_content.split())

    if old_words == 0:
        return [] # Nothing to compare

    change_percent = abs(new_words - old_words) / old_words * 100

    if change_percent > max_change_percent:
        return [
            f"Content changed by {change_percent:.0f}% "
            f"({old_words} → {new_words} words). Verify this is intentional."
        ]
```

```
return []
```

Building Your Own Skills

By now you've seen how skills work as behavior patterns defined in context files. Here's how to design your own from scratch.

The Skill Design Process

Step 1: Define the trigger

What words or conditions activate this skill? Be specific to avoid false triggers.

Bad: "when I mention automation" (triggers too broadly) Good: "when I say 'run pipeline {name}'" (specific trigger phrase)

Step 2: Define the inputs

What data does this skill need?

- From context files?
- From user input?
- From external sources?

Step 3: Define the process

Step-by-step, what happens?

Step 4: Define the outputs

What files get created? What messages get sent? What state changes?

Step 5: Define failure behavior

What happens if something goes wrong at each step?

Example: Building a "Weekly Review" Skill

Trigger: "weekly review" or cron at Friday 4pm

Inputs:

- Current task list (`/clawd/tasks/current.md`)
- This week's memory logs (`/clawd/memory/YYYY-MM-DD.md` for Mon-Fri)
- Active projects (`/clawd/projects/*/`)
- Git activity (from `git log`)

Process:

1. Read all inputs
2. Extract: completed items, blocked items, new items added
3. Summarize each project's status in one sentence
4. Identify the 3 highest-leverage things for next week
5. Note anything that's been delayed more than once

Outputs:

- Write to `/clawd/reviews/weekly/YYYY-WW.md`
- WhatsApp summary (under 300 words)
- Update MEMORY.md with key learnings

Failure behavior:

- If any input file is missing, note it and continue with available data
- If git commands fail, skip that section
- Always produce some output, even if incomplete

Implementation in SOUL.md:

```
## Weekly Review Skill

**Trigger:** Message containing "weekly review" OR Friday cron

**Process:**
1. Read /clawd/tasks/current.md – extract pending, completed, blocked
2. Read this week's memory files from /clawd/memory/ (Mon–Fri)
3. For each dir in /clawd/projects/ modified this week, read implementation-
summary.md
4. Run: git log --oneline --since="7 days ago" --all for each project in
/clawd/projects/
5. Compile findings into /clawd/reviews/weekly/YYYY-WW.md:
  - What shipped
  - What's still in progress
  - What's blocked and why
  - Next week's top 3 priorities (my recommendation)
6. Send condensed version via WhatsApp (≤ 300 words)
7. Update MEMORY.md with: "Week YYYY-WW: [one-line summary of key outcome]"

**On failure:** Note which data sources were unavailable. Continue with rest.
```

This gives you everything you need to build arbitrarily sophisticated automation systems. The patterns are all here. The tools are all available. What remains is figuring out which parts of your life are worth automating and then actually building it.

That last part — actually building it — is the only thing that separates the people who have a working automation system from the people who just thought about having one.

Start with the annoying thing. Build the smallest possible version. Make sure it works. Then add to it.

Appendix F: The Practitioner's Field Guide

This appendix is different from the rest of the book. It's organized around questions that come up during real implementation work — not the "how does it work" questions that the main chapters answer, but the "I'm in the middle of building this and I'm stuck" questions.

When You're Getting Started

"I don't know what to automate first"

This is the most common paralysis point. Here's the exercise that breaks it:

Write down everything you did last week. Not what you planned to do — what you actually did. Include the small stuff: checking the same dashboard four times, copying data from one spreadsheet to another, sending nearly identical emails, looking up the same information from a few different places.

Look at your list. Circle anything that:

- Took more than 10 minutes
- You did more than once
- Felt mindless while you were doing it

Pick the smallest item you circled. Not the one that would save the most time — the one you can get running fastest. Get something working. That first win makes the second one feel achievable.

"My workspace is a mess and I don't know where to start"

Start with structure. Before writing a single automation, spend 30 minutes setting up your workspace properly:

```
cd /Users/austenallred/clawd

# Create the core directories
mkdir -p
{memory,logs,data,scripts,reports,projects,tasks,notes,templates,config,secrets,protocols}

# Set secrets permissions immediately
chmod 700 secrets/

# Create placeholder files for the important ones
touch SOUL.md MEMORY.md AGENTS.md USER.md
```

Then write the three most important files before doing anything else:

SOUL.md (10 minutes): Who are you? What are you building? What tone do you want?

MEMORY.md (5 minutes): Put today's date, a brief description of your setup, and one note about what you're trying to accomplish.

USER.md (5 minutes): Your timezone, your communication preferences, what matters to you.

With those in place, everything you build will have somewhere sensible to live and something to read at startup.

"I can't tell if my cron jobs are actually running"

Add a trivial canary to each cron job:

```
{
  "schedule": "0 7 * * *",
  "task": "Morning tasks: 1) Write 'HEARTBEAT OK' with timestamp to
```

```
/clawd/logs/cron-canary.log. 2) [rest of your task]"
}
```

Then check the canary:

```
tail -20 /Users/austenallred/clawd/logs/cron-canary.log
```

If you see recent entries, the cron job ran. If you don't, it didn't — and you know before you've spent an hour wondering.

"The agent keeps making up file paths that don't exist"

This is a hallucination problem. The agent is guessing at what's in your workspace instead of checking.

Fix it by being explicit in your task descriptions:

```
BAD: "Check the sales data and send me a summary"
GOOD: "Read the file at /clawd/data/sales/2026-02-22.csv and send me a summary"
```

If the agent needs to discover what's available, tell it to look first:

```
"First run: ls -la /clawd/data/sales/ to see what files are available.
Then read the most recent file and send me a summary."
```

Giving the agent a discovery step before an action step dramatically reduces made-up paths.

When You're Building Pipelines

"My pipeline keeps failing at the same step"

Three possibilities:

1. **The step is genuinely broken:** The tool call is wrong, the data isn't what you expect, or there's a real error.
2. **The preceding step is producing bad output:** The step looks like it succeeds but produces something the next step can't handle.
3. **The step has an environment issue:** Works locally but fails in cron (path problems, permissions, missing environment variables).

Debug in this order:

First, run the failing step in isolation with known-good inputs. If it fails with known-good inputs, the step itself is broken. If it succeeds, the problem is the inputs it's receiving.

Second, add output logging to the step before the failing one. Print exactly what it's producing. Read that output. Is it what you expected?

Third, if the step works in isolation but fails in cron, check: PATH, working directory, file permissions, environment variables.

"I need to process 10,000 rows and it's too slow"

Batch processing large datasets has a predictable set of solutions. Apply them in order:

First: Parallelize. If the rows are independent of each other, spawn multiple sub-agents and split the work. 4 agents processing 2,500 rows each is 4x faster than 1 agent processing 10,000.

Second: Use the right tool. Python with pandas processes CSV faster than reading it line-by-line. Use `pd.read_csv()` and vectorized operations instead of loops.

Third: Cache what you can. If you're fetching external data for each row, cache it. If row 1000 and row 5000 both call the same API endpoint with the same parameters, you should be making that call once.

Fourth: Process incrementally. Don't try to do all 10,000 at once. Process 100 at a time, save state, resume where you left off. This also makes it restartable when it fails.

Here's a concrete parallel processing pattern:

```
#!/usr/bin/env python3
"""
parallel-processor.py
Splits a large CSV into chunks and processes them in parallel.
"""
import json
import sys
import math
import subprocess
from pathlib import Path
import pandas as pd
from datetime import datetime

INPUT_FILE = Path("/Users/austenallred/clawd/data/large-dataset.csv")
OUTPUT_DIR = Path("/Users/austenallred/clawd/data/chunks")
FINAL_OUTPUT = Path("/Users/austenallred/clawd/data/large-dataset-processed.csv")
CHUNK_SIZE = 1000
WORKER_SCRIPT = Path("/Users/austenallred/clawd/scripts/process-chunk.py")

def split_into_chunks():
    df = pd.read_csv(INPUT_FILE)
    total = len(df)
    num_chunks = math.ceil(total / CHUNK_SIZE)

    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    chunk_files = []
    for i in range(num_chunks):
        start = i * CHUNK_SIZE
        end = min((i + 1) * CHUNK_SIZE, total)
        chunk = df[start:end]

        chunk_file = OUTPUT_DIR / f"chunk-{i:04d}.csv"
        chunk.to_csv(chunk_file, index=False)
        chunk_files.append(chunk_file)

    print(f"Split {total} rows into {num_chunks} chunks of ~{CHUNK_SIZE}")
```

```

return chunk_files

def process_chunks_parallel(chunk_files, max_workers=4):
    """Process chunks in parallel using subprocess."""
    processes = []

    for chunk_file in chunk_files:
        result_file = chunk_file.with_suffix(".processed.csv")
        proc = subprocess.Popen(
            ["python3", str(WORKER_SCRIPT), str(chunk_file), str(result_file)],
            stdout=subprocess.PIPE,
            stderr=subprocess.PIPE
        )
        processes.append((proc, chunk_file, result_file))

    # Limit parallelism
    if len(processes) >= max_workers:
        proc, chunk_file, result_file = processes.pop(0)
        proc.wait()

    # Wait for remaining
    for proc, chunk_file, result_file in processes:
        proc.wait()

    return [p[2] for p in processes]

def merge_results(result_files):
    dfs = [pd.read_csv(f) for f in result_files if f.exists()]
    merged = pd.concat(dfs, ignore_index=True)
    merged.to_csv(FINAL_OUTPUT, index=False)
    print(f"Merged {len(result_files)} chunks into {FINAL_OUTPUT}")
    print(f"Final row count: {len(merged)}")

if __name__ == "__main__":
    start = datetime.now()
    chunks = split_into_chunks()

    # In a real setup, you'd spawn OpenClaw subagents here
    # For simplicity, showing subprocess-based parallelism
    print(f"Processing {len(chunks)} chunks with up to 4 parallel workers...")
    results = process_chunks_parallel(chunks)
    merge_results(results)

    elapsed = (datetime.now() - start).seconds
    print(f"Done in {elapsed}s")

```

"My automation is producing different results at different times"

Non-determinism in automation is frustrating. Track down the source:

Time-dependent behavior: If you're running `datetime.now()` inside your pipeline, results change based on when you run it. Fix: pass the date/time as a parameter so it's explicit.

State that accumulates: If you append to a file each run without deduplication, over time you'll have duplicate data. Fix: always write state as a complete snapshot, not as accumulated appends.

External data that changes: Prices, stock data, weather — it changes. If your test and production runs happen at different times, they'll get different data. Fix: cache the external data at the start of the pipeline run and use the cached version throughout.

Random in the process: If you're using any randomization (sampling, shuffling), pin the random seed:

```
import random
random.seed(42) # Same results every time
```

"I want to automate sending emails, but I don't want to accidentally spam people"

Good instinct. A few safeguards:

Draft mode first. Never send on the first run. Generate drafts to a review folder. Run your pipeline in "draft mode" for a week. If the drafts look right, then think about automation.

Hard recipient limits. In your sending script, add a check:

```
MAX_RECIPIENTS_PER_RUN = 10 # Never send to more than 10 at once

if len(recipients) > MAX_RECIPIENTS_PER_RUN:
    raise ValueError(
        f"Too many recipients: {len(recipients)}. "
        f"Max per run: {MAX_RECIPIENTS_PER_RUN}. "
        "Review the recipient list manually."
    )
```

Dry run mode. Add a `--dry-run` flag to your script that prints what would be sent without actually sending:

```
import argparse

parser = argparse.ArgumentParser()
parser.add_argument("--dry-run", action="store_true",
                    help="Print emails without sending")
args = parser.parse_args()

for recipient in recipients:
    email = generate_email(recipient)
    if args.dry_run:
        print(f"WOULD SEND to {recipient['email']}:")
        print(email)
        print("----")
    else:
        send_email(recipient["email"], email)
```

Unsubscribe tracking. Keep a list of people who've opted out. Check it before every send.

Bounce handling. Track bounced emails and remove them from future sends.

When Your System Is Running

"Something broke overnight and I don't know what"

Start with the logs. Always the logs.

```
# What cron jobs ran? Did they succeed?
cat /Users/austenallred/clawd/logs/cron-history.log | tail -50

# Any error log entries?
grep -i "error\|fail\|exception\|critical" /Users/austenallred/clawd/logs/*.log |
sort | tail -50

# What files changed overnight?
find /Users/austenallred/clawd -newer /tmp/checkpoint -type f 2>/dev/null
# (Create checkpoint with: touch /tmp/checkpoint)

# Disk space?
df -h

# Memory?
vm_stat | grep "Pages free"
```

Work backwards from the last known good state. The logs will tell you approximately when things went wrong. Then look for what changed before that point.

"A cron job ran but produced no output"

Four possibilities:

1. The task description was too vague and the agent didn't know what to produce.
2. The output was written somewhere other than where you expected.
3. The task ran but had an error that was silently swallowed.
4. The task ran and produced output that was logged but not saved.

Check:

```
# Where did the agent write output?
find /Users/austenallred/clawd -newer /tmp/yesterday-checkpoint -type f -name "*.md"
-o -name "*.json" -o -name "*.log"

# What did the cron session log?
cat /Users/austenallred/clawd/logs/cron-history.log | grep "$(date +%Y-%m-%d)"
```

Add explicit output confirmation to your task descriptions:

```
After completing the analysis, write the results to /clawd/reports/analysis-YYYY-MM-DD.md.
Then write one line to /clawd/logs/cron-history.log: "TIMESTAMP | analysis | COMPLETE
| {row count} items"
```

If the log line is there but the report isn't, you have a write failure. If neither is there, the task itself failed or the output went somewhere else.

"I'm getting alerts at 3am for things that aren't actually emergencies"

Alert fatigue is real and it's dangerous — you start ignoring all alerts, including the real ones.

Fix your alert thresholds:

```
BAD: Alert if disk > 70%
GOOD: Alert if disk > 85% (warning) or > 90% (critical)

BAD: Alert for every error
GOOD: Alert if error rate exceeds 5% of total requests, or if same error happens 3+
times in 10 minutes

BAD: Alert 24/7
GOOD: Alert immediately for CRITICAL, delay WARNING alerts until 8am
```

Also check: are your alerts actually actionable? If you get an alert and there's nothing you can do about it at 3am, it shouldn't be a 3am alert.

Update your `protocols/ALERTS.md` with reviewed thresholds. Then update the cron jobs that generate alerts to match.

"Sub-agents keep failing but I don't know why"

The most common sub-agent failure modes:

Context too large: The task description plus loaded context files plus tool output exceeds context limits. The agent gets confused.

Fix: Make tasks smaller and more focused. A task that says "do this entire research project" will fail; a task that says "search for X and summarize the top 5 results" will succeed.

Instructions too vague: The agent doesn't know what "done" looks like. It either does too much, too little, or loops.

Fix: Add explicit completion criteria. "Your task is complete when you have written {file}. Stop when the file exists."

File permission issues: The sub-agent tries to write to a location it doesn't have permission to write to.

Fix: Pre-create the output directories:

```
mkdir -p /Users/austenallred/clawd/projects/20260222-143000/research-artifacts/
```

Network failures: The sub-agent tried to fetch a URL that timed out or returned an error.

Fix: Add retry logic in your task description: "If a fetch fails, try once more. If it fails again, note the failure and continue with other sources."

When You're Scaling Up

"I have too many cron jobs and I can't keep track of them all"

Write a cron inventory:

```
# /clawd/config/CRON-INVENTORY.md

## Active Cron Jobs

| Schedule | Description | Output | Owner |
|-----|-----|-----|-----|
| 0 13 * * 1-5 | Morning briefing | WhatsApp message | main |
| 0 */2 * * * | Heartbeat check | /logs/heartbeat.log | main |
| 0 9 * * 1 | Weekly CI report | /reports/ci-YYYY-WW.md | main |
| 0 16 * * 1-5 | Code review | /reviews/daily-YYYY-MM-DD.md | main |
| 0 22 * * * | Notes index | /notes/INDEX.md | main |
| 30 17 * * 5 | Weekly report | /reports/weekly/YYYY-WW.md | main |
| 0 10 1 * * | Memory compaction | /memory/ | main |
| 0 3 * * 0 | Weekly cleanup | /logs/, /tmp/ | main |

## Disabled Jobs (and why)

| Job | Disabled Date | Reason |
|-----|-----|-----|
| Hourly competitor check | 2026-02-15 | Too aggressive, switched to weekly |
```

Review this quarterly. Disabled jobs that stay disabled for 3 months should be deleted.

"My workspace is getting huge — 50GB of files"

First, understand where the space is going:

```
du -sh /Users/austenallred/clawd/*/ | sort -rh | head -20
```

Common culprits:

- Old log files that never get rotated
- Screenshots piling up in /screenshots/
- Downloaded/cached web content
- Unarchived project files
- Old backup files (.bak, .old)

Add aggressive rotation to your weekly cleanup cron:

```
{
  "schedule": "0 3 * * 0",
  "task": "Weekly cleanup:
1. Delete log files in /clawd/logs/ older than 30 days
2. Delete screenshots in /clawd/screenshots/ older than 14 days
3. Delete files in /clawd/tmp/ older than 7 days
4. Compress project directories for projects completed more than 90 days ago (tar -czf)
5. Delete .DS_Store and Thumbs.db files everywhere in /clawd/
```

```
6. Run: df -h and report disk usage before and after"
}
```

"I want to share my automation system with a team"

Multi-user automation adds complexity. Key considerations:

Namespace your workspace:

```
/clawd/
├─ shared/          # Shared context files
│   ├── SOUL.md
│   └── AGENTS.md
├─ users/
│   ├── austen/     # Your personal workspace
│   └── sarah/      # Sarah's personal workspace
└─ team/
    ├── projects/   # Shared projects
    └── reports/    # Shared outputs
```

Access control:

Different users should have different permissions. Don't let everyone's cron jobs write to the same directories.

Shared context vs. personal context:

SOUL.md describes the team's agent, not any individual. But USER.md is personal — each person has their own.

Coordination:

Who triggers what? What happens when two people trigger the same agent simultaneously? Build these rules into your AGENTS.md before it becomes a problem.

"I need audit trails for compliance reasons"

Add structured audit logging to every action your automation takes:

```
#!/usr/bin/env python3
"""
audit-logger.py
Writes structured audit log entries.
"""

import json
import os
from datetime import datetime
from pathlib import Path

AUDIT_LOG = Path("/Users/austenallred/clawd/logs/audit.jsonl")

def audit_log(action, resource, result, user="automation", details=None):
    """Write an immutable audit log entry."""
```

```

entry = {
    "timestamp": datetime.utcnow().isoformat() + "Z",
    "user": user,
    "action": action,
    "resource": resource,
    "result": result,
    "details": details or {},
    "pid": os.getpid()
}

AUDIT_LOG.parent.mkdir(parents=True, exist_ok=True)
with open(AUDIT_LOG, "a") as f:
    f.write(json.dumps(entry) + "\n")

```

Use it throughout your automation:

```

audit_log("read", "/clawd/data/customers.csv", "success", details={"rows": 1000})
audit_log("write", "/clawd/reports/monthly.md", "success")
audit_log("send_message", "whatsapp:+15551234567", "success")
audit_log("delete", "/clawd/tmp/file.txt", "success")

```

The `.jsonl` format (JSON Lines) makes it easy to process with standard tools:

```

# Find all actions for a specific resource
grep '"resource": "/clawd/data/customers.csv"' /clawd/logs/audit.jsonl | jq .

# Find all failures in the last hour
awk "$(date -v-1H '+%Y-%m-%dT%H' 2>/dev/null || date -d '1 hour ago' '+%Y-%m-%dT%H')" /clawd/logs/audit.jsonl | \
jq 'select(.result == "failure")'

```

Mental Models for Automation

These are the mental models that make experienced automation engineers consistently better than beginners. You can apply them immediately.

"The contract model"

Every tool call is a contract:

- **Preconditions:** What must be true before the call
- **Postconditions:** What will be true after the call succeeds
- **Error conditions:** What happens if it fails

Write these out for your critical operations:

```

def update_customer_status(customer_id, new_status):
    """
    Preconditions:
    - customer_id must exist in database

```

```

- new_status must be one of: active, suspended, cancelled

Postconditions:
- Customer's status is updated to new_status
- A status_change_log entry is created
- If new_status is 'cancelled', send_cancellation_email is called

Error conditions:
- CustomerNotFound: raises ValueError, no state changed
- DatabaseError: raises DatabaseError, state may be partially changed (check
log)
- EmailError: status IS updated but email not sent (log and alert)
"""

```

This discipline catches a huge class of bugs before they happen.

"The idempotency-first model"

Before writing any automation, ask: "What happens if this runs twice?" If the answer is "bad things," redesign until the answer is "the same correct result."

Practically: prefer "set this value" over "add this value." Prefer "write this file" over "append to this file." Prefer "upsert this record" over "insert this record."

"The explicit state model"

Automation that relies on implicit state (order of operations, assumed file contents, external system state) is fragile. Automation that makes state explicit (reads and writes it to known files) is robust.

Every time you catch yourself thinking "the next step will assume X is true," write X to a file.

"The boundary model"

Draw clear boundaries between:

- **Your automation** (what you control)
- **External systems** (what others control)
- **User inputs** (what humans provide)

At every boundary, validate. Don't trust data from external systems. Don't trust data from users. Validate it before using it.

```

def process_external_data(raw_data):
    # Never pass raw_data directly to your functions
    # Always validate at the boundary

    if not isinstance(raw_data, dict):
        raise ValueError(f"Expected dict, got {type(raw_data)}")

    if "price" not in raw_data:
        raise ValueError("Missing required field: price")

    try:
        price = float(raw_data["price"])

```

```

except (TypeError, ValueError):
    raise ValueError(f"Invalid price value: {raw_data['price']}")

if price < 0 or price > 1_000_000:
    raise ValueError(f"Price out of valid range: {price}")

# Now it's safe to use
return {"price": price, "...": "..."}

```

"The minimal surface area model"

Every tool you use, every external service you touch, every cron job you add is a potential failure point. Keep the surface area minimal.

Ask: "Can I accomplish this with fewer moving parts?" Often the answer is yes.

- Use one scheduling mechanism, not three
- Write data to one place, not five
- Call external APIs through one wrapper, not directly everywhere

Less surface area means fewer things to break and fewer things to debug.

"The reversibility model"

Before any destructive operation (delete, overwrite, send), ask: "Can I undo this?"

If not: add a recovery path. Before deleting, archive. Before overwriting, backup. Before sending, draft.

```

def safe_overwrite(target_file, new_content):
    """Overwrite a file, keeping a backup."""
    target = Path(target_file)

    # Backup first
    if target.exists():
        backup = target.with_suffix(".bak")
        backup.write_text(target.read_text())

    # Now overwrite
    target.write_text(new_content)

    # Verify the write
    if target.read_text() != new_content:
        # Restore backup
        if backup.exists():
            target.write_text(backup.read_text())
        raise IOError("Write verification failed, restored from backup")

```

Failure Stories (and What They Teach)

These are things that actually went wrong in real automation systems. Learning from them is cheaper than repeating them.

The Wrong Timezone Deletion

What happened: A daily cleanup job deleted "files older than 1 day" but ran in UTC while files were timestamped in local time (CST, UTC-6). The job ran at midnight UTC, which was 6pm local time the day before. "Files older than 1 day" in UTC included files from today that were created before 6pm local time — essentially wiping today's work at 6pm every day.

Discovery: User noticed today's report was missing. Checked the deletion log. Timestamps didn't match expectations.

Fix: Store a state file that explicitly records "last cleanup ran at TIMESTAMP." Delete files created before that timestamp, not files older than some calculated duration. Timestamps are unambiguous; relative durations depend on timezone interpretation.

Lesson: Be explicit about what "older than" means. Store timestamps, not durations.

The Duplicate WhatsApp Alert Storm

What happened: A heartbeat monitor checked for errors in the log. It sent a WhatsApp alert for each new error found since the last check. The last check timestamp was stored in memory (not persisted). When the Gateway restarted, the timestamp reset to zero — so every error in the entire log history was "new." 47 WhatsApp messages arrived at 3am.

Discovery: The phone vibrating 47 times at 3am.

Fix: Persist the last-checked timestamp to a file:

```
STATE_FILE = Path("/clawd/data/monitor-state.json")

def load_state():
    if STATE_FILE.exists():
        return json.loads(STATE_FILE.read_text())
    return {"last_checked": datetime.min.isoformat()}

def save_state(last_checked):
    STATE_FILE.write_text(json.dumps({"last_checked": last_checked.isoformat()}))
```

Lesson: Never store state only in memory. If the process restarts, memory is gone. Persist everything that matters.

The Infinite Research Loop

What happened: A research sub-agent was tasked with "researching X thoroughly." It ran search queries. Found 10 sources. Decided it needed more depth. Searched for more. Found 20 more sources. Decided it still wasn't thorough enough. Searched again. This continued for 3 hours consuming significant API cost before the context window forced a stop.

Discovery: Unusual API bill. No research output.

Fix: Add explicit stopping criteria to research tasks:

```
Research X. You are done when:
- You have found at least 5 credible sources, OR
```


- You have spent 30 minutes researching, OR
 - You have made 10 search queries
- whichever comes first.

Then write your synthesis. Stop researching.

Lesson: "Be thorough" is not a stopping condition. Every task needs explicit termination criteria.

The Schema Mismatch Silent Failure

What happened: A pipeline read a CSV, transformed it, and wrote JSON output. A schema change in the source CSV (a column was renamed from "customer_id" to "customerId") caused the transformation to silently produce null values for the customer ID field. The JSON output looked valid. Downstream systems received null customer IDs for two weeks before anyone noticed.

Discovery: Customer complained about reports showing "Unknown" customer.

Fix: Add schema validation at every boundary:

```
def validate_csv_schema(df):
    REQUIRED_COLUMNS = ["customer_id", "date", "amount", "product"]
    missing = set(REQUIRED_COLUMNS) - set(df.columns)
    if missing:
        raise ValueError(f"CSV missing required columns: {missing}")

    # Check for unexpected nulls in key fields
    null_ids = df["customer_id"].isnull().sum()
    if null_ids > 0:
        raise ValueError(f"Found {null_ids} null customer_id values")
```

Lesson: Null values and missing data flow silently. Validate at input boundaries, not just at output. A failed validation that raises an error is better than silent corruption.

The Cron Job That Never Stopped

What happened: A data processing job was moved from a cron schedule to a "run continuously with a sleep" pattern (for performance reasons). The sleep was 5 minutes. Everything was fine. Then the processing started taking longer than 5 minutes. Two instances ran at the same time. They both read the same input, processed the same data, and both wrote output. The second write overwrote the first. Some data was processed twice, some was lost.

Discovery: Data count discrepancies in the monthly audit.

Fix: Use a lock file to prevent concurrent execution:

```
import fcntl
import sys
from pathlib import Path

LOCK_FILE = Path("/tmp/my-process.lock")

def acquire_lock():
```

```

lock_fd = open(LOCK_FILE, "w")
try:
    fcntl.flock(lock_fd, fcntl.LOCK_EX | fcntl.LOCK_NB)
    return lock_fd
except BlockingIOError:
    print("Another instance is running. Exiting.")
    sys.exit(0)

# At the start of your script:
lock = acquire_lock()
# ... do work ...
# Lock is released automatically when process exits

```

Lesson: Concurrent automation processes are trouble. If two instances might run, design for it explicitly or prevent it with locks.

The Agent That Rewrote Production Config

What happened: A maintenance agent was tasked with "cleaning up old config files in /clawd/config/." The agent found a config file that looked outdated based on its modification date. It deleted it. The file was actually `openclaw.json` — the Gateway's config file. The Gateway couldn't restart without it.

Discovery: Nothing worked. Gateway wouldn't start.

Fix 1 (immediate): Restore from git:

```
git checkout HEAD openclaw.json
```

Fix 2 (preventive): Never let maintenance agents touch files in the root workspace directory without explicit file lists:

```

Clean up old config files. ONLY delete files from these specific directories:
- /clawd/config/old/
- /clawd/config/archive/

NEVER touch:
- /clawd/openclaw.json
- /clawd/SOUL.md
- /clawd/MEMORY.md
- /clawd/AGENTS.md

```

Fix 3 (structural): Use git. If your important files are version-controlled, accidental deletion is recoverable.

Lesson: Be explicit about what maintenance agents can touch. Use allowlists, not blocklists. And use version control.

The Cascading Alert Storm

What happened: One service dependency went down. This caused:

- Service A to start returning errors
- Service B (which depends on A) to start returning errors

- Service C (which depends on B) to start returning errors

The alerting system sent alerts for each failing service, plus alerts for the elevated error rates, plus alerts for the failed health checks. 23 alerts in 2 minutes.

Discovery: The 23 alerts. Also the service being down.

Fix: Alert deduplication and correlation:

```
def should_alert(alert_key, cooldown_minutes=30):
    """Suppress duplicate alerts within cooldown period."""
    state_file = Path(f"/clawd/data/alert-state-{alert_key}.json")

    if state_file.exists():
        state = json.loads(state_file.read_text())
        last_alerted = datetime.fromisoformat(state["last_alerted"])
        if datetime.now() - last_alerted < timedelta(minutes=cooldown_minutes):
            return False # In cooldown

    # Update last alerted time
    state_file.write_text(json.dumps({"last_alerted": datetime.now().isoformat()}))
    return True # Alert

# Usage:
if error_count > threshold and should_alert("service-a-errors"):
    send_whatsapp_alert("Service A errors elevated")
```

Lesson: Alerts need cooldowns. The second alert for the same problem 30 seconds after the first adds no information and lots of noise.

The Philosophy of Automation

I'm going to take two pages to get philosophical. Skip this if you're not interested. But I think it matters.

What Automation Is Actually For

The standard pitch for automation is time savings. Automate repetitive tasks, free up time for creative work.

That's real, but it's underselling it.

The deeper value of automation is *reliability*. A human doing something repetitive will eventually skip a step, forget a detail, or just have a bad day and do it wrong. A well-built automation does it the same way every time.

If you're running a business and you have processes that need to happen correctly every single time — compliance checks, customer communications, financial records — automation isn't just faster. It's safer. Humans make mistakes. Well-designed automation doesn't.

That said: automation makes new kinds of mistakes. A human who does something wrong usually does it wrong once. Automation that does something wrong does it wrong every time, until someone notices. The mistake amplifier is the other side of the reliability story.

This is why validation, logging, and monitoring aren't optional add-ons. They're what makes automation actually more reliable than humans, rather than just faster.

The Human-in-the-Loop Question

For any automation you build, you should explicitly decide where humans are in the loop.

Full automation: Humans set it up, then it runs indefinitely without human involvement. Right for: low-stakes, reversible, well-understood processes. Examples: log rotation, status reports, file organization.

Draft automation: Automation does the work, humans review and approve before output is acted on. Right for: higher-stakes outputs that need judgment. Examples: customer communications, significant code changes, financial actions.

Decision automation: Automation gathers information and presents options; humans decide. Right for: complex tradeoffs, situations requiring judgment, anything with real consequences. Examples: release decisions, architectural changes, hiring.

Alert automation: Automation monitors for conditions and pings humans to act. Right for: time-sensitive situations where human judgment is essential. Examples: production incidents, security events, anomalies.

Most people default to full automation for everything because it feels like "real" automation. But the right level of human oversight depends on the consequences of getting it wrong. Build the appropriate level of human checkpoints in from the start.

On AI Agents and Trust

OpenClaw agents are impressive. They can write code, navigate websites, coordinate complex workflows, and manage persistent state across weeks of operation. It's easy to over-trust them.

A few things to keep in mind:

Agents can be confidently wrong. They'll sometimes do the wrong thing without any indication that they know it's wrong. This is why validation, logging, and monitoring matter.

Agents don't understand consequences. They don't have skin in the game. If they accidentally delete something important, they don't lose anything. You do. Design your automation with this asymmetry in mind.

Agents are as good as their instructions. Vague instructions produce inconsistent results. Precise instructions produce reliable ones. The quality of your automation is largely the quality of your task specifications.

Trust should be earned incrementally. Start with low-stakes tasks. Validate heavily. As you develop confidence in specific patterns, reduce validation overhead. Don't start with high-stakes automation and assume it'll be fine.

None of this means you shouldn't use agents for serious work. You absolutely should. It means you should do it with appropriate discipline — the same discipline you'd apply to any critical system.

The Automation Mindset Shift

People who use automation well think about their work differently from people who don't.

Before automation mindset: "I need to do X." Followed by actually doing X.

Automation mindset: "I need X to happen. How can I set up the world so X happens reliably without me doing it?"

The shift is from doing to designing. You stop being the worker and start being the architect of systems that do the work.

This shift takes time. At first, setting up automation feels slower than just doing the thing. It is slower — the first time. The payoff comes from the tenth time, the hundredth time, the thousandth time the thing happens without you touching it.

Build the automation first for processes that will definitely repeat. That's where the return on investment is obvious. Over time, you'll develop an intuition for what's worth automating and what isn't. That intuition is worth cultivating. It's the skill underneath all the technical skills this book teaches.

Good luck. Build useful things.

Appendix G: Full Chapter Expansions

Expanded: Understanding AI Models in OpenClaw

Most of this book treats the AI model as a black box — you give it instructions, it does things. That's fine for getting started, but understanding a bit more about how the models work will make you a better automation designer.

Models Available

OpenClaw supports multiple AI models. The current default is typically a Claude model from Anthropic. The model you use affects:

- **Quality:** Bigger models generally produce better results
- **Speed:** Smaller models respond faster
- **Cost:** More capable models cost more per token
- **Context:** Models have different context window sizes

For most interactive work, you'll use the default model. For cron jobs that run frequently but don't need peak quality, consider configuring a smaller, faster model.

In `openclaw.json` :

```
{
  "model": "anthropic/claude-sonnet-4-5",
  "cron": [
    {
      "schedule": "*/15 * * * *",
      "task": "Quick health check",
      "model": "anthropic/claude-haiku-3-5"
    }
  ]
}
```

The haiku model handles simple "check this file, if X then Y" logic fine and costs significantly less. Reserve the more capable models for tasks that actually need reasoning.

How Context Windows Work

A "token" is roughly 4 characters of text, or about 0.75 words. A sentence is 10-20 tokens. A paragraph is 50-100 tokens. This page is about 400 tokens.

Modern models handle 100,000+ tokens in their context. That sounds like a lot, but it fills up faster than you'd expect when you're loading multiple context files at session start.

Here's roughly what happens at session start for a typical OpenClaw agent:

System prompt:	2,000 tokens
SOUL.md:	500 tokens
MEMORY.md:	1,500 tokens
AGENTS.md:	3,000 tokens
USER.md:	300 tokens
Today's memory log:	800 tokens
Recent conversation history:	5,000 tokens

Session start overhead:	~13,100 tokens

You've used 13,000 tokens before the agent does anything. Then each tool call adds input and output tokens. A web_fetch of a large page might add 10,000+ tokens. A file read of a large log file might add 5,000 tokens.

For a 100,000 token context, 13,000 at startup leaves 87,000 for actual work. That's plenty for most sessions. But for long sessions with many large file reads, you'll start hitting limits.

Practical implication: Keep your context files concise. MEMORY.md that's 200 lines vs. 50 lines is a real difference. Run memory compaction regularly.

Thinking vs. Not Thinking

Some Claude models support "extended thinking" mode, where the model reasons through problems step-by-step before responding. This produces better results for complex problems but costs more and takes longer.

For automation tasks:

- **Extended thinking:** Worth it for complex analysis, multi-step planning, difficult code generation
- **Standard mode:** Fine for simple tasks, routine operations, straightforward tool calls

You can configure this in your prompts or via model selection. Check current OpenClaw docs for exact configuration.

Why Models Sometimes Fail

Understanding common failure modes helps you design around them:

Hallucination: The model generates plausible-sounding but false information. Especially common for specific facts (dates, names, numbers), API signatures, and file paths.

Mitigation: Verify anything the model claims about the external world. Don't trust it to "know" your API's current endpoint structure — have it check.

Instruction drift: In long sessions, the model gradually loses track of early instructions. It's not ignoring them; they've just become less prominent in its attention.

Mitigation: For important constraints, repeat them. "Remember: do not modify files outside /clawd/projects/" is worth repeating in long task descriptions.

Context contamination: Information from earlier in the session bleeds into current responses. You asked about competitor A an hour ago; now when you ask about pricing, the model keeps bringing up competitor A.

Mitigation: Start fresh sessions for unrelated tasks. Don't let one session span too many different topics.

Overconfidence: The model acts on incorrect assumptions without flagging uncertainty. It shouldn't say "I think" when it should say "I checked and confirmed."

Mitigation: Explicitly prompt for uncertainty acknowledgment: "If you're not sure about something, say so rather than assuming."

Expanded: The Complete Workspace Setup Guide

If you're starting from scratch, this section walks you through a complete workspace setup in one sitting.

Hour 1: Basic Infrastructure

Start the Gateway:

```
openclaw gateway start
openclaw gateway status
```

Verify it's running and note the workspace path.

Create directory structure:

```
cd /Users/austenallred/clawd

mkdir -p memory logs data scripts reports projects tasks notes
mkdir -p templates config protocols secrets
mkdir -p reports/{daily,weekly,monthly}
mkdir -p data/{metrics,prices,health}
mkdir -p logs/{archive}

chmod 700 secrets/
```

Write SOUL.md:

```
cat > SOUL.md << 'EOF'
# SOUL.md

## Identity
You are a personal automation assistant for [Your Name].
You help with research, writing, code, and managing scheduled tasks.
You're direct, competent, and focused on working solutions.

## Principles
1. Verify before acting – read the state before changing it
2. Log what matters – write important actions to memory logs
3. Prefer reversible actions – backup before overwriting
```

4. Ask when uncertain – don't assume

```
## Communication Style
- Direct and concise
- Lead with the answer
- WhatsApp messages: under 200 words

## Always Do
- Write significant events to memory/YYYY-MM-DD.md
- Check disk space before large operations
- Confirm before any deletion

## Never Do
- Delete without confirmation
- Send messages to third parties without review
- Store credentials in memory or conversation
EOF
```

Write MEMORY.md:

```
cat > MEMORY.md << 'EOF'
# MEMORY.md

## Setup
- Created: $(date +%Y-%m-%d)
- Workspace: /Users/austenallred/clawd
- Gateway: running on this machine

## System Configuration
- Scripts: /clawd/scripts/
- Data: /clawd/data/
- Reports: /clawd/reports/

## Preferences
[Add preferences as you discover them]
EOF
```

Write USER.md:

```
cat > USER.md << 'EOF'
# USER.md

## About Me
Name: [Your Name]
Timezone: America/Chicago
Location: Austin, TX

## Communication Preferences
- Preferred channel: WhatsApp
- Best time for non-urgent notifications: 8am–6pm CST
- Quiet hours: 10pm–7am (only critical alerts)
```



```
## Work Context
[Add context about your work]
EOF
```

Hour 2: First Automations

Create the morning briefing:

```
cat > scripts/morning-brief.sh << 'SCRIPT'
#!/bin/bash
WORKSPACE="/Users/austenallred/clawd"
DATE=$(date +%Y-%m-%d)
BRIEFING="$WORKSPACE/briefings/$DATE.md"

mkdir -p "$WORKSPACE/briefings"

cat > "$BRIEFING" << BRIEF
# Briefing - $(date '+%A, %B %-d, %Y')
Generated: $(date +%H:%M)

## Top Priorities
$(grep "^- \[" "]" "$WORKSPACE/tasks/current.md" 2>/dev/null | head -3 | sed 's/^- \[
\] /- /' || echo "- No tasks file found")

## Overnight Alerts
$(grep "$(date -v-12H +%Y-%m-%dT)" "$WORKSPACE/logs/alerts.log" 2>/dev/null | head
-5 || echo "- No alerts")

BRIEF

cat "$BRIEFING"
SCRIPT

chmod +x scripts/morning-brief.sh
```

Create your first task file:

```
cat > tasks/current.md << 'EOF'
# Active Tasks

## High Priority
- [ ] Set up morning briefing automation
- [ ] Test the complete workspace setup

## Normal Priority
- [ ] Read the rest of the OpenClaw handbook

## Someday/Maybe
```

```
- [ ] Automate weekly reports
EOF
```

Add cron jobs to openclaw.json:

```
# Read the current openclaw.json first
cat openclaw.json

# Then add cron section (via agent or manual edit)
```

Tell the agent:

```
Add these cron jobs to openclaw.json:
1. Daily at 7am UTC: Morning briefing - run /clawd/scripts/morning-brief.sh, fetch
weather for Austin TX, send me a WhatsApp summary
2. Every 2 hours: Heartbeat - check disk space, log result to
/clawd/logs/heartbeat.log, alert if disk > 80%

Restart the gateway after making changes.
```

Hour 3: Test and Verify

Test the morning briefing manually:

```
Run the morning briefing task right now for testing. Show me the output.
```

Verify cron is configured:

```
openclaw gateway status
cat openclaw.json | grep -A 20 '"cron"'
```

Test WhatsApp if configured:

```
Send me a test WhatsApp message: "Workspace setup complete. Everything looks good."
```

Set up git:

```
cd /Users/austenallred/clawd

# Create .gitignore
cat > .gitignore << 'EOF'
secrets/
logs/*.log
logs/archive/
reports/
data/
briefings/
memory/
*.bak
.DS_Store
```

```
EOF
```

```
git init
git add .
git commit -m "Initial workspace setup"
```

Your workspace is now functional. You have:

- Core context files (SOUL, MEMORY, USER)
- Morning briefing cron job
- Heartbeat monitoring
- Version control
- Clean directory structure

From here, add automations one at a time as you identify what you need.

Expanded: Prompt Engineering for Automation

The prompts (task descriptions) you write for your automation are as important as the code. Here's how to write better ones.

The Anatomy of a Good Automation Prompt

A complete automation task description has six parts:

1. Trigger context (what just happened or why this is running)

```
This task runs as a daily cron job at 7am weekdays.
```

2. Inputs (what data to use and where to find it)

```
Read: /clawd/tasks/current.md for today's task list
Read: /clawd/memory/YYYY-MM-DD.md for yesterday's memory log
```

3. Processing steps (numbered, explicit)

1. Extract all unchecked tasks from current.md
2. Sort by priority (High first, then Normal)
3. Take the top 5 only

4. Outputs (what to create/send/write)

```
Write the summary to /clawd/briefings/YYYY-MM-DD.md
Send a WhatsApp message with the top 5 tasks
```

5. Error handling (what to do if steps fail)

```
If current.md doesn't exist: send WhatsApp saying "No tasks file found"
If WhatsApp fails: log to /clawd/logs/errors.log and continue
```

6. Completion signal (how to know you're done)

Task is complete when the WhatsApp message is sent and the briefing file exists.

Putting this together:

This task runs as the daily morning briefing at 7am weekdays.

Inputs:

- Task list: /clawd/tasks/current.md
- Yesterday's log: /clawd/memory/{yesterday's date}.md

Processing:

1. Read current.md, extract unchecked tasks (lines starting with "- []")
2. Sort by section: High Priority first, then Normal Priority
3. Take the top 5 tasks only (3 high + 2 normal if available)
4. Get current weather for Austin, TX via web search

Output:

1. Write formatted briefing to /clawd/briefings/{today's date}.md
2. Send WhatsApp message with: today's date, weather (1 line), top 5 tasks (numbered)
Keep message under 150 words.

Error handling:

- If current.md missing: include "No task file found" in briefing
- If web search fails: skip weather, note "weather unavailable"
- If WhatsApp fails: log error to /clawd/logs/errors.log

Complete when: briefing file exists AND WhatsApp sent (or error logged).

This prompt is longer, but it produces reliable results. Vague prompts produce inconsistent results.

Prompt Patterns That Work

The "verify then act" pattern:

Before making any changes:

1. Read the current state of [file]
2. Confirm [condition] is true
3. Only then proceed with [action]
4. If [condition] is false: [alternative action]

This prevents the agent from acting on incorrect assumptions.

The "write as you go" pattern:

After each major step, write a one-line status to /clawd/logs/pipeline-trace.log.

Format: TIMESTAMP | STEP_NAME | STATUS | DETAIL

This creates a trace of what happened, invaluable for debugging.

The "explicit outputs" pattern:

Create exactly these files:

- /clawd/reports/YYYY-MM-DD-summary.md (required)

- /clawd/data/metrics-YYYY-MM-DD.json (required)
- /clawd/logs/pipeline-YYYY-MM-DD.log (created as side effect)

If any required file is missing at the end, report an error.

Explicit output requirements make it obvious when something went wrong.

The "bounded search" pattern:

Search for relevant information using at most 5 search queries.
Read at most 8 web pages total.
Stop when you have found 3+ credible sources.
Do not continue searching indefinitely.

Without bounds, research tasks can run forever.

The "format spec" pattern:

Output format requirements:

- WhatsApp message: plain text, under 200 words, no markdown
- Report file: markdown with H2 sections, include source URLs
- Data file: valid JSON, minified (no indentation)

Format specifications eliminate a whole class of "the output looks wrong" bugs.

Common Prompt Mistakes

Missing file paths:

BAD: "Read the sales data and generate a report"
GOOD: "Read /clawd/data/sales/2026-02-22.csv and write the report to /clawd/reports/sales/2026-02-22.md"

Missing error handling:

BAD: "Fetch the pricing page and extract prices"
GOOD: "Fetch https://example.com/pricing. If the fetch fails or returns non-200, write 'FETCH_FAILED' to /clawd/data/prices-status.txt and exit. Otherwise extract prices..."

Vague stopping conditions:

BAD: "Research competitors thoroughly"
GOOD: "Research competitors. Stop when you have found pricing for at least 3 competitors OR completed 10 search queries, whichever comes first."

Missing output validation:

BAD: "Generate the analysis and save it"
GOOD: "Generate the analysis and save to /clawd/reports/analysis.md. Verify the file was created and is non-empty. If the file is empty or missing, report an error."

Undeclared dependencies:

```
BAD: "Compare this week's metrics to last week's"
GOOD: "Read this week's metrics from /clawd/data/metrics-2026-W08.json and last week's
from /clawd/data/metrics-2026-W07.json. If either file is missing, note which is
missing and abort."
```

Expanded: Integration Cookbook

Integrating with Google Sheets

Google Sheets is where lots of business data lives. Here's how to get data in and out.

Reading from Sheets (via export):

The simplest approach: export as CSV and read the CSV.

```
Every morning, fetch https://docs.google.com/spreadsheets/d/{SHEET_ID}/export?
format=csv&gid={GID}
Save to /clawd/data/sheet-data-YYYY-MM-DD.csv
Then process normally.
```

This works if the sheet is publicly accessible or if you're logged in (via Chrome relay).

Writing to Sheets (via Apps Script API):

Google Sheets has a REST API. You can write data via HTTP:

```
import requests
import json

def append_to_sheet(spreadsheet_id, range_name, values, api_key):
    url = (
        f"https://sheets.googleapis.com/v4/spreadsheets/{spreadsheet_id}"
        f"/values/{range_name}:append"
    )

    response = requests.post(
        url,
        params={
            "valueInputOption": "RAW",
            "key": api_key
        },
        json={
            "values": values
        }
    )

    return response.json()

# Usage:
result = append_to_sheet(
    spreadsheet_id="your-sheet-id",
    range_name="Sheet1!A:D",
```

```

values=[["2026-02-22", "Revenue", 15000, "Notes here"]],
api_key=get_api_key("google-sheets")
)

```

Integrating with Notion

If you use Notion, the API lets you read and write database entries.

```

import requests

NOTION_TOKEN = get_api_key("notion")
DATABASE_ID = "your-database-id"

def query_notion_database(filter_params=None):
    response = requests.post(
        f"https://api.notion.com/v1/databases/{DATABASE_ID}/query",
        headers={
            "Authorization": f"Bearer {NOTION_TOKEN}",
            "Notion-Version": "2022-06-28",
            "Content-Type": "application/json"
        },
        json={"filter": filter_params} if filter_params else {}
    )
    return response.json()

def create_notion_page(properties):
    response = requests.post(
        "https://api.notion.com/v1/pages",
        headers={
            "Authorization": f"Bearer {NOTION_TOKEN}",
            "Notion-Version": "2022-06-28",
            "Content-Type": "application/json"
        },
        json={
            "parent": {"database_id": DATABASE_ID},
            "properties": properties
        }
    )
    return response.json()

```

Syncing task completions to Notion:

```

{
  "schedule": "0 17 * * 1-5",
  "task": "Sync today's completions to Notion: Read /clawd/memory/{today}.md and
extract any lines marked with ✅. For each completion, create a Notion page in the
'Completions' database using the Python Notion integration at /clawd/scripts/notion-
sync.py."
}

```

Integrating with GitHub

If you work with code, GitHub integration is high-leverage.

Check PR status:

```
import requests

def get_open_prs(owner, repo, github_token):
    response = requests.get(
        f"https://api.github.com/repos/{owner}/{repo}/pulls",
        headers={"Authorization": f"token {github_token}"},
        params={"state": "open", "per_page": 20}
    )

    prs = response.json()
    return [{
        "number": pr["number"],
        "title": pr["title"],
        "author": pr["user"]["login"],
        "created_at": pr["created_at"],
        "review_requested": bool(pr.get("requested_reviewers"))
    } for pr in prs]
```

Daily PR digest:

```
{
  "schedule": "0 9 * * 1-5",
  "task": "GitHub PR digest: Run python3 /clawd/scripts/github-prs.py for each repo
in /clawd/config/repos.json. Compile open PRs, their age, and review status. Send me
a WhatsApp if any PR is older than 3 days without review. Save full digest to
/clawd/reports/prs/YYYY-MM-DD.md."
}
```

Integrating with Slack

If your team uses Slack, you can send messages and read channel activity via the Slack API.

```
from slack_sdk import WebClient
from slack_sdk.errors import SlackApiError

def send_slack_message(channel, message, token):
    client = WebClient(token=token)
    try:
        result = client.chat_postMessage(
            channel=channel,
            text=message
        )
        return {"ok": True, "ts": result["ts"]}
    except SlackApiError as e:
        return {"ok": False, "error": str(e)}
```



```
def get_channel_messages(channel, count=20, token=None):
    client = WebClient(token=token)
    result = client.conversations_history(
        channel=channel,
        limit=count
    )
    return result["messages"]
```

Automated Slack standups:

```
{
  "schedule": "0 9 * * 1-5",
  "task": "Post daily standup to Slack: Read /clawd/memory/{yesterday}.md for
yesterday's work. Read /clawd/tasks/current.md for today's top 3 tasks. Format as
standup (Yesterday / Today / Blockers). Post to Slack #engineering channel using
/clawd/scripts/slack-standup.py."
}
```

Expanded: Monitoring and Observability

Real automation systems need monitoring. Here's a complete monitoring setup.

The Monitoring Stack

Your monitoring stack has four layers:

Layer 1: Application logging Every script logs what it does and what happened.

Layer 2: Cron execution tracking Every cron job logs its execution result.

Layer 3: Health checks Periodic checks of system state (disk, memory, services).

Layer 4: Alerting Notifications when something needs attention.

Application Logging

Every automation script should log:

- Start time
- Key inputs used
- What was processed
- What was output
- Any errors
- End time and status

Standard log format:

```
2026-02-22T09:00:01Z | INFO | morning-brief | start
2026-02-22T09:00:02Z | INFO | morning-brief | read tasks: 7 items
2026-02-22T09:00:04Z | INFO | morning-brief | fetch weather: OK
2026-02-22T09:00:05Z | INFO | morning-brief | send whatsapp: OK
2026-02-22T09:00:05Z | INFO | morning-brief | complete: success in 4s
```

Python logging module setup:

```
import logging
import sys
from pathlib import Path

def setup_logging(script_name):
    log_file = Path(f"/Users/austenallred/clawd/logs/{script_name}.log")
    log_file.parent.mkdir(parents=True, exist_ok=True)

    formatter = logging.Formatter(
        "%(asctime)s | %(levelname)-5s | {name} | %(message)s".format(name=script_name),
        datefmt="%Y-%m-%dT%H:%M:%SZ"
    )

    # File handler
    file_handler = logging.FileHandler(log_file)
    file_handler.setFormatter(formatter)

    # Stream handler (for agent to read output)
    stream_handler = logging.StreamHandler(sys.stdout)
    stream_handler.setFormatter(formatter)

    root = logging.getLogger()
    root.setLevel(logging.INFO)
    root.addHandler(file_handler)
    root.addHandler(stream_handler)

    return logging.getLogger(script_name)

# Usage:
log = setup_logging("morning-brief")
log.info("start")
log.info(f"read tasks: {len(tasks)} items")
```

Metrics Collection

Track metrics over time to understand trends:

```
#!/usr/bin/env python3
"""
metrics-collector.py
Collects and stores operational metrics.
"""

import json
from datetime import datetime
from pathlib import Path

METRICS_DIR = Path("/Users/austenallred/clawd/data/metrics")
```

```

def record_metric(name, value, unit=None, tags=None):
    """Record a single metric data point."""
    METRICS_DIR.mkdir(parents=True, exist_ok=True)

    metric_file = METRICS_DIR / f"{name}.jsonl"

    entry = {
        "timestamp": datetime.utcnow().isoformat() + "Z",
        "name": name,
        "value": value,
        "unit": unit,
        "tags": tags or {}
    }

    with open(metric_file, "a") as f:
        f.write(json.dumps(entry) + "\n")

def read_metric_history(name, days=30):
    """Read historical values for a metric."""
    metric_file = METRICS_DIR / f"{name}.jsonl"
    if not metric_file.exists():
        return []

    from datetime import timedelta
    cutoff = datetime.utcnow() - timedelta(days=days)

    entries = []
    for line in metric_file.read_text().splitlines():
        if not line.strip():
            continue
        entry = json.loads(line)
        ts = datetime.fromisoformat(entry["timestamp"].rstrip("Z"))
        if ts > cutoff:
            entries.append(entry)

    return entries

# Usage in your scripts:
# record_metric("disk_usage_percent", 45.2, unit="%", tags={"path": "/"})
# record_metric("pipeline_duration_seconds", 23, tags={"pipeline": "morning-brief"})
# record_metric("items_processed", 47, tags={"source": "sales-csv"})

```

Weekly metrics report:

```

{
    "schedule": "0 9 * * 1",
    "task": "Weekly metrics: Read /clawd/data/metrics/ for all metric files. For each
metric, calculate: last 7 days average, trend (up/down/stable), any anomalies
(values > 2x average). Write report to /clawd/reports/metrics-YYYY-WW.md. If any
metric shows concerning trends (disk growing, errors increasing, performance

```

```
declining), include in WhatsApp summary."
}
```

Setting Up Dashboards

For a simple dashboard, write a status file that any tool can read:

```
#!/usr/bin/env python3
"""
update-dashboard.py
Writes a current system status to the dashboard file.
"""

import json
import shutil
from datetime import datetime
from pathlib import Path

DASHBOARD = Path("/Users/austenallred/clawd/STATUS.md")

def get_disk_status():
    usage = shutil.disk_usage("/Users/austenallred/clawd")
    pct = (usage.used / usage.total) * 100
    status = "🔴" if pct > 90 else "🟡" if pct > 75 else "🟢"
    return f"{status} Disk: {pct:.0f}% used ({usage.free / 1024**3:.1f} GB free)"

def get_cron_status():
    log_file = Path("/Users/austenallred/clawd/logs/cron-history.log")
    if not log_file.exists():
        return "⚠️ Cron: No history log"

    lines = log_file.read_text().strip().splitlines()
    if not lines:
        return "⚠️ Cron: History log empty"

    last_line = lines[-1]
    return f"✅ Cron: Last run {last_line[:19]}"

def main():
    sections = [
        f"## System Status",
        f"Updated: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}",
        "",
        "## System",
        get_disk_status(),
        get_cron_status(),
        "",
        f"*Auto-updated every 30 minutes*"
    ]

    DASHBOARD.write_text("\n".join(sections))
```

```
print("Dashboard updated")

if __name__ == "__main__":
    main()
```

```
{
  "schedule": "*/30 * * * *",
  "task": "Update dashboard: Run python3 /clawd/scripts/update-dashboard.py"
}
```

Expanded: Working with Local Language Models

Sometimes you don't want to send data to external APIs — for privacy reasons, cost reasons, or to work offline. OpenClaw can work with locally running language models via Ollama or similar tools.

Setting Up Ollama

Ollama runs open-source language models locally on your machine:

```
# Install (macOS)
brew install ollama

# Pull a model
ollama pull llama3.2
ollama pull mistral

# Start the server
ollama serve
```

Once running, Ollama exposes a local API at `http://localhost:11434` .

Calling Local Models from Scripts

```
import requests
import json

def ask_local_model(prompt, model="llama3.2"):
    """Send a prompt to a local Ollama model."""
    response = requests.post(
        "http://localhost:11434/api/generate",
        json={
            "model": model,
            "prompt": prompt,
            "stream": False
        },
        timeout=120
    )

    result = response.json()
```

```
    return result.get("response", "")

# Usage:
summary = ask_local_model(
    "Summarize this in 3 bullet points: " + document_text,
    model="llama3.2"
)
```

When to Use Local Models

Use local models for:

- Processing sensitive data you don't want to send externally
- High-volume, simple tasks where cost matters
- Offline work
- Tasks where a smaller model is sufficient (classification, extraction, summarization)

Stick with cloud models for:

- Complex reasoning
- Code generation
- Tasks requiring broad knowledge
- Quality-critical outputs

A hybrid approach that works well: use local models for preprocessing and classification, then pass only the relevant, non-sensitive portions to cloud models for reasoning.

```
def process_sensitive_document(text):
    # Step 1: Extract and redact PII locally
    redacted_text = ask_local_model(
        f"Extract and replace all names, addresses, phone numbers, and emails "
        f"with [REDACTED]. Return only the redacted text:\n\n{text}"
    )

    # Step 2: Classify the document locally
    category = ask_local_model(
        f"Classify this document as: contract, invoice, report, email, or other. "
        f"Respond with one word only:\n\n{redacted_text[:500]}"
    ).strip().lower()

    # Step 3: Deep analysis with cloud model (no PII)
    # Now safe to send to cloud because PII was removed
    analysis = cloud_model_analyze(redacted_text, category)

    return analysis
```

Expanded: Automation for Developers

If you write code professionally, OpenClaw can automate a surprising amount of your development workflow.

The Development Loop Automation

Developers spend a lot of time on routine tasks that surround the actual coding: running tests, checking coverage, reviewing linting output, updating documentation, syncing dependencies. Most of this can be automated.

Pre-commit checklist:

```
cat > /Users/austenallred/clawd/scripts/pre-commit-check.sh << 'SCRIPT'
#!/bin/bash
set -e
REPO_PATH="${1:-.}"
cd "$REPO_PATH"

echo "=== Pre-commit checks ==="

# Run linter
echo "Linting..."
if command -v ruff &> /dev/null; then
    ruff check . || { echo "FAIL: Linting failed"; exit 1; }
fi

# Run type checker
echo "Type checking..."
if command -v mypy &> /dev/null; then
    mypy . --ignore-missing-imports || { echo "FAIL: Type check failed"; exit 1; }
fi

# Run tests
echo "Running tests..."
if [ -f "pytest.ini" ] || [ -f "pyproject.toml" ]; then
    python -m pytest tests/ -x -q || { echo "FAIL: Tests failed"; exit 1; }
fi

# Check for debug code
echo "Checking for debug artifacts..."
if grep -r "pdb.set_trace\|breakpoint()\|console.log\|debugger;" --include="*.py" --include="*.js" --include="*.ts" .; then
    echo "FAIL: Debug code found (see above)"
    exit 1
fi

echo "=== All checks passed ==="
SCRIPT

chmod +x /Users/austenallred/clawd/scripts/pre-commit-check.sh
```

Automated dependency auditing:

```
{
  "schedule": "0 10 * * 1",
  "task": "Weekly dependency audit: For each Python project in /clawd/projects/,
run: pip list --outdated --format=json > /tmp/outdated.json. Also run: safety check
```

```
(if installed) to check for security vulnerabilities. Write combined report to
/clawd/reports/dependencies-YYYY-WW.md. Send WhatsApp if any critical security
vulnerabilities found."
}
```

API documentation generator:

Read the FastAPI/Flask routes in /projects/myapp/app/routes/.
For each route, extract: method, path, parameters, return type.
Generate OpenAPI-compatible documentation.
Save to /projects/myapp/docs/api-reference.md.
If any routes lack docstrings, list them as "needs documentation."

Code Quality Tracking

Track code quality metrics over time:

```
#!/usr/bin/env python3
"""
code-quality-metrics.py
Collects and tracks code quality metrics.
"""

import subprocess
import json
from datetime import datetime
from pathlib import Path

REPO_PATH = Path("/Users/austenallred/projects/myapp")
METRICS_FILE = Path("/Users/austenallred/clawd/data/code-quality.jsonl")

def count_files_by_extension(path):
    counts = {}
    for f in path.rglob("*"):
        if f.is_file() and not any(p.startswith(".") for p in f.parts):
            ext = f.suffix.lower()
            counts[ext] = counts.get(ext, 0) + 1
    return counts

def count_lines_of_code(path):
    result = subprocess.run(
        ["find", str(path), "-name", "*.py", "-exec", "wc", "-l", "{}", "+"],
        capture_output=True, text=True
    )
    lines = result.stdout.strip().splitlines()
    if lines:
        # Last line is the total
        total_line = lines[-1].strip()
        return int(total_line.split()[0])
    return 0
```



```

def count_test_coverage():
    result = subprocess.run(
        ["python", "-m", "pytest", "--co", "-q", "--tb=no"],
        cwd=REPO_PATH,
        capture_output=True, text=True
    )
    lines = result.stdout.strip().splitlines()
    test_count = sum(1 for l in lines if "::" in l)
    return test_count

def main():
    metrics = {
        "timestamp": datetime.now().isoformat(),
        "files": count_files_by_extension(REPO_PATH),
        "lines_of_code": count_lines_of_code(REPO_PATH / "src"),
        "test_count": count_test_coverage(),
    }

    METRICS_FILE.parent.mkdir(parents=True, exist_ok=True)
    with open(METRICS_FILE, "a") as f:
        f.write(json.dumps(metrics) + "\n")

    print(json.dumps(metrics, indent=2))

if __name__ == "__main__":
    main()

```

```

{
    "schedule": "0 22 * * *",
    "task": "Nightly code metrics: Run python3 /clawd/scripts/code-quality-metrics.py
for each repo in /clawd/config/repos.json. Track: lines of code, test count, file
count by type. Read last 7 days of metrics from /clawd/data/code-quality.jsonl and
check trends. If test count is declining or lines of code grew significantly without
tests growing, flag it in the morning briefing."
}

```

Automated Release Notes

Generate release notes for the changes in the current branch vs. main:

1. Run: `git log main..HEAD --oneline`
2. Group commits by type: feat, fix, docs, refactor, test, chore
3. For each commit, expand the one-liner into a proper release note sentence
4. Format as markdown with sections by type
5. Save to `/clawd/drafts/release-notes-YYYY-MM-DD.md`

Keep technical language but make it understandable to non-developers.

Expanded: The OpenClaw Community Patterns

These are patterns that have emerged from the OpenClaw user community. They're not in the official docs — they're things people discovered through use.

The "Dead Man's Switch" Pattern

A dead man's switch sends an alert if something *doesn't* happen, rather than if something *does*.

Use case: "If I don't check in within 48 hours, something might be wrong."

Implementation:

```
#!/usr/bin/env python3
"""
dead-mans-switch.py
Alerts if a check-in hasn't happened within the threshold.
"""

import json
from datetime import datetime, timedelta
from pathlib import Path

CHECKIN_FILE = Path("/Users/austenallred/clawd/data/last-checkin.json")
THRESHOLD_HOURS = 48
ALERT_SENT_FILE = Path("/Users/austenallred/clawd/data/dms-alert-sent.json")

def record_checkin():
    """Call this when you check in."""
    CHECKIN_FILE.write_text(json.dumps({
        "timestamp": datetime.now().isoformat(),
        "type": "manual"
    }))

    # Clear any pending alert
    if ALERT_SENT_FILE.exists():
        ALERT_SENT_FILE.unlink()

    print("Check-in recorded")

def check_status():
    """Returns True if check-in is overdue."""
    if not CHECKIN_FILE.exists():
        return True # Never checked in

    data = json.loads(CHECKIN_FILE.read_text())
    last_checkin = datetime.fromisoformat(data["timestamp"])
    overdue = datetime.now() - last_checkin > timedelta(hours=THRESHOLD_HOURS)

    return overdue

def alert_already_sent():
```

```

    return ALERT_SENT_FILE.exists()

def mark_alert_sent():
    ALERT_SENT_FILE.write_text(json.dumps({
        "sent_at": datetime.now().isoformat()
    }))

if __name__ == "__main__":
    import sys

    if len(sys.argv) > 1 and sys.argv[1] == "checkin":
        record_checkin()
    else:
        overdue = check_status()
        already_alerted = alert_already_sent()

        print(json.dumps({
            "overdue": overdue,
            "already_alerted": already_alerted,
            "should_alert": overdue and not already_alerted
        }))

        if overdue and not already_alerted:
            mark_alert_sent()

```

```

{
    "schedule": "0 */6 * * *",
    "task": "Dead man's switch check: Run python3 /clawd/scripts/dead-mans-switch.py.
If output shows should_alert: true, send a WhatsApp to my emergency contact
(+1XXXXXXXXXX): 'No check-in from automation system in 48+ hours. Please verify
everything is OK.'"
}

```

To check in, say to the agent: "Record my check-in" or trigger it via a WhatsApp message keyword.

The "Shadow Run" Pattern

Before deploying a new automation, run it in "shadow mode" — it does all the work but doesn't actually send messages or make external changes. You review the would-be outputs instead.

Implementation:

Add a `SHADOW_MODE` flag to your scripts:

```

import os

SHADOW_MODE = os.getenv("SHADOW_MODE", "false").lower() == "true"

def send_alert(message):
    if SHADOW_MODE:
        print(f"[SHADOW] Would send alert: {message}")

```

```
# Log to shadow output file
with open("/clawd/logs/shadow-output.log", "a") as f:
    f.write(f"{datetime.now().isoformat()} | ALERT | {message}\n")
else:
    # Actually send
    send_whatsapp(message)
```

Run in shadow mode:

```
SHADOW_MODE=true python3 /clawd/scripts/my-automation.py
```

Review shadow output:

```
cat /clawd/logs/shadow-output.log
```

When you're confident the outputs look correct, run without shadow mode.

The "Chaos Engineering" Pattern

Deliberately break things in controlled ways to verify your monitoring and recovery work.

Weekly chaos test:

```
{
  "schedule": "0 10 * * 2",
  "task": "Weekly chaos test (Tuesday mornings): Choose one of these tests at random
and execute it: 1) Delete /clawd/logs/ and verify the logging recreation works
correctly, 2) Write a fake error to /clawd/logs/alerts.log and verify the alert
system fires, 3) Set disk usage check threshold to 1% and verify alert fires, then
reset. Log which test was run and the result to /clawd/logs/chaos-tests.log."
}
```

This verifies your safety systems actually work, not just that they're configured.

The "Context Summary on Demand" Pattern

For long projects, maintain a running summary that you can ask for at any time:

```
When I say "summarize project {id}":
1. Read /clawd/projects/{id}/context.md
2. Read /clawd/projects/{id}/decisions.md
3. Read the latest entry from /clawd/memory/{today}.md relevant to this project
4. Generate a 5-bullet executive summary: current status, key decisions made, what's
working, what's blocked, what's next
5. Send via WhatsApp
```

Add to SOUL.md:

```
## Project Summary Command
When I say "summarize [project]" or "status [project]":
1. Find the project directory in /clawd/projects/ matching that name/ID
```

2. Read context.md and latest implementation/planning artifacts
3. Generate: status (one line), decisions made (bullets), next steps (bullets)
4. Send as WhatsApp message, < 200 words

The "Agent Observation" Pattern

Sometimes you want to watch what the agent is doing in real-time, not just see the final output.

Verbose mode:

Add to your task description:

```
Execute this task. After each major step, write a progress update to stdout.  
Format: "STEP: {step_name} | STATUS: {status} | {brief detail}"
```

Then tail the output:

```
# For a background process session  
openclaw process tail {session-id}
```

Or have the agent write to a live-updated file:

```
Execute the pipeline. After each step, append a one-line update to  
/clawd/logs/live-progress.log so I can watch progress.
```

Then in another terminal:

```
tail -f /Users/austenallred/clawd/logs/live-progress.log
```

This completes the handbook. If you've read this far, you have more than enough to build a sophisticated automation system. The only thing left is to build it.

Appendix H: Extended Troubleshooting Reference

This section is a comprehensive troubleshooting index. Every significant failure mode is here, with diagnosis steps and fixes.

Gateway Issues

Gateway starts but immediately stops

Symptoms: `openclaw gateway start` appears to work, then `openclaw gateway status` shows it's not running.

Check the error log:

```
# macOS  
cat ~/Library/Logs/openclaw/gateway.log | tail -50
```

```
# Linux
journalctl -u openclaw-gateway -n 50
```

Common causes:

1. **Port conflict:** Something else is running on port 3147.

```
lsof -i :3147
```

Fix: Stop the conflicting process or change Gateway port in config.

2. **Config file syntax error:** Invalid JSON in `openclaw.json`.

```
cat ~/.openclaw/openclaw.json | python3 -m json.tool
```

Fix: Correct the JSON syntax error.

3. **Missing workspace:** The workspace directory doesn't exist.

```
ls /Users/austenallred/clawd
```

Fix: Create the workspace or update the path in config.

4. **Permission error on workspace:** Can't read/write the workspace.

```
ls -la /Users/austenallred/ | grep clawd
```

Fix: `chmod 755 /Users/austenallred/clawd`

Gateway runs but doesn't process requests

Symptoms: Gateway status shows running, but messages sent to it get no response.

Check:

```
# Is the Gateway actually listening?
lsof -i :3147

# Can you connect to it?
curl http://localhost:3147/health
```

Common causes:

1. **Wrong port:** Gateway is running on a different port than configured.
2. **Firewall:** Local firewall blocking the port.
3. **Channel not configured:** The channel you're sending from isn't configured in `openclaw.json`.

Gateway crashes during a cron job

Symptoms: Cron jobs don't run, Gateway crashes periodically.

Check for memory issues:

```
# Watch memory while Gateway runs
top -pid $(pgrep -f openclaw-gateway)
```

Likely cause: A cron job triggers very large context loading or a process that uses too much memory.

Fix: Add memory limits to expensive processes. Break large tasks into smaller sub-tasks.

Tool Failures

`read` returns empty even though file exists

Check:

```
ls -la /path/to/file
wc -l /path/to/file
file /path/to/file
```

Causes:

- File is binary (not text) — the read tool returns content for text files
- File is 0 bytes (empty)
- File has unusual encoding (try `file` command to check)
- Path has trailing whitespace in the task description

Fix for encoding:

```
# Convert to UTF-8
iconv -f WINDOWS-1252 -t UTF-8 input.txt > output.txt
```

`exec` fails with "command not found" but works in terminal

The exec tool's `PATH` is restricted. Full paths or explicit PATH setting needed.

```
# Find where the command actually is
which python3
which node
which jq
```

Then use full paths in exec, or add to env:

```
{
  "tool": "exec",
  "command": "python3 script.py",
  "env": {
    "PATH": "/opt/homebrew/bin:/usr/local/bin:/usr/bin:/bin"
  }
}
```

`web_fetch` returns old content

Web pages may be cached. The tool respects cache headers, so stale content can appear.

Force fresh: There isn't a direct "bypass cache" option, but you can add a cache-busting parameter to some URLs:

```
https://example.com/page?_cb=1234567890
```

For time-sensitive data, consider using the browser tool instead — it always fetches fresh.

browser snapshot returns empty

Check:

1. Is the browser started? `browser action=status`
2. Is a tab open? `browser action=tabs`
3. Did navigation succeed? Check for any error in the navigate response.

Common fix: Navigate explicitly before snapshotting:

```
{
  "tool": "browser",
  "action": "navigate",
  "targetUrl": "https://example.com"
}
```

Then snapshot.

message fails to send

Check:

1. Is WhatsApp configured in `openclaw.json` ?
2. Is the recipient number/identifier correct?
3. Is the target format correct? (may need `+` prefix for international numbers)

```
{
  "channels": {
    "whatsapp": {
      "enabled": true
    }
  }
}
```

Test with a simple direct message:

```
Send a WhatsApp to my number (+1XXXXXXXXXX): "Test message"
```

nodes can't reach a node

Check node connection:


```
{
  "tool": "nodes",
  "action": "status"
}
```

Common causes:

- Node device is offline, sleeping, or lost connection
- Network changed (different WiFi)
- Node app was force-quit
- Authentication expired

Reconnection: Usually requires opening the OpenClaw app on the node device and re-pairing.

Cron Job Failures

Cron job runs but agent does nothing

The agent received the task but produced no observable output.

Add instrumentation:

```
{
  "schedule": "0 7 * * *",
  "task": "Morning task: [your actual task]. After completing (or failing), write
one line to /clawd/logs/cron-results.log: 'TIMESTAMP | morning-task | COMPLETE |
[brief result]' or 'TIMESTAMP | morning-task | FAILED | [error]'"
}
```

If the log shows nothing was written: The agent crashed or the task description caused an immediate error. Check Gateway logs.

If the log shows COMPLETE but no visible output: The agent completed successfully but wrote output somewhere other than where you expected. Check the task description's output path.

Cron job runs multiple times

DST transitions, system time changes, or Gateway restarts can cause duplicate runs.

Prevention: Make all cron operations idempotent (safe to run twice). Use state files with timestamps to detect and skip duplicate runs:

```
STATE_FILE = Path("/clawd/data/cron-state.json")

def should_run_today(job_name):
    """Returns True if this job hasn't run today."""
    today = datetime.now().strftime("%Y-%m-%d")

    if STATE_FILE.exists():
        state = json.loads(STATE_FILE.read_text())
        if state.get(job_name) == today:
            return False # Already ran today
```

```

    return True

def mark_run_today(job_name):
    today = datetime.now().strftime("%Y-%m-%d")

    state = {}
    if STATE_FILE.exists():
        state = json.loads(STATE_FILE.read_text())

    state[job_name] = today
    STATE_FILE.write_text(json.dumps(state))

```

Cron job works manually but not on schedule

Nine times out of ten: environment variable or PATH issue.

Verify by simulating the cron environment:

```

# macOS: simulate cron environment
env -i HOME=/Users/austenallred PATH=/usr/local/bin:/usr/bin:/bin
/Users/austenallred/clawd/scripts/your-script.sh

```

If it fails with the restricted environment but works normally, you've found your issue. Fix by using full paths in your scripts.

Cron job silently fails at a specific time

Common time-based causes:

1. **DST transition:** Clocks spring forward or fall back. A 2am cron job during "spring forward" might not run (that hour doesn't exist). A 1:30am job during "fall back" might run twice.
2. **Monthly anomaly:** A job scheduled for `0 9 31 * *` (9am on the 31st) won't run in April, June, September, or November because those months don't have 31 days.
3. **Leap year:** February 29 doesn't exist in most years.

Solution: For important jobs, use more robust schedules. Instead of "run on the 31st," use "run on Monday of each month" or similar.

Agent Behavior Issues

Agent keeps asking for clarification instead of acting

The task description is too ambiguous. The agent is making the right call — it doesn't know what to do.

Fix: Be more specific. Use the prompt anatomy from Chapter 15: explicit inputs, numbered steps, defined outputs.

If you want the agent to make a reasonable choice without asking, say so explicitly:

If you're unsure which file to use, pick the most recent one.
If a step fails, skip it and continue with the rest.
Do not ask for clarification – make your best judgment and document the choice.

Agent does the right thing but formats output wrong

Format is almost always something you can specify:

Output format requirements:

- WhatsApp: plain text, under 200 words, no markdown formatting
- Report file: GitHub-flavored markdown, H2 headers, bullet lists
- Data file: JSON, one object per line (JSON Lines format), no pretty-printing

Agent does extra work you didn't ask for

Common when the task description implies a broader scope than intended.

Fix: Add explicit scope boundaries:

Your task is **ONLY** to [specific thing].
Do **NOT** [list of things that seem related but aren't the task].
When you have completed [specific output], stop. Do not continue.

Agent forgets previous instructions in long sessions

Context drift — older instructions become less prominent as the session grows.

Fix: Repeat important constraints. For the most critical constraints, add them to SOUL.md so they're always at the start of context:

```
## Hard Constraints (always active)  
– Never delete files without explicit confirmation  
– Never send messages to customers without review  
– Always write to /clawd/ paths, never to external locations
```

Agent confidently gives wrong information

Hallucination about facts it can't know (current prices, current code, current state of a file).

Fix: Don't ask the agent to know things — ask it to look things up:

BAD: "What's the current state of the project?"
GOOD: "Read /clawd/projects/20260222/context.md and summarize the current state."

BAD: "Is the API available?"
GOOD: "Run: curl -s -o /dev/null -w '%{http_code}' https://api.example.com/health and check the status code."

Agents are bad at knowing things. They're good at looking things up. Use the right capability.

File System Issues

Files not where you expect them

The most common debugging exercise in automation. Systematic approach:

Step 1: Check the exact path:

```
ls -la /Users/austenallred/clawd/the/path/you/expected/
```

Step 2: Find the file by name:

```
find /Users/austenallred/clawd -name "the-file-you-want*" 2>/dev/null
```

Step 3: Check what was recently created:

```
find /Users/austenallred/clawd -newer /tmp/timestamp-file -type f 2>/dev/null  
# Create timestamp: touch /tmp/timestamp-file
```

Step 4: Search by content:

```
grep -r "unique string from the file" /Users/austenallred/clawd/ 2>/dev/null
```

File encoding issues

Text files from different systems have different encodings and line endings. This causes tools to fail silently or produce garbled output.

Check encoding:

```
file -I /path/to/file
```

Check line endings:

```
cat -A /path/to/file | head -5  
# Windows line endings show as ^M$  
# Unix line endings show as just $
```

Convert:

```
# Remove Windows line endings  
sed -i '' 's/\r//' /path/to/file  
  
# Convert encoding  
iconv -f UTF-16 -t UTF-8 input.txt > output.txt
```

Disk fills up during pipeline

Large pipelines can produce surprisingly large output, especially when:

- Saving all intermediate steps (add compression or cleanup)

- Caching web content (add TTL-based cache expiry)
- Writing uncompressed logs (add logrotate)
- Not cleaning up temp files

Set up logrotate:

```
# /usr/local/etc/logrotate.d/openclaw (macOS with homebrew logrotate)
/Users/austenallred/clawd/logs/*.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
}
```

Add cleanup to pipeline scripts:

```
import atexit
import tempfile

# Create temp directory that auto-cleans
temp_dir = tempfile.mkdtemp()
atexit.register(lambda: shutil.rmtree(temp_dir, ignore_errors=True))
```

Performance Issues

Everything is slow

Diagnose first. Don't optimize blindly.

```
# Time a specific script
time python3 /clawd/scripts/slow-script.py

# Profile a Python script
python3 -m cProfile -s cumulative /clawd/scripts/slow-script.py 2>&1 | head -30
```

Common slow things and their fixes:

1. Synchronous HTTP requests in a loop:

```
# SLOW: Sequential requests
for url in urls:
    response = requests.get(url)
    process(response)

# FAST: Parallel requests
import concurrent.futures

def fetch_and_process(url):
```

```

response = requests.get(url, timeout=10)
return process(response)

with concurrent.futures.ThreadPoolExecutor(max_workers=5) as executor:
    results = list(executor.map(fetch_and_process, urls))

```

2. Reading a large file multiple times:

```

# SLOW: Read file on each iteration
for item in items:
    config = json.loads(Path("/clawd/config/config.json").read_text())
    process(item, config)

# FAST: Read once
config = json.loads(Path("/clawd/config/config.json").read_text())
for item in items:
    process(item, config)

```

3. Subprocess overhead in a loop:

```

# SLOW: Spawn subprocess for each item
for item in items:
    result = subprocess.run(["jq", ".name", f], capture_output=True)

# FAST: Use Python's json module instead
import json
for item in items:
    data = json.loads(Path(f).read_text())
    name = data["name"]

```

4. Large context loading every session:

Review your context files. Every character you remove from SOUL.md, MEMORY.md, and AGENTS.md makes every session start faster.

WhatsApp/Communication Issues

WhatsApp messages not arriving

1. Check `openclaw.json` has WhatsApp configured and enabled.
2. Verify the target identifier is correct.
3. Check if there's a rate limit (too many messages in a short period).
4. Look for error messages in Gateway logs.

Test with minimal message:

```
Send a WhatsApp message to my number: "Test"
```

If this fails, the channel configuration is broken. If this works but your automation's messages don't, the issue is in how the automation is calling the message tool.

Messages being sent to wrong recipient

The target field is incorrect. For WhatsApp, it's typically a phone number with country code.

Debug: Ask the agent:

```
What value are you using as the target for WhatsApp messages?
```

Check SOUL.md and AGENTS.md for any hardcoded targets.

TTS not playing

TTS requires an audio output device. If OpenClaw is running on a headless server, TTS output has nowhere to go.

Check: Is the Gateway running locally (with speakers) or remotely?

If remotely: consider delivering audio via a node device that has speakers, or switch to text-only alerts for remote deployments.

Sub-agent Issues

Sub-agent completes but main agent doesn't get the result

Sub-agents communicate results through files, not through direct message passing. The sub-agent writes output to agreed-upon file paths; the main agent reads those files.

Fix: Be explicit in sub-agent tasks about where to write output:

```
When your task is complete:  
1. Write your summary to /clawd/projects/{id}/research-artifacts/research-summary.md  
2. Include in the last line: "STATUS: COMPLETE" or "STATUS: FAILED"
```

Then the main agent reads that file to get results.

Sub-agents conflicting with each other

Two sub-agents writing to the same file at the same time will corrupt it.

Fix: Give each sub-agent its own output directory:

```
Sub-agent 1 output: /clawd/projects/{id}/research/agent-1/  
Sub-agent 2 output: /clawd/projects/{id}/research/agent-2/  
Sub-agent 3 output: /clawd/projects/{id}/research/agent-3/
```

Main agent merges results after all sub-agents complete:

```
When all sub-agents report complete, read all three output directories and merge.
```

Sub-agent context too large

Sub-agents load the same context files as the main agent (SOUL.md, MEMORY.md, etc.) plus their task description. If these are large, the sub-agent may have little room for actual work.

Fix: For sub-agents, minimize context loading. Add to the sub-agent task:

For this task, you do NOT need to load AGENTS.md or MEMORY.md.
You only need: the task description below and direct tool access.

Security and Privacy Issues

Agent read a file it shouldn't have

Review your tool policy in `openclaw.json`. The `read` tool by default can access any file the Gateway process has permission to read.

Restrict read access:

```
{
  "tools": {
    "read": {
      "allowedPaths": ["/Users/austenallred/clawd/"]
    }
  }
}
```

This limits the read tool to your workspace directory only.

Credentials appearing in logs

Never should happen but sometimes does through agent logging of tool parameters.

Immediate fix: Find and redact the logs:

```
# Find log files with credentials
grep -rl "sk-\\|password\\|secret\\|token" /Users/austenallred/clawd/logs/
```

Prevention:

1. Store credentials in `/clawd/secrets/` only
2. In task descriptions, say "read credentials from `/clawd/secrets/api-keys.json`" rather than including the actual credentials
3. Add `/clawd/secrets/` to `.gitignore` and never commit it

Automation sent a message to the wrong person

If you've accidentally sent a message to an unintended recipient:

1. If it's a WhatsApp message: You can delete it on your end, but they may have already seen it.
2. If it contained sensitive information: Treat it as a data breach and act accordingly.

Prevention: Always include a "dry run" step in automations that send messages to people other than yourself. Only automate the actual send after verifying dry run outputs are correct.

Appendix I: Glossary

Agent: An AI model instance with tool access, running within a session. Can spawn sub-agents.

Artifact: A file produced by an agent as output of a task — research summaries, planning documents, code, reports.

Context window: The amount of text an AI model can consider at once, measured in tokens.

Cron: A Unix time-based job scheduler. OpenClaw uses cron expressions to schedule tasks.

Gateway: The OpenClaw daemon process that manages agents, channels, cron jobs, and node connections.

Gate: A quality checkpoint between pipeline phases. Each gate has criteria for PASS/FAIL.

Handoff: Structured transfer of work from one agent to another, including all relevant context and artifacts.

Heartbeat: A periodic automated check that verifies system health and takes any needed actions.

Idempotent: An operation that produces the same result whether run once or multiple times.

Node: A paired device (phone, laptop, server) accessible to the OpenClaw Gateway for commands, camera, screen, and notifications.

Pipeline: A series of automated steps that transforms input data into output artifacts.

RALPH: Retry And Learn Protocol. The escalation system triggered when sub-agents fail repeatedly.

Session: An isolated agent context with its own message history and state.

Skill: A packaged behavior, available through ClawHub, that extends OpenClaw's capabilities.

Sub-agent: An agent spawned by another agent to handle a specific delegated task.

TEA audit: Test, Evaluate, Assess — the quality audit phase in the software factory pipeline.

Token: The basic unit of AI model processing. Roughly 4 characters or 0.75 words.

Tool: A specific capability an agent can use to interact with the world (read, write, exec, browser, etc.).

Workspace: The directory on your local machine that OpenClaw uses as the root for all file operations.

Appendix J: Version Notes

This handbook was written for OpenClaw as it existed in early 2026. The platform evolves. Some specifics may have changed.

Things most likely to be current:

- Core concepts (sessions, tools, Gateway, workspace)
- Mental models and patterns
- Python code examples
- Unix/shell patterns
- Cron expressions

Things to verify against current docs:

- Exact tool parameter names
- Model names and configuration keys
- Channel configuration format
- Node pairing procedures

- ClawHub skill availability

When in doubt, check the official docs. The patterns described here are stable; the exact API surface evolves.

The source of truth is always the current documentation at openclaw.dev.

End of Appendices

Appendix K: Complete Worked Examples

This appendix contains four complete, end-to-end worked examples. Each one shows every file, every script, and every configuration needed to run a specific automation from scratch.

Worked Example 1: The Weekly Intelligence Briefing

What it does: Every Monday at 6am, researches your chosen topics, compiles a weekly intelligence briefing, and sends it to your WhatsApp.

Time to set up: 30 minutes **Running cost:** ~5 minutes of AI time per week

Files to create

`/Users/austenallred/clawd/config/intelligence-topics.json :`

```
{
  "topics": [
    {
      "id": "ai-tools",
      "name": "AI Development Tools",
      "description": "New AI coding assistants, model releases, benchmarks",
      "queries": [
        "AI coding assistant release 2026",
        "new LLM model benchmark 2026",
        "AI developer tools news this week"
      ],
      "sources": []
    },
    {
      "id": "saas-market",
      "name": "SaaS Market Trends",
      "description": "Funding rounds, acquisitions, pricing changes",
      "queries": [
        "SaaS startup funding this week",
        "SaaS acquisition announcement 2026"
      ],
      "sources": [
        "https://techcrunch.com/category/enterprise"
      ]
    }
  ]
}
```

```
]
}
```

/Users/austenallred/clawd/scripts/compile-intelligence.py :

```
#!/usr/bin/env python3
"""
compile-intelligence.py
Reads research results from /clawd/research/intel/ and compiles a briefing.
Designed to be called after the agent has done the research and saved results.
"""

import json
import sys
from pathlib import Path
from datetime import datetime

INTEL_DIR = Path("/Users/austenallred/clawd/research/intel")
BRIEFING_DIR = Path("/Users/austenallred/clawd/reports/intelligence")

def compile_briefing():
    week = datetime.now().strftime("%Y-W%V")
    results_dir = INTEL_DIR / week
    briefing_file = BRIEFING_DIR / f"{week}.md"

    if not results_dir.exists():
        print(json.dumps({"error": f"No results for {week}", "path":
str(results_dir)}))
        sys.exit(1)

    BRIEFING_DIR.mkdir(parents=True, exist_ok=True)

    sections = [
        f"# Weekly Intelligence Briefing - {week}",
        f"Generated: {datetime.now().strftime('%Y-%m-%d %H:%M')}",
        ""
    ]

    topic_count = 0
    finding_count = 0

    for result_file in sorted(results_dir.glob("*.json")):
        data = json.loads(result_file.read_text())
        topic_name = data.get("topic_name", result_file.stem)
        findings = data.get("findings", [])

        if not findings:
            continue

        sections.append(f"## {topic_name}")
```

```

for finding in findings[:5]: # Max 5 per topic
    sections.append(f"- **{finding.get('title', 'Untitled')}**")
    if finding.get("summary"):
        sections.append(f" {finding['summary']}")
    if finding.get("url"):
        sections.append(f" Source: {finding['url']}")
    finding_count += 1

sections.append("")
topic_count += 1

briefing_content = "\n".join(sections)
briefing_file.write_text(briefing_content)

print(json.dumps({
    "ok": True,
    "topics": topic_count,
    "findings": finding_count,
    "file": str(briefing_file),
    "week": week
}))

if __name__ == "__main__":
    compile_briefing()

```

Cron job in `openclaw.json` :

```

{
  "schedule": "0 12 * * 1",
  "task": "Weekly intelligence briefing (runs Monday at 6am CST = 12pm UTC):\n\n1.
Read topics from /clawd/config/intelligence-topics.json\n\n2. For each topic, run
the defined search queries using web_search. Read the top 2 URLs from each search
result using web_fetch. Extract key findings: headline, 1-sentence summary, source
URL.\n\n3. Save results for each topic as JSON to /clawd/research/intel/YYYY-
WNN/{topic-id}.json with format:\n  {\"topic_name\": \"...\", \"week\": \"...\",
\"findings\": [{\"title\": \"...\", \"summary\": \"...\", \"url\": \"...\"}]}\n\n4.
Run: python3 /clawd/scripts/compile-intelligence.py\n\n5. Read the compiled briefing
at the path shown in the script output.\n\n6. Create a condensed WhatsApp summary:
topic name + top finding for each topic. Keep under 300 words. Send to me.\n\n7.
Log: write 'COMPLETE: YYYY-WNN briefing, N topics, N findings' to /clawd/logs/cron-
history.log"
}

```

Testing:

```

# Create test intel directory
mkdir -p /Users/austenallred/clawd/research/intel/2026-W08

# Create a test result file
cat > /Users/austenallred/clawd/research/intel/2026-W08/ai-tools.json << 'EOF'
{

```

```

"topic_name": "AI Development Tools",
"week": "2026-W08",
"findings": [
  {
    "title": "Anthropic releases Claude 4",
    "summary": "Significant capability improvement with 200K context window and improved code generation.",
    "url": "https://anthropic.com/news/claude-4"
  }
]
}
EOF

```

```

# Test compilation
python3 /Users/austenallred/clawd/scripts/compile-intelligence.py
cat /Users/austenallred/clawd/reports/intelligence/2026-W08.md

```

Worked Example 2: The Expense Auto-Logger

What it does: When you send a photo of a receipt to the agent, it extracts the expense data and logs it automatically, updating your monthly expense tracker.

Time to set up: 20 minutes

Files to create

`/Users/austenallred/clawd/data/expenses.json` (initial):

```

{
  "version": "1.0",
  "expenses": [],
  "categories": [
    "groceries",
    "dining",
    "transportation",
    "utilities",
    "software",
    "entertainment",
    "health",
    "office",
    "travel",
    "other"
  ]
}

```

`/Users/austenallred/clawd/scripts/log-expense.py` :

```

#!/usr/bin/env python3
"""
log-expense.py
Adds an expense entry to the expense log.

```

```

Reads expense data from stdin as JSON.
"""

import json
import sys
from datetime import datetime
from pathlib import Path

EXPENSES_FILE = Path("/Users/austenallred/clawd/data/expenses.json")

def load_expenses():
    if EXPENSES_FILE.exists():
        return json.loads(EXPENSES_FILE.read_text())
    return {"version": "1.0", "expenses": [], "categories": []}

def save_expenses(data):
    EXPENSES_FILE.write_text(json.dumps(data, indent=2))

def add_expense(merchant, amount, date, category, notes=""):
    data = load_expenses()

    expense = {
        "id": f"exp-{datetime.now().strftime('%Y%m%d%H%M%S')}",
        "date": date,
        "merchant": merchant,
        "amount": round(float(amount), 2),
        "category": category,
        "notes": notes,
        "logged_at": datetime.now().isoformat()
    }

    data["expenses"].append(expense)
    save_expenses(data)

    return expense

def monthly_total(year, month):
    data = load_expenses()
    month_str = f"{year}-{month:02d}"

    total = sum(
        e["amount"] for e in data["expenses"]
        if e["date"].startswith(month_str)
    )

    by_category = {}
    for e in data["expenses"]:
        if e["date"].startswith(month_str):
            cat = e["category"]
            by_category[cat] = by_category.get(cat, 0) + e["amount"]

    return {

```

```

        "month": month_str,
        "total": round(total, 2),
        "by_category": {k: round(v, 2) for k, v in sorted(by_category.items(),
key=lambda x: -x[1])}
    }

if __name__ == "__main__":
    command = sys.argv[1] if len(sys.argv) > 1 else "add"

    if command == "add":
        data = json.loads(sys.stdin.read())
        expense = add_expense(
            merchant=data["merchant"],
            amount=data["amount"],
            date=data.get("date", datetime.now().strftime("%Y-%m-%d")),
            category=data.get("category", "other"),
            notes=data.get("notes", "")
        )
        print(json.dumps({"ok": True, "expense": expense}))

    elif command == "monthly":
        year = int(sys.argv[2]) if len(sys.argv) > 2 else datetime.now().year
        month = int(sys.argv[3]) if len(sys.argv) > 3 else datetime.now().month
        print(json.dumps(monthly_total(year, month)))

```

Keyword routing in SOUL.md :

Expense Logging

When I send a photo and say "log expense" or when I send a message like "log: \$47.30 at Whole Foods" or similar:

1. If it's a photo: analyze the image to extract merchant name, total amount, and date.
If any field is unclear, use "Unknown" for merchant and today's date.
2. Determine category from these options:
groceries, dining, transportation, utilities, software, entertainment, health, office, travel, other
3. Create JSON: {"merchant": "...", "amount": 0.00, "date": "YYYY-MM-DD", "category": "..."}
4. Pipe JSON to: `python3 /clawd/scripts/log-expense.py add`
5. Confirm via WhatsApp: "Logged: \$AMOUNT at MERCHANT (CATEGORY)"
6. If I say "expense summary" or "spending report":
Run: `python3 /clawd/scripts/log-expense.py monthly`
Format the output as a readable summary and send via WhatsApp.

Monthly summary cron:

```
{
  "schedule": "0 9 1 * *",
  "task": "Monthly expense summary: Run: python3 /clawd/scripts/log-expense.py
monthly. Format the spending by category as a WhatsApp message. Compare total to
/clawd/data/budget.json (if it exists). Flag any categories over budget. Send to
me."
}
```

Worked Example 3: The Code Review Assistant

What it does: Runs automated code review on changed files every weekday afternoon, checks against your standards, and sends you a summary.

Time to set up: 45 minutes

Files to create

`/Users/austenallred/clawd/protocols/CODE-REVIEW-STANDARDS.md :`

```
# Code Review Standards

## Critical (must fix before merge)
- Hardcoded credentials or API keys in source code
- SQL queries built by string concatenation (SQL injection risk)
- Unhandled exceptions on all external calls (network, database, file I/O)
- eval() or exec() with user-supplied input
- Passwords or secrets in log output

## High (fix soon, ideally before merge)
- Functions over 50 lines (break them up)
- Duplicate code blocks (3+ lines, 3+ occurrences)
- No type hints on function signatures (Python only)
- Missing docstrings on public functions
- print() debug statements in production code

## Medium (improve when you can)
- Variable names under 3 characters (except conventional i, j, k, n)
- Nested conditionals more than 3 levels deep
- TODO comments older than 30 days
- Missing test for new function

## Low (nice to have)
- Long lines over 120 characters
- Inconsistent naming conventions
```

`/Users/austenallred/clawd/scripts/code-review.py :`


```

#!/usr/bin/env python3
"""
code-review.py
Automated code quality checker.
Checks Python and JavaScript files against defined standards.
"""

import re
import sys
import json
import subprocess
from pathlib import Path
from datetime import datetime

STANDARDS_FILE = Path("/Users/austenallred/clawd/protocols/CODE-REVIEW-
STANDARDS.md")
REVIEW_DIR = Path("/Users/austenallred/clawd/reviews")

PATTERNS = {
    "python": {
        "critical": [
            (r'(api_key|password|secret|token)\s*=\s*["\'](?:^"\'+["\']|',
            "Possible hardcoded credential"),
            (r'eval\s*\(', "eval() usage – review for safety"),
        ],
        "high": [
            (r'\bprint\s*\(', "print() statement – debug artifact?"),
        ]
    },
    "javascript": {
        "critical": [
            (r'(apiKey|password|secret)\s*[:=]\s*["\'](?:^"\'+["\']|',
            "Possible hardcoded credential"),
            (r'\beval\s*\(', "eval() usage – review for safety"),
        ],
        "high": [
            (r'\bconsole\.(log|debug|warn)\s*\(', "console.log – debug artifact?"),
        ]
    }
}

def check_file(file_path):
    path = Path(file_path)
    if not path.exists():
        return None

    content = path.read_text(errors="replace")
    lines = content.splitlines()
    issues = []

    # Determine language

```

```

ext = path.suffix.lower()
lang = "python" if ext == ".py" else "javascript" if ext in (".js", ".ts") else
None

if lang and lang in PATTERNS:
    for severity, pattern_list in PATTERNS[lang].items():
        for pattern, description in pattern_list:
            for lineno, line in enumerate(lines, 1):
                if re.search(pattern, line):
                    issues.append({
                        "severity": severity.upper(),
                        "line": lineno,
                        "description": description,
                        "code": line.strip()[:80]
                    })

# Check function length (Python)
if lang == "python":
    in_function = False
    function_start = 0
    function_name = ""
    indent_level = 0

    for lineno, line in enumerate(lines, 1):
        if re.match(r'\s*def ', line):
            if in_function and lineno - function_start > 50:
                issues.append({
                    "severity": "HIGH",
                    "line": function_start,
                    "description": f"Function '{function_name}' is {lineno -
function_start} lines (max 50)"
                })
            in_function = True
            function_start = lineno
            name_match = re.search(r'def (\w+)', line)
            function_name = name_match.group(1) if name_match else "unknown"

    return issues

def get_changed_files(repo_path, since_hours=8):
    try:
        result = subprocess.run(
            ["git", "diff", "--name-only", f"HEAD@{'{'{since_hours} hours ago'}"}"],
            cwd=repo_path,
            capture_output=True,
            text=True,
            timeout=10
        )
        files = result.stdout.strip().splitlines()
        return [Path(repo_path) / f for f in files if f.endswith((".py", ".js",
".ts"))]
    except subprocess.SubprocessError:

```

```

        return []

def main():
    repo_path = sys.argv[1] if len(sys.argv) > 1 else "."

    changed_files = get_changed_files(repo_path)

    if not changed_files:
        print(json.dumps({"ok": True, "message": "No changed files to review"}))
        return

    all_issues = {"critical": [], "high": [], "medium": [], "low": []}

    for file_path in changed_files:
        issues = check_file(file_path)
        if issues:
            for issue in issues:
                severity = issue["severity"].lower()
                if severity in all_issues:
                    issue["file"] = str(file_path)
                    all_issues[severity].append(issue)

    # Write review file
    today = datetime.now().strftime("%Y-%m-%d")
    review_file = REVIEW_DIR / f"auto-review-{today}.md"
    REVIEW_DIR.mkdir(parents=True, exist_ok=True)

    lines = [f"# Automated Code Review - {today}", ""]

    total_issues = sum(len(v) for v in all_issues.values())

    if total_issues == 0:
        lines.append("✅ No issues found in changed files.")
    else:
        lines.append(f"Found {total_issues} issues in {len(changed_files)} files.\n")

        for severity in ["critical", "high", "medium", "low"]:
            if all_issues[severity]:
                lines.append(f"## {severity.upper()} ({len(all_issues[severity])})\n")
                for issue in all_issues[severity]:
                    lines.append(f"- **{issue['file']}:{issue.get('line', '?')}** - {issue['description']}")
                    if issue.get("code"):
                        lines.append(f"  `{issue['code']}`")
                    lines.append("")

        review_file.write_text("\n".join(lines))

    print(json.dumps({
        "ok": True,

```

```

        "files_reviewed": len(changed_files),
        "total_issues": total_issues,
        "critical": len(all_issues["critical"]),
        "high": len(all_issues["high"]),
        "review_file": str(review_file)
    })

if __name__ == "__main__":
    main()

```

Cron task:

```

{
  "schedule": "0 22 * * 1-5",
  "task": "Afternoon code review (5pm CST = 10pm UTC): Run: python3
/clawd/scripts/code-review.py /Users/austenallred/projects/myapp. Parse the JSON
output. If critical > 0: send me an immediate WhatsApp alert with the critical
issues. If total_issues > 0: send a WhatsApp summary with counts by severity and the
review file path. If total_issues == 0: just log 'review complete, no issues' to
/clawd/logs/cron-history.log (no WhatsApp needed).\"
}

```

Worked Example 4: The Research Pipeline

What it does: Takes a research topic, runs a complete research workflow, and produces a structured research report with sources, key findings, and a synthesis.

When to use it: Any time you need to quickly get up to speed on a topic.

Time to set up: 15 minutes

How to run it

Just ask the agent:

Run a full research pipeline on: [your topic]

Save the research report to /clawd/research/reports/YYYY-MM-DD-{slug}.md

The agent will execute this internally defined pipeline (add to SOUL.md):

Research Pipeline

When asked to "run a full research pipeline on [topic]" or "research [topic] thoroughly":

****Phase 1: Query generation (2 minutes)****

Generate 5–7 diverse search queries covering:

- Broad overview of the topic
- Recent developments (add "2026" to queries)
- Specific aspects or controversies

- Comparison with alternatives (if applicable)
- Expert perspectives or case studies

****Phase 2: Initial research (5-10 minutes)****

For each query:

- Run `web_search`
- *From results, identify the 2 most credible/useful sources*
- *Fetch each source with `web_fetch`*

Keep track of: source URL, key points, publication date, credibility signals

****Phase 3: Synthesis (3 minutes)****

From all gathered information:

- Identify consensus points (most sources agree)
- Identify disputed points (sources disagree)
- Identify knowledge gaps (things no source covers well)
- Identify the most important 3-5 findings

****Phase 4: Report generation****

Write report to the specified file with sections:

1. Executive Summary (3-5 bullets, key takeaways)
2. Background (what this is and why it matters)
3. Key Findings (numbered, with source citations)
4. Disputed Areas (where sources conflict, present both sides)
5. Knowledge Gaps (what you couldn't find out)
6. Sources (all URLs used, with brief description)
7. Research Metadata (queries run, sources consulted, time)

****Quality checks:****

- All claims should have at least one source
- Sources should be from the last 2 years for recent topics
- Disputed areas must present both sides fairly
- Executive summary must accurately reflect the full report

Template for the output report:

Research Report: [Topic]

****Date:**** YYYY-MM-DD

****Queries run:**** N

****Sources consulted:**** N

****Confidence level:**** High/Medium/Low

Executive Summary

- Finding 1
- Finding 2
- Finding 3

Background

```

[What this is, why it matters, relevant context]

## Key Findings

### 1. [Finding title]
[2-3 paragraphs supporting this finding]
Sources: [URL1], [URL2]

### 2. [Finding title]
...

## Disputed Areas

### [Point of contention]
**Position A:** [What some sources say]
**Position B:** [What other sources say]
**Assessment:** [Which seems better supported and why]

## Knowledge Gaps

- [Thing no sources covered well]
- [Question that remains unanswered]

## All Sources

| # | URL | Description | Date |
|---|-----|-----|-----|
| 1 | https://... | Description | 2026-01 |

## Research Metadata

- Queries: [list of queries run]
- Time: approximately N minutes
- Model: [which model was used]

```

This pattern is repeatable across any topic. Once you've got SOUL.md set up with the research pipeline, every research request uses the same rigorous process.

These four worked examples represent the most common patterns you'll use. Study them, adapt them to your situation, and use them as templates for your own automations. The structure is always the same: clear inputs, numbered steps, explicit outputs, defined error handling. Get that structure right and the implementation details take care of themselves.

Appendix L: Rapid Reference — The 50 Most Useful Commands

If you forget everything else in this book, keep this appendix. These are the commands and patterns you'll use most often.

Gateway Management

```
# Status
openclaw gateway status

# Start
openclaw gateway start

# Stop
openclaw gateway stop

# Restart (required after openclaw.json changes)
openclaw gateway restart

# Check logs
tail -f ~/.openclaw/logs/gateway.log
```

File Operations (as agent instructions)

```
# Read a file
Read /clawd/MEMORY.md

# Read large file in chunks
Read lines 100-200 of /clawd/logs/big-log.log

# Write a new file
Write to /clawd/data/output.json: {json content}

# Edit a file precisely
Edit /clawd/config.json: find "enabled": false, replace with "enabled": true

# Find files
Run: find /clawd -name "*.md" -newer /tmp/checkpoint
```

Shell Commands (as agent instructions)

```
# List directory
Run: ls -la /clawd/data/

# Check disk space
Run: df -h

# Find recent files
Run: find /clawd -mtime -1 -type f

# Check a process
Run: ps aux | grep python
```

```
# Count lines
Run: wc -l /clawd/logs/app.log

# Search file contents
Run: grep -r "error" /clawd/logs/ --include="*.log"
```

Python Quick Patterns

```
# Read JSON file
import json
from pathlib import Path
data = json.loads(Path("/clawd/data/file.json").read_text())

# Write JSON file
Path("/clawd/data/output.json").write_text(json.dumps(data, indent=2))

# Append to log
with open("/clawd/logs/app.log", "a") as f:
    f.write(f"{datetime.now().isoformat()} | {message}\n")

# Read CSV with pandas
import pandas as pd
df = pd.read_csv("/clawd/data/input.csv")

# Get today's date string
from datetime import datetime
today = datetime.now().strftime("%Y-%m-%d")

# File exists check
if Path("/clawd/data/file.json").exists():
    ...

# Create directory and parents
Path("/clawd/data/new-dir").mkdir(parents=True, exist_ok=True)
```

Cron Job Patterns

```
// Every day at 7am UTC (1am CST, 2am CDT)
{"schedule": "0 7 * * *", "task": "..."}

// Weekdays at 7am CST (13:00 UTC)
{"schedule": "0 13 * * 1-5", "task": "..."}

// Every 2 hours
{"schedule": "0 */2 * * *", "task": "..."}

// Every 15 minutes
```



```
{"schedule": "*/15 * * * *", "task": "..."}

// Monday at 9am CST (15:00 UTC)
{"schedule": "0 15 * * 1", "task": "..."}

// First of month at midnight UTC
{"schedule": "0 0 1 * *", "task": "..."}

// Weekday evenings at 5pm CST (23:00 UTC)
{"schedule": "0 23 * * 1-5", "task": "..."}

```

Web Operations (as agent instructions)

```
# Search the web
Search: competitor pricing comparison 2026

# Fetch a page (static content)
Fetch: https://example.com/pricing (as markdown)

# Browser screenshot
Take a screenshot of https://status.example.com

# Browser navigate and extract
Navigate to https://app.example.com/dashboard. Wait for content to load.
Extract all table data using:
evaluate: Array.from(document.querySelectorAll('tr')).map(r => r.textContent)

```

Message and Notification Commands

```
# WhatsApp to yourself
Send me a WhatsApp: "Task complete"

# WhatsApp with formatting
Send me a WhatsApp:
*Status Update*
✅ Pipeline: Complete
📊 Items: 47
⚠️ Errors: 2

# Text to speech
Read this text aloud: "Your morning briefing is ready"

# Node notification
Send system notification to my-iphone: "Pipeline complete"

```

Agent Management Commands

```
# Check running agents
List all running subagents

# Kill a stuck agent
Kill subagent: research-lead

# Send instruction to running agent
Steer subagent research-lead: "Also check Product Hunt for recent releases"

# Check background process
Poll background session {id} for output
```

Debugging Commands

```
# Check what ran recently
tail -50 /clawd/logs/cron-history.log

# Find errors
grep -i "error\|fail" /clawd/logs/*.log | tail -30

# Check file was created
ls -lt /clawd/reports/ | head -10

# Disk usage breakdown
du -sh /clawd/*/

# File encoding
file -I /path/to/file
```

File Permissions Quick Reference

```
# Script (executable)
chmod 755 /clawd/scripts/my-script.sh

# Data file (read/write for owner)
chmod 644 /clawd/data/output.json

# Secrets (owner only)
chmod 600 /clawd/secrets/api-keys.json
chmod 700 /clawd/secrets/

# Check permissions
ls -la /clawd/secrets/
```

SOUL.md Snippets

Routing Keywords

When I say "research X": trigger research pipeline for X

When I say "status": report current system status

When I say "log expense": trigger expense logging

When I say "brief me": generate and send morning briefing now

Output Preferences

WhatsApp: plain text, under 200 words, no markdown

Reports: GitHub Markdown, H2 headers

Data files: JSON, pretty-printed

Hard Rules

Never delete without confirmation.

Never send to third parties without review.

Always log to memory/YYYY-MM-DD.md.

MEMORY.md Entry Templates

Routing Decisions

– 2026-02-22: Research tasks → research-lead (not main agent). Reason: cleaner separation.

Operator Preferences

- Briefings: max 5 items or they don't get read
- Reports: prefer bullets over paragraphs
- Alerts: no alerts after 10pm unless critical

System Config

- API version: v3 (v2 deprecated Feb 2026)
- Primary codebase: /projects/myapp/
- Production server: via home-server node

Known Issues

- competitor.com: use browser, not `web_fetch` (*returns incomplete HTML*)
- Sales CSV: date column sometimes *MM/DD/YYYY*, normalize to *YYYY-MM-DD*

Error Handling Template

```
import logging
import sys

log = logging.getLogger(__name__)

def safe_operation(fn, description, fallback=None):
    """Run fn, log errors, return fallback on failure."""
    try:
        return fn()
```

```
except FileNotFoundError as e:
    log.error(f"{description}: file not found: {e}")
    return fallback
except PermissionError as e:
    log.error(f"{description}: permission denied: {e}")
    return fallback
except Exception as e:
    log.error(f"{description}: unexpected error: {type(e).__name__}: {e}")
    return fallback

# Usage:
result = safe_operation(
    lambda: json.loads(Path("/clawd/data/input.json").read_text()),
    description="loading input data",
    fallback={}
)
```

That's the complete rapid reference. Print it, keep it handy, and refer to it when you're building something and can't remember the exact syntax.

Afterword: What to Build Next

You've read the book. You understand the tools, the patterns, the architecture. Now what?

Here's a suggested build order if you're starting from scratch:

Week 1: Foundation

Set up the workspace structure properly. Write SOUL.md, MEMORY.md, USER.md. Get the morning briefing running. Verify cron works. Get at least one WhatsApp notification firing successfully. Keep it simple — you're building trust in the infrastructure.

Week 2: First automation

Pick the single most annoying repetitive task in your work life. Automate it. Don't pick something complex — pick something you do every day and hate. Get it working completely, including error handling and logging.

Week 3: Second and third automations

By now you know the infrastructure works. Add two more automations. These can be slightly more complex. Start building the habits around reading daily memory logs and verifying things ran.

Month 2: Multi-step pipelines

Build your first pipeline that has more than one step. Research → report, or data → transform → output. Learn what breaks at the handoffs between steps. Add gate validation.

Month 3: Multi-agent

Once you have several working single-agent automations, introduce a sub-agent. Pick a task where parallelism would help — three research threads running simultaneously instead of sequentially. Watch how much faster it goes.

Month 4+: The system

By now you have enough running pieces that you need to think about the system as a whole. Add monitoring. Add the heartbeat. Build the weekly review cron job that checks what ran, what didn't, and what needs attention. Your automation system is now monitoring itself.

That's roughly 4 months to go from zero to a production automation system. Each week builds on the last. None of the individual steps are complicated — it's the accumulation that creates something powerful.

The Honest Assessment

Here's what most people don't tell you about building automation systems: the first version is always worse than doing it manually. Setting up a morning briefing takes 4 hours; just opening Twitter takes 5 minutes.

The ROI shows up at roughly month 2 for most people. That's when the accumulated time savings from things that just run start to outweigh the ongoing setup and maintenance cost.

Some automations you'll build will break after a month and you'll decide they're not worth fixing. That's fine. You learned something.

Some automations will run flawlessly for years and you'll completely forget they exist until the day they stop and you suddenly realize how much was being done for you.

The goal isn't a perfect system. The goal is a useful one — a system that saves you real time, surfaces real information, and handles the parts of your work that don't require your judgment.

Build it incrementally. Run it in production early. Fix what breaks. Add what helps. Remove what doesn't. That's how all good systems get built.

Staying Current

OpenClaw is actively developed. Capabilities that didn't exist when this book was written probably exist now. Check the official docs and ClawHub regularly for new tools, new skills, and new patterns.

The community is where the freshest patterns live. What other people have figured out — and are willing to share — will save you the same discovery time they spent.

One thing won't change: the fundamentals. Files, tools, context, sessions, agents, cron. The specific API surface will evolve; the underlying concepts are stable. If you understand this book's mental models, you'll be able to adapt to whatever changes.

Good luck. Build something useful.

Acknowledgments and Credits

This handbook draws on the accumulated knowledge of the OpenClaw community. Every pattern documented here was discovered by someone who built something, watched it break, and figured out why.

Particular thanks to the community members who documented their failures as thoroughly as their successes. Failure docs are harder to write and more useful to read.

The humanizer skill patterns in this book (Appendix D as extended) are drawn from the Wikipedia WikiProject AI Cleanup initiative, which has done genuinely useful work cataloging the specific ways AI-generated text differs from human-written text.

The code examples in this book were written to work, not just to illustrate. That means they're occasionally longer than a pure illustration would be — because real code has error handling, real code has logging, real code has comments. The clean version is useless in production. The verbose version is what you'd actually write.

A Note on AI Assistance

Parts of this book were drafted with AI assistance and then edited heavily by human hands. The patterns for detecting that — and removing it — are in Appendix D. If you find places where we missed the mark, the repo is public. Submit a fix.

The irony of using AI to write a book about AI automation is not lost on us.

Technical Credits

- OpenClaw platform by the OpenClaw team
 - Humanizer skill based on Wikipedia:Signs of AI writing (WikiProject AI Cleanup)
 - Shell examples tested on macOS 15 (Darwin) and Ubuntu 24.04
 - Python examples tested on Python 3.12
 - All cron expressions verified against crontab.guru
-

Contact and Contributions

If you found an error, want to add a recipe, or have a pattern worth documenting:

- Open an issue in the OpenClaw repository
- Post in the community forums
- Submit a pull request against the book source

This is a living document. The version you're reading won't be the last.

Index of Key Concepts

Agent — Chapter 2, Chapter 7 **AGENTS.md** — Chapter 2.7, Chapter 7.3 **Batch processing** — Chapter 3.5 **Browser automation** — Chapter 4.4, Chapter 10 **Cron jobs** — Chapter 6, Appendix A (Reference) **Context window** — Chapter 8.1, Chapter 8.6 **Dead man's switch** — Appendix G (Community Patterns) **exec tool** — Chapter 3.3, Appendix G (Extended) **Gateway** — Chapter 2.2 **Heartbeat** — Chapter 6.4, AGENTS.md pattern **Idempotency** — Appendix F (Practitioner's Guide), Appendix G **Memory system** — Chapter 8 **MEMORY.md** — Chapter 2.7, Chapter 8.3 **Nodes** — Chapter 9 **Pipelines** — Chapter 3.6, Chapter 11 **RALPH escalation** — Chapter 7.6, AGENTS.md pattern **Security** — Chapter 15.5, Appendix H **Sessions** — Chapter 2.3 **Skills** — Appendix B **SOUL.md** — Chapter 2.7, Chapter 8.2 **Subagents** — Chapter 7.2, Appendix A **Time zones** — Chapter 6.5 **Tools (reference)** — Appendix A **WhatsApp** — Chapter 5.2 **Workspace** — Chapter 2.5